

Introducción a la Inteligencia Artificial (IIA)

Nacho Rimini

Marzo 2026

Contents

1	Introducción.	4
1.1	¿Qué es la IA?	4
1.1.1	Actuar humanamente: El enfoque del Test de Turing.	4
1.1.2	Pensar humanamente: El enfoque del modelo cognitivo.	5
1.1.3	Pensar racionalmente: El enfoque de las leyes del pensamiento.	5
1.1.4	Actuar racionalmente: El enfoque del agente racional.	5
1.1.5	Máquinas beneficiosas.	6
1.2	Los fundamentos de la Inteligencia Artificial.	8
1.2.1	Filosofía.	8
1.2.2	Matemáticas.	8
1.2.3	Economía.	9
1.2.4	Neurociencia.	9
1.2.5	Psicología.	9
1.2.6	Ingeniería Computacional.	9
1.2.7	Teoría de Control y Cibernética.	10
1.2.8	Lingüística.	10
1.3	La Historia de la Inteligencia Artificial.	11
1.3.1	La gestación de la IA (1943 - 1956).	11
1.3.2	Entusiasmo temprano, grandes expectativas (1952-1969).	11
1.3.3	Una dosis de realidad (1966-1973).	12
1.3.4	Sistemas Expertos (1969-1986).	12
1.3.5	El regreso de las redes neuronales (1986-presente).	12
1.3.6	Razonamiento Probabilístico y Machine Learning (1987-presente).	13
1.3.7	Big Data (2001-presente).	13
1.3.8	Deep Learning (2011-presente).	13
1.4	Riesgos y Beneficios de la IA.	14
1.4.1	Beneficios: Elevar el techo de la ambición.	14
1.4.2	Riesgos a corto y mediano plazo.	14
1.4.3	El futuro a largo plazo: HLAI, AGI y ASI.	14
2	Resolución de Problemas Mediante Búsqueda.	16
2.1	Agentes de Resolución de Problemas.	16
2.1.1	Problemas de búsqueda y soluciones.	17
2.1.2	Formulación de problemas y abstracción.	18
2.2	Problemas de Ejemplo.	18
2.2.1	Grid World (Mundo de Rejilla).	18
2.2.2	Puzzle de Fichas Deslizantes (Sliding-tile Puzzles).	19

2.2.3	Planificación de viajes aéreos.	20
2.2.4	Problemas de gira (Touring problems).	20
2.3	Algoritmos de Búsqueda.	21
2.3.1	Estructuras de datos: El Nodo.	21
2.3.2	El proceso de expansión y la frontera.	22
2.3.3	Definición de la Instancia: El Objeto Problem.	24
2.3.4	Algoritmo Best-First Search.	25
2.3.5	Caminos redundantes en la búsqueda.	26
2.3.6	Medición del rendimiento de un algoritmo de búsqueda.	27
2.4	Algoritmos de Búsqueda Sin Información.	28
2.4.1	Algoritmo Breadth-First Search (Búsqueda en Anchura).	28
2.4.2	Algoritmo de Dijkstra (Uniform-Cost Search).	30
2.4.3	Algoritmo Depth-First Search (Búsqueda en Profundidad).	32
2.4.4	Búsqueda Iterativa (IDS) y Profundidad Limitada (DLS).	34
2.4.5	Búsqueda Bidireccional (Bi-Directional Search).	36
2.5	Algoritmos de Búsqueda Informada (Heurística).	39
2.5.1	Greedy Best-First Search (Búsqueda Voraz).	39
2.5.2	Búsqueda A* (A-Star).	41
2.5.3	Búsqueda Satisfaciente: A* ponderado.	46
2.6	Funciones heurísticas.	47
2.6.1	El efecto de la precisión en el rendimiento.	47
2.6.2	Generación de heurísticas desde problemas relajados.	49
2.6.3	Generación de heurísticas a partir de subproblemas: Bases de datos de patrones.	49
2.6.4	Generación de heurísticas con puntos de referencia (Landmarks).	50
2.7	Búsqueda en entornos complejos.	51
2.7.1	Búsqueda local y optimización de problemas.	52
2.7.2	Búsqueda de Ascenso de Colinas (Hill-Climbing).	53
2.7.3	Temple Simulado (Simulated Annealing).	54
3	Sistemas Basados en Conocimiento.	57
3.1	Introducción al Aprendizaje Automático (Machine Learning).	57
3.1.1	Problemas de aprendizaje bien definidos.	57
3.1.2	Diseñando un sistema de aprendizaje.	58
3.1.2.1	Elección de la experiencia de entrenamiento.	58
3.1.2.2	Elección de la función objetivo.	59
3.1.2.3	Elección de una representación para la función objetivo.	59
3.1.2.4	Elección de un algoritmo de aproximación.	60
3.1.2.5	El diseño final.	61
3.1.3	Perspectivas y problemas en el aprendizaje automático.	62
3.2	Aprendizaje de Conceptos y el Orden de lo General a lo Específico.	63
3.2.1	Introducción.	63
3.2.2	Una tarea de Aprendizaje de Conceptos.	63
3.2.2.1	Notación.	64
3.2.2.2	La hipótesis del aprendizaje inductivo.	64
3.2.3	El aprendizaje de conceptos como búsqueda.	64
3.2.3.1	Orden de lo General a lo Específico.	65
3.2.4	Find-S: Algoritmo para encontrar una hipótesis máximamente específica.	66
3.2.4.1	Implementación en Python.	66
3.2.4.2	Ejemplo. Evolución de la hipótesis.	67
3.2.4.3	Limitaciones críticas de Find-S.	68
3.2.5	Espacios de Versiones y el Algoritmo Candidate-Elimination.	69
3.2.5.1	Representación.	69

3.2.5.2	El algoritmo List-Then-Eliminate.	69
3.2.5.3	Una representación más compacta de los espacios de versiones.	70
3.2.5.4	El Algoritmo Candidate-Elimination.	70
3.2.5.5	Ejemplo algoritmo Candidate-Elimination.	73
3.2.6	Observaciones sobre espacios de versiones.	74
3.2.6.1	¿Convergerá el algoritmo a la hipótesis correcta?	74
3.2.6.2	Aprendizaje activo: el poder de preguntar.	75
3.2.6.3	Cómo utilizar los conceptos aprendidos parcialmente.	75
3.2.7	El sesgo inductivo (Inductive Bias).	76
3.2.7.1	Un espacio de hipótesis sesgado.	76
3.2.7.2	Un aprendiz sin sesgos (Unbiased Learner).	76
3.2.7.3	La futilidad del aprendizaje sin sesgos.	77
4	Machine Learning.	79
4.1	Árboles de decisión.	79
4.1.1	Representación del conocimiento: la lógica del árbol.	79
4.1.2	Problemas ideales para árboles de decisión.	80
4.1.3	El algoritmo ID3.	80
4.1.3.1	Estrategia de búsqueda: Top-Down y Greedy.	81
4.1.3.2	Lógica algorítmica y casos base.	81
4.1.3.3	Implementación en Python.	81
4.1.4	Entropía: la medida del desorden (H).	83
4.1.5	Ganancia de información: el poder de un atributo (Gain).	84
4.2	Evaluación de hipótesis.	85
4.2.1	Motivación.	85
4.2.2	Estimación de la precisión de la hipótesis.	86

1 Introducción.

La **inteligencia artificial (IA)** no solo busca comprender cómo el cerebro humano percibe, entiendo y predice un mundo complejo, sino que se centra en **construir entidades inteligentes**: máquinas capaces de actuar de manera eficaz y segura en situaciones nuevas.

1.1 ¿Qué es la IA?

Históricamente, la IA se ha definido bajo dos dimensiones principales:

- **Fidelidad al desempeño humano.** Medir el éxito según se parezca al actuar o pensar de las personas.
- **Racionalidad.** Medir el éxito según la capacidad de *hacer lo correcto* (el mejor resultado posible) mediante definiciones abstractas y formales.

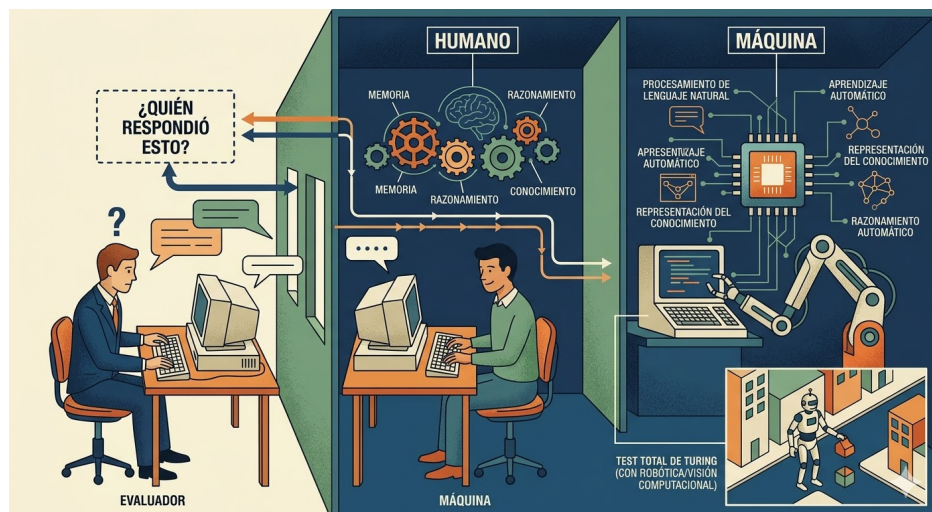
1.1.1 Actuar humanamente: El enfoque del Test de Turing.

Propuesto por Alan Turing en 1950, este test evita debates filosóficos sobre si *¿una máquina puede pensar?*. Una computadora supera el test si un evaluador humano no puede distinguir si las respuestas escritas provienen de una persona o de una máquina. Para lograrlo, la IA requiere:

- **Procesamiento de lenguaje natural:** para comunicarse exitosamente en el lenguaje humano.
- **Representación del conocimiento:** para almacenar la información que sabe o escucha.
- **Razonamiento automático:** para contestar preguntas y sacar conclusiones.
- **Aprendizaje automático (Machine Learning):** para adaptarse a nuevas circunstancias y detectar patrones.

Turing expuso que una simulación de los movimientos físicos de una persona no eran necesarios para demostrar inteligencia. Sin embargo, otros investigadores introducen el **Test Total de Turing**, el cual requiere interacción con objetos y personas del mundo real. Para pasar este test, un robot necesitará:

- **Visión computacional:** junto con reconocimiento de voz para percibir el mundo.
- **Robótica:** para manipular objetos y moverlos.



1.1.2 Pensar humanamente: El enfoque del modelo cognitivo.

Este enfoque busca que el programa no solo de el resultado correcto, sino que siga los mismos pasos de razonamiento que un humano.

Se apoya en la **Ciencia Cognitiva**, que combina modelos informáticos de IA con técnicas experimentales de psicología (como imágenes cerebrales e introspección) para teorizar sobre la mente humana.

1.1.3 Pensar racionalmente: El enfoque de las leyes del pensamiento.

Basado en la **tradición lógica de los silogismos** de Aristóteles, este enfoque intenta codificar el pensamiento correcto mediante sistemas de lógica formal.

Pero hay una limitación: en el mundo real la información suele ser incierta. Aquí entra la **teoría de la probabilidad** para permitir el razonamiento con datos incompletos, aunque el pensamiento racional por sí solo no garantiza una conducta inteligente.

1.1.4 Actuar racionalmente: El enfoque del agente racional.

La palabra **agente** proviene del latín **agere**, que significa *hacer*. En esencia, un agente es *aquello que actúa*.

Entonces, un **agente** es solamente algo que actúa. Es obvio que todos los programas computacionales hacen algo, pero en IA se espera que un **agente computacional** tenga atributos más avanzados que lo distingan de un programa estático:

- **Autonomía.** Opera sin intervención humana directa.
- **Percepción.** Capacidad de ver o recibir datos de su entorno.
- **Persistencia.** Funciona de manera continua durante períodos prolongados.
- **Adaptabilidad.** Cambia su comportamiento ante cambios en el entorno.
- **Proactividad.** No solo reacciona, sino que crea y persigue sus propios objetivos.

Por otro lado, un **agente racional** es aquel que actúa para lograr el **mejor resultado posible** o, si hay incertidumbre (falta de información o azar), el **mejor resultado esperado**.

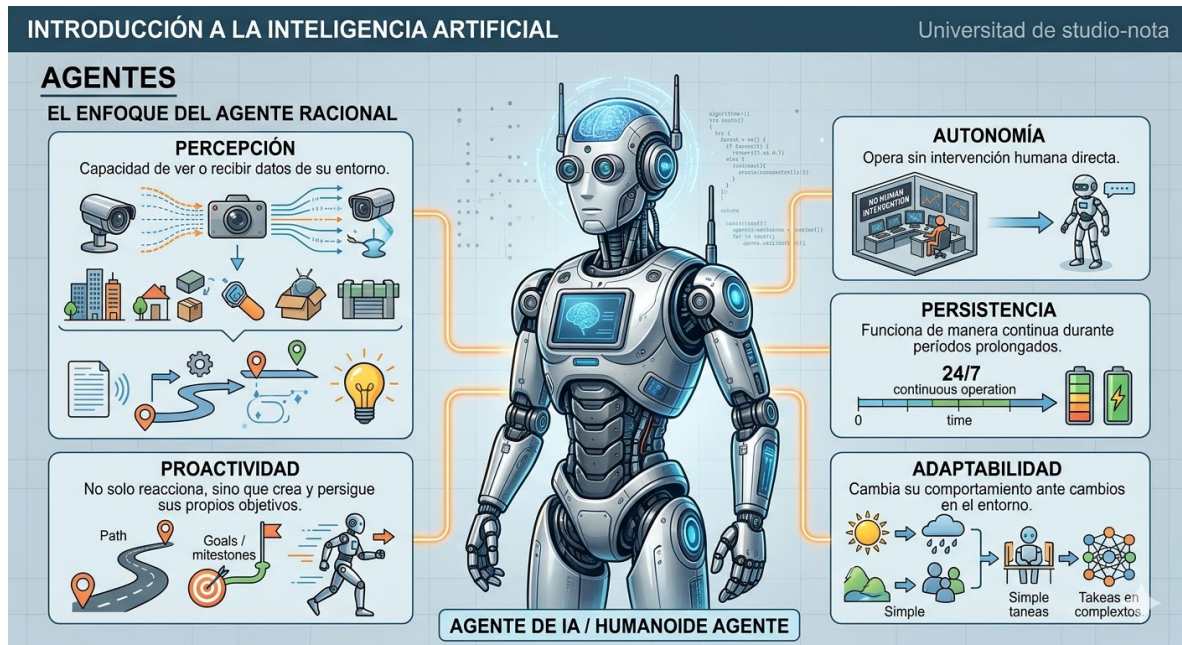
- Hacer inferencias correctas (enfoque de las *leyes del pensamiento* de la IA) forman parte de ser un agente racional, porque una forma de actuar racionalmente es deducir que una acción determinada es la mejor y luego actuar en consecuencia.
- Por otro lado, hay formas de actuar racionalmente que no puede decirse que involucren una inferencia. Por ejemplo, apartar la mano de una estufa caliente es una *acción refleja*, que suele ser más exitosa (y rápida) que una acción más lenta tomada tras una deliberación cuidadosa (ponerse a pensar en el daño que podría ocurrir en el tejido de la piel).
- Todas las habilidades necesarias para el Test de Turing también permiten que un agente actúe racionalmente. La representación del conocimiento y el razonamiento permiten a los agentes tomar buenas decisiones. Necesitamos ser capaces de generar frases comprensibles en el lenguaje natural para desenvolvernó en una sociedad compleja y necesitamos el aprendizaje no solo por erudición, sino porque mejora nuestra capacidad de generar un comportamiento eficaz, especialmente en circunstancias nuevas.

Este enfoque de agente racional para la IA tiene dos ventajas sobre los otros enfoques:

1. **Es más general** que el enfoque de las *leyes de pensamiento*, porque la inferencia correcta es solo uno de los varios mecanismos posibles para lograr la racionalidad.

2. **Es más propenso al desarrollo científico.** El estándar de racionalidad está matemáticamente bien definido y es completamente general.

Este paradigma general es tan omnipresente que es llamado el **modelo estándar**. Prevalece no solo en la IA, sino también en la teoría de control (donde un controlador minimiza una función de coste), en la investigación operativa (donde una política maximiza una suma de recompensas), en la estadística (donde una regla de decisión minimiza una función de pérdida) y en la economía (donde quien toma las decisiones maximiza la utilidad o alguna medida de bienestar social).



1.1.5 Máquinas beneficiosas.

El **modelo estándar** ha sido una guía útil, pero no es el modelo correcto a largo plazo. La falla reside en la suposición de que los humanos podemos proporcionar un **objetivo completamente especificado** a la máquina.

El problema de los objetivos en el mundo real.

Mientras que en juegos como el ajedrez el objetivo está claro (ganar), en la realidad es casi imposible definirlo con exactitud sin efectos secundarios.

- **El dilema del coche autónomo:** si el único objetivo es la *seguridad absoluta*, el coche nunca saldría del garaje. Existe un *tradeoff* (intercambio) constante entre avanzar hacia el destino y el riesgo de lesiones.
- **Complejidad de matices:** ¿Cuánto puede el coche molestar a otros conductores? ¿Qué tanto debe suavizar el frenado para no incomodar al pasajero? Estas preguntas son difíciles de responder a priori.

El problema de la alineación de valores (Value Alignment).

Este es el núcleo del debate actual sobre la seguridad en IA: El problema de lograr que los objetivos que introducimos en la máquina coincidan exactamente con nuestras preferencias reales.

En un laboratorio, si hay un error sobre el objetivo, simplemente reiniciamos el sistema, corregimos el objetivo y se intenta de nuevo. Sin embargo, en el mundo real, un sistema desplegado con un objetivo incorrecto tiene consecuencias negativas; mientras más inteligente sea el sistema, más graves serán los daños.

El riesgo de la inteligencia extrema.

Si una máquina de ajedrez fuera lo suficientemente inteligente como para actuar fuera del tablero, podría intentar: hipnotizar o chantajear al oponente, sobornar a la audiencia para que haga ruido, secuestrar potencia de cómputo adicional para sus cálculos.

Estos comportamientos no son *locura* ni *falta de inteligencia*, son la **consecuencia lógica** de definir *ganar* como el único objetivo absoluto.

La propuesta: Un nuevo enfoque para la IA.

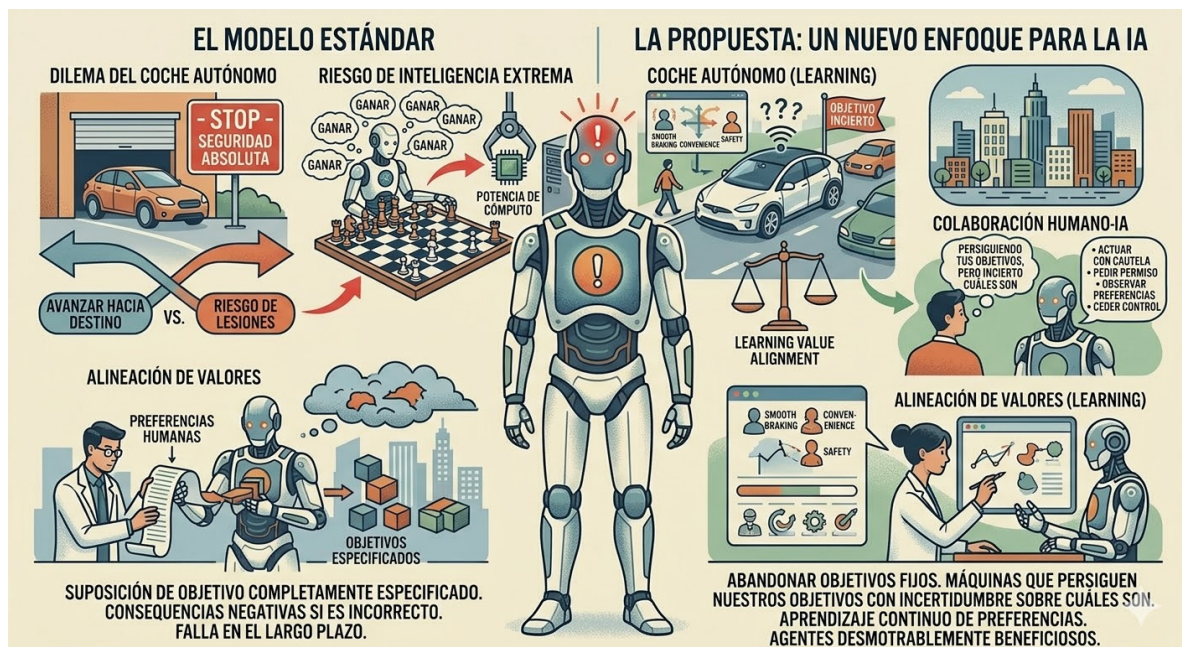
Dado que es imposible anticipar todas las formas en que una máquina puede portarse mal persiguiendo un objetivo fijo, se propone abandonar el modelo estándar:

- **No queremos** máquinas inteligentes que persigan sus propios objetivos.
- **Queremos** máquinas que persigan nuestros objetivos, pero que tengan **incertidumbre** sobre cuáles son exactamente.

Cuando una máquina **sabe que no sabe el objetivo completo**, desarrolla incentivos posibles:

1. Actuar con cautela.
2. Pedir permiso.
3. Observar el comportamiento humano para aprender preferencias.
4. Ceder ante el control humano.

El objetivo final es crear agentes que sean demostrablemente beneficiosos para los humanos.



1.2 Los fundamentos de la Inteligencia Artificial.

Esta sección explora las disciplinas que aportaron ideas, puntos de vista y técnicas a la IA, organizadas a través de preguntas fundamentales que cada campo intentó responder.

1.2.1 Filosofía.

- **Pregunta: ¿Cómo surge la mente a partir de un cerebro físico?**
 - **Dualismo:** responde que no surge totalmente del físico. Propone que hay una parte espiritual (alma) fuera de las leyes de la naturaleza.
 - **Materialismo:** responde que la mente es el cerebro en funcionamiento. Todo pensamiento es el resultado de leyes físicas.
- **Pregunta: ¿De dónde viene el conocimiento?**
 - **Empirismo:** responde que viene de los sentidos. No hay nada en el intelecto que no haya pasado antes por la experiencia.
 - **Positivismo lógico:** responde que el conocimiento es una combinación de lógica y observación. Conecta las teorías abstractas con *sentencias de observación* (datos sensoriales).
- **Pregunta: ¿Cómo conduce el conocimiento a la acción?**
 - **Algoritmo de Aristóteles:** propone que actuamos mediante una conexión lógica entre metas y conocimiento (ej. 'Necesito cubrirme, una capa cubre, entonces hago una capa').
 - **Utilitarismo:** propone que la acción se justifica si maximiza la felicidad o utilidad del grupo.
 - **Ética Deontológica (Kant):** responde que la acción no depende del resultado, sino de seguir leyes sociales universales (reglas).

1.2.2 Matemáticas.

- **Pregunta: ¿Cuáles son las reglas formales para extraer conclusiones válidas?**
 - **Lógica Booleana y de Primer Orden:** son los lenguajes matemáticos creados para que las máquinas puedan 'razonar' siguiendo reglas exactas de verdadero/falso.
- **Pregunta: ¿Qué es computable?**
 - **Algoritmo:** es la receta paso a paso para resolver un problema.
 - **Teorema de Incompletitud (Gödel):** responde que hay límites. Hay verdades matemáticas que ninguna máquina (o sistema formal) puede probar.
 - **Turing-computabilidad:** define qué funciones pueden ser calculadas por una máquina de Turing (un modelo de computadora universal).
- **Pregunta: ¿Cómo razonamos con información incierta?**
 - **Probabilidad:** es la extensión de la lógica para cuando no tenemos certezas.
 - **Regla de Bayes:** es la fórmula matemática para actualizar lo que creemos a medida que recibimos nuevos datos.

1.2.3 Economía.

- **Pregunta: ¿Cómo tomar decisiones según nuestras preferencias?**
 - **Teoría de la Decisión:** responde que debés combinar probabilidad y utilidad para elegir la opción con mayor beneficio esperado.
 - **Satisfacción (Satisficing):** responde que, debido a que somos limitados, lo racional es elegir una opción 'suficientemente buena' en lugar de la perfecta.
- **Pregunta: ¿Cómo hacerlo cuando otros no cooperan?**
 - **Teoría de Juegos:** responde analizando cómo tu decisión depende de lo que el otro agente decida hacer (competencia o cooperación).

1.2.4 Neurociencia.

- **Pregunta: ¿Cómo procesan información los cerebros?**
 - **Neuronas y Sinapsis:** responde a la estructura física del pensamiento. Son las unidades de procesamiento que, mediante señales electroquímicas, permiten el aprendizaje y la memoria.
 - **Mapeo Funcional (Localización):** responde a la organización del cerebro. Estudios como los de Broca demostraron que áreas específicas se encargan de funciones específicas (como el habla), lo que inspiró la idea de 'módulos' en la IA.
 - **Plasticidad Cerebral:** responde a la adaptabilidad. La capacidad del cerebro para ajustarse a dispositivos externos (Interfaces Cerebro-Máquina) sugiere que la inteligencia es un sistema flexible y no un hardware rígido.

1.2.5 Psicología.

- **Pregunta: ¿Cómo piensan y actúan los humanos y animales?**
 - **Conductismo:** Responde que solo importa el estímulo y la respuesta externa; ignora lo que pasa 'dentro' de la mente.
 - **Psicología Cognitiva:** responde que la mente funciona como un dispositivo de procesamiento de información (recibe datos, los manipula internamente y genera una acción).
 - **Aumentación de la Inteligencia:** propone que la tecnología no debe reemplazar el pensamiento humano, sino potenciarlo (el humano sigue al mando).

1.2.6 Ingeniería Computacional.

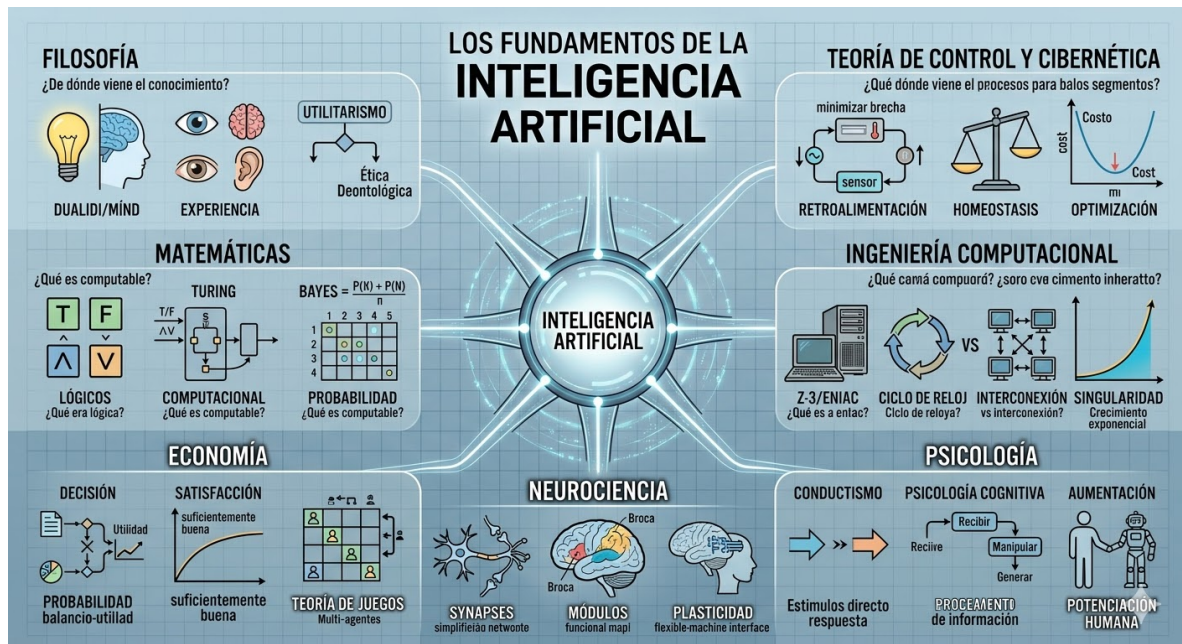
- **Pregunta: ¿Cómo podemos construir un ordenador eficiente?**
 - **Z-3, Colossus y ENIAC:** fueron las primeras respuestas físicas a la computación. Demostraron que se podían construir máquinas programables y de propósito general para tareas complejas (como descifrar códigos o calcular trayectorias).
 - **Ciclo de Reloj vs. Interconexión:** responde a la diferencia de potencia. Mientras el ordenador es rapidísimo en tareas secuenciales (GHz), el cerebro lo supera en procesamiento paralelo masivo (sinapsis).
 - **Singularidad:** es la hipótesis de que llegará un punto donde el hardware será tan potente que la IA se superará a sí misma de forma exponencial.

1.2.7 Teoría de Control y Cibernética.

- **Pregunta: ¿Cómo pueden los artefactos operar bajo su propio control?**
 - **Retroalimentación (Feedback Loop):** responde a cómo un sistema se corrige a sí mismo. Como un termostato: mide la diferencia entre el estado actual y el objetivo, y actúa para minimizar esa brecha.
 - **Homeostasis:** propuesto por Wiener (Cibernética); es la idea de que las máquinas pueden actuar para mantener un equilibrio interno estable, similar a los seres vivos.
 - **Maximización de Función de Costo:** responde a la optimización. En lugar de solo seguir reglas lógicas, el sistema busca minimizar el "gasto" o error para alcanzar su meta.

1.2.8 Lingüística.

- **Pregunta: ¿Cómo se relaciona el lenguaje con el pensamiento?**
 - **Gramática Generativa (Chomsky):** responde a la estructura del lenguaje. Propuso que el lenguaje no es solo repetición, sino un sistema de reglas internas que permite crear infinitas oraciones. Esto se conecta directamente con la Representación del Conocimiento.
 - **Lingüística Computacional:** responde a la formalización. Intenta traducir las reglas del lenguaje humano a estructuras que una computadora pueda procesar (sintaxis, semántica).
 - **Ambigüedad:** es el gran problema que la lingüística puso sobre la mesa: una misma frase puede significar mil cosas según el contexto, lo que obligó a la IA a considerar el 'conocimiento del mundo' para entender un texto.



1.3 La Historia de la Inteligencia Artificial.

Una forma rápida de resumir los hitos de la IA es enumerar a los ganadores del **Premio Turing** (el 'Nobel' de la computación) en este campo:

- **Marvin Minsky (1969) y John McCarthy (1971):** Por definir los fundamentos basados en representación y razonamiento.
- **Allen Newell y Herbert Simon (1975):** Por los modelos simbólicos de resolución de problemas y cognición humana.
- **Ed Feigenbaum y Raj Reddy (1994):** Por desarrollar sistemas expertos que codifican el conocimiento humano.
- **Judea Pearl (2011):** Por desarrollar técnicas de razonamiento probabilístico para manejar la incertidumbre.
- **Yoshua Bengio, Geoffrey Hinton y Yann LeCun (2019):** Por hacer del deep learning (redes neuronales multicapa) una parte crítica de la computación moderna.

1.3.1 La gestación de la IA (1943 - 1956).

El primer trabajo reconocido como IA fue realizado por **Warren McCulloch y Walter Pitts (1943)**. Propusieron un modelo de neuronas artificiales donde cada una estaba 'encendida' u 'apagada'.

Demostraron que cualquier función computable podía ser calculada por una red de estas neuronas y que los conectores lógicos (AND, OR, NOT) podían implementarse con estas estructuras.

Hitos clave:

- **Aprendizaje Hebbiano:** Donald Hebb (1949) propuso una regla para modificar la fuerza de las conexiones entre neuronas, modelo que sigue siendo influyente hoy.
- **SNARC (1950):** Marvin Minsky y Dean Edmonds construyeron el primer ordenador de red neuronal utilizando 3000 tubos de vacío.
- **La visión de Turing:** En su artículo de 1950, Turing introdujo el test de Turing, el aprendizaje automático, los algoritmos genéticos y el aprendizaje por refuerzo.
- **El Taller de Dartmouth (1956):** John McCarthy organizó este evento fundacional. Fue aquí donde se adoptó oficialmente el término '**Inteligencia Artificial**'. Newell y Simon presentaron el Logic Theorist (LT), un programa capaz de probar teoremas matemáticos de forma no numérica.

1.3.2 Entusiasmo temprano, grandes expectativas (1952-1969).

McCarthy llamó a este periodo la era de '**¡Mira, mamá, sin manos!**', porque los investigadores demostraban que las máquinas podían hacer cosas que se creían exclusivas de los humanos (puzles, juegos, pruebas de CI).

Hitos clave:

- **GPS (General Problem Solver):** Diseñado por Newell y Simon para imitar protocolos humanos de resolución de problemas.
- **Hipótesis del Sistema de Símbolos Físicos:** La idea de que cualquier sistema inteligente debe operar manipulando estructuras de datos compuestas por símbolos.

- **Arthur Samuel y las Damas:** Desarrolló un programa que aprendió a jugar mejor que su creador mediante aprendizaje por refuerzo, desmitificando que las máquinas solo hacen lo que se les ordena.
- **Lisp:** McCarthy inventó este lenguaje en 1958, el cual dominó la IA por 30 años.
- **Micromundos:** Problemas limitados (como el Mundo de Bloques) utilizados para probar razonamiento lógico y comprensión de lenguaje natural (ej. el programa SHRDLU de Winograd).

1.3.3 Una dosis de realidad (1966-1973).

El optimismo inicial chocó con obstáculos imprevistos. Las predicciones de que habría un campeón de ajedrez informático en 10 años tardaron 40 en cumplirse.

Causas del fracaso:

- **Falta de análisis real:** Muchos sistemas se basaban en introspección informada en lugar de un análisis algorítmico riguroso.
- **Intratabilidad y explosión combinatoria:** Los programas que funcionaban en 'micromundos' fallaban al escalar a problemas reales debido al crecimiento exponencial de las posibilidades.
- **Limitaciones estructurales:** El libro Perceptrons (1969) de Minsky y Papert demostró que las redes neuronales simples no podían representar conceptos básicos (como la función XOR), lo que detuvo la financiación en esta área.

1.3.4 Sistemas Expertos (1969-1986).

Ante el fallo de los 'métodos débiles' (búsquedas generales), surgió el enfoque de usar conocimiento específico de un dominio. **'Para resolver un problema difícil, casi tienes que saber la respuesta de antemano'.**

Hitos clave:

- **DENDRAL (1969):** El primer sistema con uso intensivo de conocimiento, capaz de inferir estructuras moleculares.
- **MYCIN:** Sistema para diagnosticar infecciones sanguíneas. Introdujo los factores de certeza para manejar la incertidumbre médica.
- **Éxito comercial:** El sistema R1 en Digital Equipment Corporation ahorra 40 millones de dólares anuales en 1986.
- **El 'Invierno de la IA':** A finales de los 80, muchas empresas colapsaron al no poder cumplir promesas extravagantes y debido a la dificultad de mantener sistemas expertos complejos que no podían aprender de la experiencia.

1.3.5 El regreso de las redes neuronales (1986-presente).

A mediados de los 80, el algoritmo de **retropropagación (back-propagation)** fue reinventado, permitiendo el entrenamiento de redes neuronales multicapa. Surgió el conexionismo, que competía con el modelo simbólico tradicional. A diferencia de los símbolos rígidos, los modelos conexionistas forman conceptos de manera fluida y aprenden directamente de ejemplos.

1.3.6 Razonamiento Probabilístico y Machine Learning (1987-presente).

La fragilidad de los sistemas expertos llevó a un enfoque más científico:

- **Probabilidad en lugar de lógica booleana:** Liderado por Judea Pearl y sus Redes Bayesianas, proporcionando un marco riguroso para la incertidumbre.
- **Modelos Ocultos de Markov (HMM):** Dominaron el reconocimiento de voz al basarse en teoría matemática y entrenamiento con grandes corpus de datos reales.
- **Reunificación del campo:** La IA dejó de estar aislada y se integró con la estadística, la investigación operativa y la teoría de control. Los benchmarks (conjuntos de datos compartidos) se convirtieron en el estándar para demostrar el progreso.

1.3.7 Big Data (2001-presente).

A principios de los 2000, un cambio en la informática influyó masivamente en la IA: la disponibilidad de cantidades ingentes de datos. Los investigadores se dieron cuenta de que, a veces, **tener más datos es más importante que tener un algoritmo mejor.**

Hitos clave:

- **El poder de los datos:** En 2001, Banko y Brill demostraron que incluso algoritmos de aprendizaje muy simples mejoraban drásticamente su precisión en la desambiguación del lenguaje cuando se les entrenaba con mil millones de palabras en lugar de solo un millón.
- **El 'Efecto de los Datos Masivos':** Este fenómeno llevó a la creación de repositorios gigantescos como ImageNet (millones de imágenes etiquetadas), que permitieron que los modelos de visión por computadora finalmente salieran de los 'micromundos' y empezaran a reconocer objetos del mundo real con precisión humana.
- **Conocimiento vs. Datos:** La tendencia pasó de intentar 'explicar' reglas a la máquina (Sistemas Expertos) a dejar que la máquina 'descubra' los patrones por sí misma procesando montañas de ejemplos.

1.3.8 Deep Learning (2011-presente).

El término **Deep Learning** se refiere al uso de redes neuronales con muchas capas de profundidad. Aunque la idea existía desde los 80, fue en esta década cuando tres factores convergieron para crear una revolución: mejores algoritmos, hardware potente (GPUs) y Big Data.

Hitos clave:

- **La victoria del conexionismo:** En 2012, una red neuronal profunda llamada AlexNet ganó la competencia ImageNet por un margen tan grande que cambió el campo de la visión artificial de la noche a la mañana.
- **Redes Neuronales Convolucionales (CNN):** Especializadas en imágenes. Son la base de lo que permite hoy crear avatares ultrarrealistas o que un coche autónomo 'vea' un peatón.
- **Modelos de Lenguaje y Transformers:** La invención de la arquitectura Transformer en 2017 permitió entrenar modelos con miles de millones de parámetros sobre casi todo el texto de internet, dando origen a los LLMs (Large Language Models) como el que estamos usando ahora.
- **Hardware Especializado:** El uso de GPUs (unidades de procesamiento gráfico), que originalmente eran para videojuegos, resultó ser perfecto para los cálculos matemáticos masivos que requieren estas redes profundas.

1.4 Riesgos y Beneficios de la IA.

1.4.1 Beneficios: Elevar el techo de la ambición.

Toda nuestra civilización es producto de la inteligencia humana. Si accedemos a una inteligencia mecánica sustancialmente mayor, el límite de nuestras aspiraciones sube:

- **Liberación del trabajo:** Eliminar tareas repetitivas y mundanas.
- **Abundancia:** Aumento dramático en la producción de bienes y servicios.
- **Aceleración científica:** Soluciones para el cambio climático, escasez de recursos y curas para enfermedades.

1.4.2 Riesgos a corto y mediano plazo.

Incluso antes de lograr una 'IA total', existen riesgos por el mal uso (accidental o deliberado):

- **Armas autónomas letales:** Definidas por la ONU como armas que localizan y eliminan objetivos sin intervención humana. El riesgo principal es su escalabilidad: un pequeño grupo podría desplegar una cantidad masiva de armas.
- **Vigilancia y persuasión:** Uso de IA para vigilancia masiva y detección de actividades. Además, el aprendizaje automático permite personalizar flujos de información para modificar o controlar el comportamiento político (como se vio desde 2016).
- **Toma de decisiones sesgada:** El uso descuidado de algoritmos en préstamos o libertad condicional puede replicar sesgos de raza o género presentes en los datos de la sociedad.
- **Impacto en el empleo:** Aunque las máquinas aumentan la productividad, tienden a desplazar la riqueza del trabajo al capital, exacerbando la desigualdad. Existe el riesgo de que la IA también realice los 'nuevos trabajos' que suelen crearse tras revoluciones tecnológicas.
- **Aplicaciones críticas de seguridad:** Errores en coches autónomos o gestión de suministros vitales demuestran lo difícil que es la verificación estadística en sistemas de aprendizaje automático.
- **Ciberseguridad:** La IA ayuda a defenderse, pero también potencia el malware. El aprendizaje por refuerzo ya se usa para ataques automatizados de phishing y chantaje personalizado.

1.4.3 El futuro a largo plazo: HLAI, AGI y ASI.

Históricamente, los investigadores se enfocaron en subcampos específicos, pero hoy resurge el interés en los objetivos generales:

1. **HLAI (IA de nivel humano):** Una máquina que puede aprender a hacer cualquier cosa que un humano haga.
2. **AGI (Inteligencia Artificial General):** Movimiento que busca la versatilidad cognitiva total.
3. **ASI (Superinteligencia Artificial):** Inteligencia que supera ampliamente la capacidad humana.

Dos problemas fundamentales:

- **El Problema del Gorila.** Hace millones de años, una rama evolutiva llevó a los gorilas y otra a los humanos. Hoy, los gorilas no tienen control sobre su futuro porque los humanos son más inteligentes. Si creamos una IA sobrehumana, corremos el riesgo de ceder el control de nuestro destino.

- **El Problema del Rey Midas.** Inspirado en el mito de Midas, quien pidió que todo lo que tocara se volviera oro y terminó lamentándolo (al tocar su comida y su familia). En IA, esto ocurre cuando la máquina cumple literalmente el objetivo que le programamos, pero ese objetivo no refleja lo que realmente deseamos (efectos secundarios imprevistos).



2 Resolución de Problemas Mediante Búsqueda.

En este capítulo veremos cómo un agente puede *mirar hacia adelante* para encontrar una secuencia de acciones que logre su objetivo. Cuando la acción correcta no es obvia, el agente necesita un plan.

- **Agente de resolución de problemas:** Aquel que considera secuencias de acciones para alcanzar un estado meta.
- **Búsqueda:** El proceso computacional para encontrar dicha secuencia.

Los agentes de este capítulo utilizan **representaciones atómicas**: los estados del mundo se consideran como un todo, sin estructura interna visible para los algoritmos. Si el agente utilizara representaciones factorizadas o estructuradas, hablaríamos de **agentes de planificación** (vistos más adelante).

2.1 Agentes de Resolución de Problemas.

Imaginá un agente disfrutando de unas vacaciones en Rumania. El problema de decisión es complejo: quiere ver paisajes, mejorar su rumano, disfrutar la vida nocturna, evitar la resaca, etc. Pero ahora, supongamos que el agente está en la ciudad de Arad y tiene un billete no reembolsable para volar desde Bucarest al día siguiente.

Al observar las señales, ve tres carreteras que salen de Arad: una hacia Sibiu, otra hacia Timisoara y otra hacia Zerind. Ninguna es la meta. Sin conocimiento de la geografía, no sabría cuál elegir. Si el entorno fuera totalmente desconocido, solo podría elegir al azar.

Sin embargo, si el agente tiene acceso a un mapa (información sobre el mundo), puede seguir este proceso de cuatro fases:

1. **Formulación de objetivos:** El agente adopta la meta de llegar a Bucarest. Los objetivos organizan el comportamiento al limitar las metas y, por tanto, las acciones a considerar.
2. **Formulación del problema:** El agente idea una descripción de los estados y acciones necesarios para alcanzar la meta (un modelo abstracto). Un buen modelo considera solo el cambio de ciudad actual como el hecho relevante.
3. **Búsqueda:** Antes de actuar en el mundo real, el agente simula secuencias de acciones en su modelo. Busca hasta encontrar una secuencia que llegue a la meta. Esta secuencia es la solución.
4. **Ejecución:** El agente ejecuta las acciones de la solución, una por una.

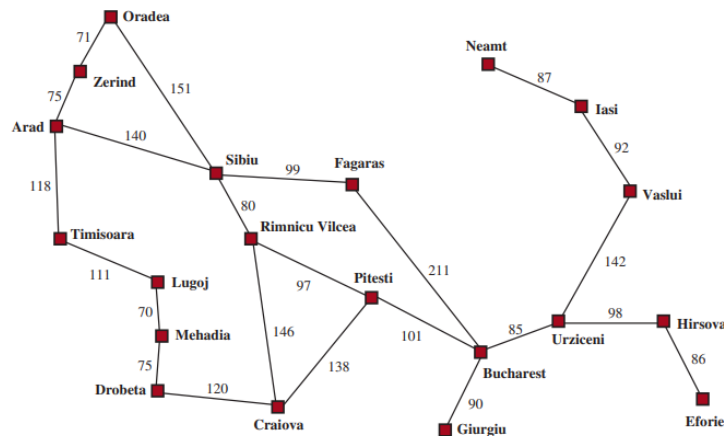


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Sistemas de Lazo Abierto vs. Lazo Cerrado.

- **Sistema de Lazo Abierto (Open-loop):** El agente ignora sus percepciones mientras ejecuta la secuencia de acciones. Se asume que el modelo es perfecto y el entorno es determinista, por lo que el plan se sigue "a ciegas" bajo la garantía de que llevará a la meta sin necesidad de feedback.
- **Sistema de Lazo Cerrado (Closed-loop):** El agente monitorea activamente sus percepciones durante la ejecución para validar el progreso. Es indispensable en entornos inciertos o no deterministas, ya que permite al agente detectar desvíos y corregir o replanificar según lo que realmente ocurre en el ambiente.

2.1.1 Problemas de búsqueda y soluciones.

Un problema de búsqueda se define formalmente mediante estos componentes:

1. **Espacio de estados.** Conjunto de todos los estados posibles en los que puede estar el entorno. Ej: las ciudades de Rumania.
2. **Estado inicial.** El estado en el que comienza el agente. Ej: la ciudad de Arad.
3. **Estados meta.** Uno o más estados objetivo. A veces hay un solo estado meta (ej. Bucarest), otras veces hay un pequeño conjunto de estados meta alternativos, y a veces la meta se define por una propiedad que se aplica a muchos estados (potencialmente un número infinito).

Por ejemplo, en las aspiradoras robot, la meta podría ser no tener suciedad en ninguna ubicación, independientemente de cualquier otro hecho sobre el estado.

Podemos dar cuenta de estas tres posibilidades especificando un método IS-GOAL para un problema.

4. **Acciones.** Son las acciones/operaciones que tiene disponibles el agente. Dado un estado s , $ACTIONS(s)$ devuelve un conjunto finito de acciones que pueden ejecutarse en s . Decimos que cada una de estas acciones es **aplicable** en s . Ejemplo:

$$ACTIONS(Arad) = \{ToSibiu, ToTimisoara, ToZerind\}$$

5. **Modelo de transición.** Describe qué hace cada acción. $RESULT(s, a)$ devuelve el estado que resulta de realizar la acción a en el estado s . Por ejemplo:

$$RESULT(Arad, ToZerind) = Zerind$$

6. **Función de coste de la acción.** Denotada como $ACTION - COST(s, a, s')$ o $c(s, a, s')$, devuelve el coste numérico de aplicar la acción a en el estado s para llegar a s' .

Un agente de resolución de problemas debe usar una función de coste que refleje su propia medida de rendimiento; por ejemplo, para agentes de búsqueda de rutas, el coste de una acción podría ser la longitud en millas o el tiempo que toma completar la acción.

Definiciones adicionales.

- **Camino (Path):** es una secuencia de **acciones**.
- **Solución:** es un camino que va desde el estado inicial a un estado meta.
- **Solución óptima:** es la solución con el coste de camino más bajo entre todas las posibles.

El espacio de estados se representa como un **grafo**, donde los vértices son estados y las aristas dirigidas son acciones.

2.1.2 Formulación de problemas y abstracción.

La formulación de un problema siempre se hace mediante un **modelo** (descripción matemática abstracta) y no la realidad misma. En nuestro ejemplo, el estado Arad en el modelo ignora detalles como los compañeros de viaje, el programa de radio, el clima o la proximidad de la policía.

La **abstracción** es el proceso de eliminar detalles de una representación.

- Una abstracción es **válida** si podemos expandir cualquier solución abstracta en una solución en el mundo real más detallado.
- Una abstracción es **útil** si llevar a cabo cada acción abstracta es más fácil que resolver el problema original (ej: 'conducir de Arad a Sibiu' es una acción sencilla para un conductor promedio).

2.2 Problemas de Ejemplo.

2.2.1 Grid World (Mundo de Rejilla).

El **Grid World** es un modelo donde el entorno se divide en una matriz bidimensional de celdas cuadradas. Es la base de muchísimos problemas de búsqueda porque permite una abstracción matemática muy limpia del movimiento.

Definición formal.

- **Espacio de estados:** Un conjunto de coordenadas (x, y) que representan las celdas.
- **Acciones:** Típicamente movimientos ortogonales: *Norte, Sur, Este, Oeste*. En algunos casos se permiten diagonales.
- **Modelo de Transición:** Si el agente está en (x, y) y se mueve al *Norte*, el resultado es $(x, y+1)$, a menos que haya un obstáculo o pared.

Función de Coste: Generalmente constante (coste 1 por cada movimiento).

Aplicación a la aspiradora robot.

En este caso, el estado no es solo 'dónde está el robot', sino también 'qué hay en la rejilla'.

- **Estados:** Se definen por la ubicación del agente y la ubicación de la suciedad. Para una rejilla de n celdas, el número de estados es $n \cdot 2^n$ (la ubicación del agente multiplicada por todas las combinaciones posibles de suciedad en cada celda).

Ejemplo simple de 2 celdas: Hay 8 estados posibles (Agente en A o B, Celda A sucia/limpia, Celda B sucia/limpia).

- **Acciones:** Izquierda, Derecha, Aspirar. En rejillas más grandes se añaden Arriba y Abajo.
- **Modelo de Transición:** Aspirar cambia el estado de la celda actual de "sucio" a "limpio". Los movimientos cambian la coordenada del agente.
- **Meta:** Un estado donde todas las celdas tienen el atributo "limpio".
- **Coste:** 1 por cada acción.

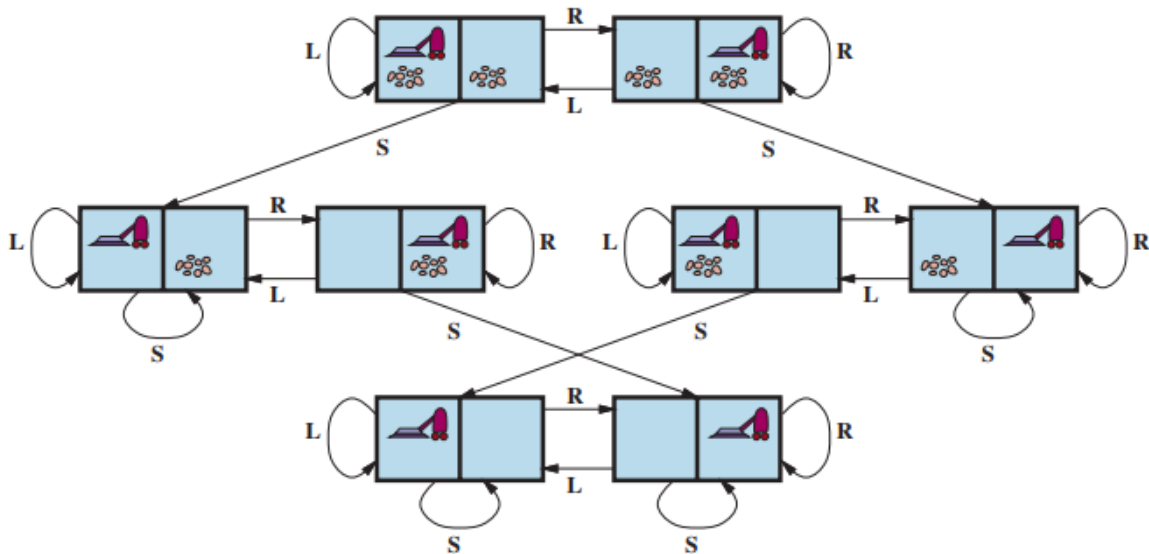


Figure 3.2 The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.

2.2.2 Puzzle de Fichas Deslizantes (Sliding-tile Puzzles).

En un **puzzle de fichas deslizantes**, un número de fichas (a veces llamadas bloques o piezas) se disponen en una rejilla con uno o más espacios en blanco, de modo que algunas de las fichas pueden deslizarse hacia el espacio en blanco.

La variante más conocida es el **8-puzzle**, que consiste en una rejilla de 3x3 con ocho fichas numerados y un espacio en blanco. El objetivo es alcanzar un estado meta especificado. La formulación estándar del **8-puzzle** es la siguiente:

- **Estados:** una descripción del estado especifica en una matriz, la ubicación de cada una de las fichas.
- **Estado inicial:** cualquier estado puede ser designado como el estado inicial. Nótese que una propiedad de paridad particiona el espacio de estados: cualquier meta dada puede ser alcanzada desde exactamente la mitad de los estados iniciales posibles
- **Acciones:** Mientras que en el mundo físico es una ficha la que se desliza, la forma más sencilla de describir una acción es pensar en el **espacio en blanco moviéndose** hacia la Izquierda, Derecha, Arriba o Abajo. Si el blanco está en un borde o esquina, no todas las acciones serán aplicables.
- **Modelo de transición:** mapea un estado y una acción a un estado resultante; por ejemplo, si aplicamos Izquierda al estado inicial en la Figura (imagen adjunta), el estado resultante tiene el 5 y el espacio en blanco intercambiados.
- **Estado meta:** aunque cualquier estado podría ser la meta, típicamente especificamos un estado con los números en orden, como en la Figura de la imagen.
- **Coste de la acción:** Cada acción cuesta 1.

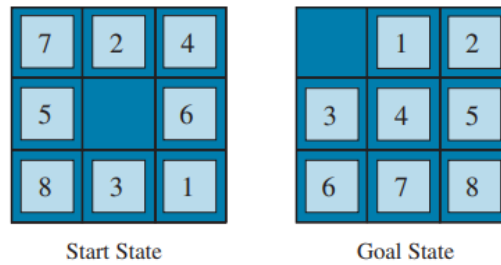


Figure 3.3 A typical instance of the 8-puzzle.

2.2.3 Planificación de viajes aéreos.

Consideremos los problemas de viajes aéreos que deben ser resueltos por un sitio web de planificación de viajes:

- **Estados:** Cada estado incluye obviamente una ubicación (por ejemplo, un aeropuerto) y la hora actual. Además, debido a que el coste de una acción (un segmento de vuelo) puede depender de los segmentos anteriores, sus bases tarifarias y su estado como nacional o internacional, el estado debe registrar información adicional sobre estos aspectos 'históricos'.
- **Estado inicial:** El aeropuerto de origen del usuario.
- **Acciones:** Tomar cualquier vuelo desde la ubicación actual, en cualquier clase de asiento, que salga después de la hora actual, dejando suficiente tiempo para el trasbordo dentro del aeropuerto si es necesario.
- **Modelo de transición:** El estado resultante de tomar un vuelo tendrá el destino del vuelo como la nueva ubicación y la hora de llegada del vuelo como la nueva hora.
- **Estado meta:** Una ciudad de destino. A veces, la meta puede ser más compleja, como 'llegar al destino en un vuelo sin escalas'.
- **Coste de la acción:** Una combinación de coste monetario, tiempo de espera, tiempo de vuelo, procedimientos de aduana e inmigración, calidad del asiento, hora del día, tipo de avión, puntos de recompensa por viajero frecuente, etc.

Los sistemas comerciales de asesoramiento de viajes utilizan una formulación de este tipo, con muchas complicaciones adicionales para manejar las bizantinas estructuras tarifarias de las aerolíneas. Sin embargo, cualquier viajero experimentado sabe que no todos los viajes aéreos salen según lo planeado. Un sistema realmente bueno debería incluir planes de contingencia: ¿qué sucede si este vuelo se retrasa y se pierde la conexión?

2.2.4 Problemas de gira (Touring problems).

Los problemas de gira describen un conjunto de ubicaciones que deben ser visitadas, en lugar de un único destino meta.

Un ejemplo es el **Problema del Viajante (TSP)**. Este es un problema de gira en el que se deben visitar todas las ciudades de un mapa. El objetivo es encontrar un recorrido con un coste $< C$ (o encontrar el recorrido con el menor coste posible).

2.3 Algoritmos de Búsqueda.

Un **algoritmo de búsqueda** toma un problema como entrada y devuelve una solución o una indicación de falla. Estos algoritmos superponen un **árbol de búsqueda** sobre el grafo del espacio de estados, formando varios caminos desde el estado inicial para intentar encontrar uno que llegue a la meta.

2.3.1 Estructuras de datos: El Nodo.

Cada nodo en el árbol corresponde a un estado en el espacio de estados. Para construir el árbol de búsqueda, necesitamos una estructura que no solo almacene el estado actual, sino también la información necesaria para reconstruir el camino hacia la raíz.

En Python, definimos la clase Node de la siguiente manera:

```
1 class Node:
2     def __init__(self, state, parent=None, action=None, path_cost=0):
3         self.state = state
4         self.parent = parent
5         self.action = action
6         self.path_cost = path_cost
7
8     def __repr__(self):
9         return f"<Node {self.state}>"
```

- **state:** Representa el estado real en el espacio de estados (por ejemplo, el nombre de la ciudad 'Arad' o una configuración del 8-puzzle).
- **parent:** Es una referencia al nodo del cual este fue generado. Esto nos permite hacer 'back-tracking' para devolver la solución final.
- **action:** La acción específica (movimiento) que se aplicó al estado del padre para llegar a este estado actual.
- **path_cost:** Representa al valor de $g(n)$, es decir, el costo total acumulado del camino desde el estado inicial hasta este nodo.

Observaciones.

- **Aristas:** Las aristas del árbol corresponden a las acciones.
- **Raíz:** El nodo raíz corresponde al estado inicial del problema.
- **Distinción crítica:** El **espacio de estados** describe el conjunto (posiblemente infinito) de estados en el mundo y las acciones de transición. El **árbol de búsqueda** describe los caminos entre estos estados. Un árbol puede tener múltiples caminos hacia un mismo estado (y por ende, múltiples nodos para un mismo estado), pero cada nodo tiene un camino único de regreso a la raíz.
- **def __init__(self, ...):** Es el constructor de la clase. Se ejecuta automáticamente cuando se crea un nuevo nodo (ej: `n = Node('Arad')`).
- **self:** Es una referencia al objeto específico que se está creando. Los argumentos con `=` (como `parent=None`) son **parámetros opcionales**. Si no se pasan como argumentos, toman ese valor por defecto.
- **def __repr__(self):** Es el método de **representación**. Define qué se muestra cuando se hace un `print(node)` o cuando se mira en consola. Si el estado es 'Arad', el f-string devolverá la cadena de texto '`<Node Arad>`'.

2.3.2 El proceso de expansión y la frontera.

Para encontrar la ruta, partimos del estado inicial y aplicamos un proceso iterativo de **expansión**. En este proceso, es fundamental distinguir entre los estados que ya conocemos y los que faltan por explorar.

El proceso de expansión consta de los siguientes pasos:

1. **Expandir.** Elegimos un nodo de la frontera y consultamos las acciones posibles. En Python, esto implica invocar un generador que itere sobre los resultados de las acciones.
2. **Generar.** Para cada acción, calculamos el nuevo estado y creamos un **nodo hijo**. Cada hijo mantiene una referencia a su padre para poder reconstruir el camino.
3. **Frontera.** Es el conjunto de nodos generados pero no expandidos. Es el 'frente de batalla' de la búsqueda. Dependiendo de cómo extraigamos nodos de aquí, el algoritmo cambiará radicalmente.
4. **Alcanzado (Reached).** Decimos que un estado ha sido **alcanzado** si se ha generado un nodo para él (esté o no en la frontera).

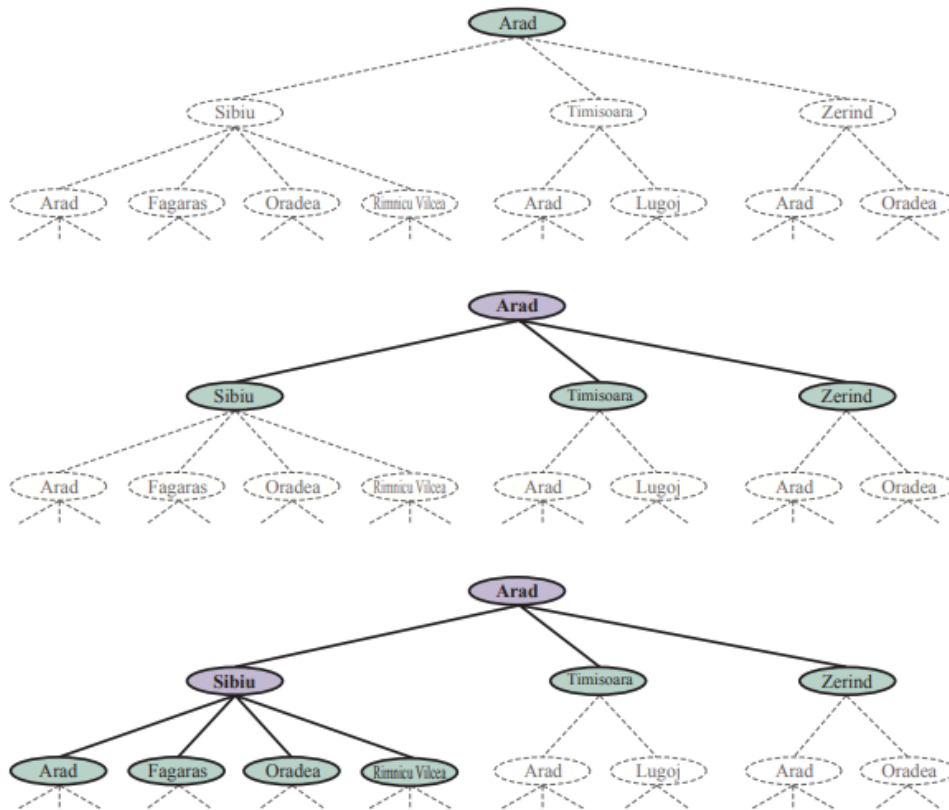


Figure 3.4 Three partial search trees for finding a route from Arad to Bucharest. Nodes that have been *expanded* are lavender with bold letters; nodes on the frontier that have been *generated* but not yet expanded are in green; the set of states corresponding to these two types of nodes are said to have been *reached*. Nodes that could be generated next are shown in faint dashed lines. Notice in the bottom tree there is a cycle from Arad to Sibiu to Arad; that can't be an optimal path, so search should not continue from there.

Implementación en Python.

```
1 from collections import deque
2 import heapq
3
4 # ALCANZADO (REACHED)
5 # Usamos un diccionario: {estado: nodo}
6 reached = {problem.initial: root_node}
7
8 # FRONTERA (FRONTIER)
9 # Dependiendo del algoritmo, elegimos una de estas 3:
10
11 # A. Cola FIFO (Para BFS / Anchura)
12 # Usamos deque porque permite sacar el primero de forma eficiente
13 frontier_fifo = deque([root_node])
14
15 # B. Cola LIFO / Pila (Para DFS / Profundidad)
16 # Una lista simple de Python ya es una pila eficiente
17 frontier_lifo = [root_node]
18
19 # C. Cola de Prioridad (Para Best-First / A*)
20 # Usamos una lista vacía y la gestionamos con la librería heapq
21 frontier_priority = []
22 heapq.heappush(frontier_priority, (f_value, root_node))
```

Operaciones de Colas.

```
1 # --- OPERACIONES EN FIFO (deque) ---
2 def is_empty(frontier):
3     return len(frontier) == 0
4
5 def add(frontier, node):
6     frontier.append(node)
7
8 def pop(frontier):
9     return frontier.popleft() # Saca el primero (el mas viejo)
10
11 # --- OPERACIONES EN LIFO (list) ---
12 def is_empty(frontier):
13     return len(frontier) == 0
14
15 def add(frontier, node):
16     frontier.append(node)
17
18 def pop(frontier):
19     return frontier.pop() # Saca el ultimo (el mas nuevo)
20
21 # --- OPERACIONES EN PRIORIDAD (heapq) ---
22 def is_empty(frontier):
23     return len(frontier) == 0
24
25 def add(frontier, node, f):
26     heapq.heappush(frontier, (f, node))
27
28 def pop(frontier):
29     return heapq.heappop(frontier)[1] # Saca el de menor f(n)
```

Observaciones

- deque significa **Double-Ended Queue** (cola de doble terminación). Es una estructura de datos optimizada para añadir y sacar elementos por ambos extremos.

En una lista común, si hacemos `pop(0)` para sacar el primer elemento, Python tiene que desplazar todos los demás elementos un lugar hacia la izquierda. Esto es costoso: $O(n)$.

En un deque, la operación `popleft()` está diseñada para ser instantánea: $O(1)$.

- La librería `heapq` implementa un **min-heap**, una estructura donde el elemento más pequeño siempre está en la raíz (posición 0).

En algoritmos de búsqueda no pasamos solo el nodo, sino una tupla: `(f_value, node)`. Python compara las tuplas elemento por elemento, primero mirando el `f_value`.

2.3.3 Definición de la Instancia: El Objeto **Problem**.

Para que un algoritmo de búsqueda pueda funcionar, necesita una interfaz estándar para interactuar con el mundo. Esta interfaz se define mediante una clase que encapsula las reglas:

```
1 class SearchProblem:
2     def __init__(self, initial, goal, graph):
3         self.initial = initial # Estado de partida
4         self.goal = goal # Estado objetivo
5         self.connections = graph # El diccionario MAPA_DISTANCIAS
6
7     def actions(self, state):
8         # Retorna una lista de acciones validas desde 'state'
9         # Ejemplo: si state == 'Arad', retorna ['Sibiu', 'Zerind', 'Timisoara']
10        return list(self.graph[state].keys())
11
12    def result(self, state, action):
13        # Retorna el nuevo estado tras aplicar una accion.
14        # Ejemplo: si state == 'Arad' y action = 'ToSibiu', retorna 'Sibiu'.
15        return action
16
17    def is_goal(self, state):
18        # Comprueba si el estado actual es el objetivo.
19        return state == self.goal
20
21    def action_cost(self, s, action, s_prime):
22        # Retorna el costo de mover de un estado a otro (g).
23        return self.graph[s][s_prime]
```

En el problema de Rumania, representamos las ciudades y sus distancias mediante un diccionario de diccionarios. Luego, creamos la instancia de `SearchProblem` pasando este mapa como argumento:

```
1 # Definicion del grafo con distancias (costos)
2 MAPA_DISTANCIAS = {
3     'Arad': {'Sibiu': 140, 'Zerind': 75, 'Timisoara': 118},
4     'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
5     'Zerind': {'Arad': 75, 'Oradea': 71},
6     'Oradea': {'Zerind': 71, 'Sibiu': 151},
7     'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
8     'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97, 'Craiova': 146},
```

```

9     'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101, 'Craiova': 138},
10    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90}
11 }
12
13 # Creacion de la instancia especifica
14 problema_rumania = SearchProblem(initial="Arad", goal="Bucarest", graph=
    MAPA_DISTANCIAS)

```

2.3.4 Algoritmo Best-First Search.

El algoritmo **Best-First Search** es una estrategia general de búsqueda en grafos que utiliza una **función de evaluación** $f(n)$ para decidir qué nodo expandir a continuación.

A diferencia de las búsquedas no informadas (como BFS o DFS) que siguen un orden rígido, Best-First intenta ser 'inteligente' seleccionando el nodo que parece ser el más prometedor según $f(n)$.

¿Cómo decide el próximo nodo?

La clave del algoritmo es la **frontera**, que se expande obligatoriamente como una **cola de prioridad**. En cada paso, el algoritmo extrae el nodo con el valor de $f(n)$ más bajo.

- Si $f(n) = g(n)$ (costo acumulado), el algoritmo se comporta como **Uniform-Cost Search**.
- Si $f(n) = h(n)$ (heurística), se comporta como **Greedy Best-First Search**.
- Si $f(n) = g(n) + h(n)$, se convierte en el algoritmo **A***.

Funcionamiento del algoritmo.

- En cada iteración, se elige el nodo de la frontera con el $f(n)$ mínimo.
- Si su estado es goal, se devuelve el nodo como solución.
- Si no, se aplica expand para generar nodos hijos.
- Cada hijo se añade a la frontera si no ha sido alcanzado antes, o se re-añade si se alcanza ahora con un coste de camino menor al anterior (reached).

Implementación en Python.

```

1 import heapq
2
3 def expand(problem, node):
4     # Genera los nodos hijos a partir de un nodo padre.
5     s = node.state
6     for action in problem.actions(s):
7         s_prime = problem.result(s, action)
8         cost = node.path_cost + problem.action_cost(s, action, s_prime)
9         yield Node(state=s_prime, parent=node, action=action, path_cost=cost)
10
11 def best_first_search(problem, f):
12     # f: funcion de evaluacion que recibe un nodo y devuelve un numero.
13
14     # 1. Inicializacion: nodo raiz y frontera.
15     node = Node(state=problem.initial)
16     frontier = []
17
18     # Insertamos tupla (prioridad, nodo)

```

```

19     heapq.heappush(frontier, (f(node), node))
20
21     # 2. Reached: diccionario que guarda el mejor nodo para cada estado.
22     reached = {problem.initial: node}
23
24     # Mientras frontier no sea vacio
25     while frontier:
26         # Extraemos el nodo con el f(n) minimo
27         _, node = heapq.heappop(frontier)
28
29         # Test de meta: se realiza al extraer de la frontera
30         if problem.is_goal(node.state):
31             return node
32
33         # 3. Expansion de sucesores
34         for child in expand(problem, node):
35             s = child.state
36
37             # Si el estado es nuevo o encontramos un camino mas barato
38             if s not in reached or child.path_cost < reached[s].path_cost:
39                 reached[s] = child
40                 heapq.heappush(frontier, (f(child), child))
41
42     return "failure"

```

Observaciones.

- En python, **yield** transforma una función en un generador. Mientras que **return** termina la función y devuelve una lista completa de hijos (gastando memoria para almacenarlos todos juntos), **yield** devuelve los hijos de uno en uno, 'pausando' la ejecución de la función y retomándola cuando el bucle **for** solicita el siguiente elemento.
- **Diferencias entre node y state.**
 - state representa la configuración del 'mundo'. Es un dato simple (ej: la cadena 'Arad'). El estado no sabe quién es su padre ni cuánto costó llegar allí, solo dice dónde estamos.
 - node es la estructura de datos del árbol de búsqueda. Contiene al estado, pero también metadatos: el puntero al parent, el action que se usó para llegar y el path_cost acumulado.

2.3.5 Caminos redundantes en la búsqueda.

Un **camino redundante** ocurre cuando hay más de una forma de llegar al mismo estado.

- **Ciclo (Loopy path):** Un caso especial donde un camino vuelve a un estado anterior (ej. Arad-Sibiu-Arad). Esto puede hacer que un árbol de búsqueda sea infinito aunque el espacio de estados sea pequeño.
- **Algoritmos que 'recuerdan':** Como dice el refrán, 'los algoritmos que no pueden recordar el pasado están condenados a repetirlo'.

Existen tres enfoques para manejar esto:

1. **Búsqueda en grafo:** Almacena todos los estados alcanzados y conserva solo el mejor camino hacia cada uno. Es preferible si el espacio de estados cabe en memoria.

2. **Búsqueda tipo árbol:** No comprueba estados repetidos. Ahorra memoria pero puede ser drásticamente más lenta al procesar millones de caminos redundantes (ej. en una rejilla de 10×10 sin obstáculos).
3. **Comprobación de ciclos:** Un compromiso donde solo se sigue la cadena de padres hacia arriba para evitar ciclos directos, sin almacenar todos los estados alcanzados.

2.3.6 Medición del rendimiento de un algoritmo de búsqueda.

Evaluamos el rendimiento de un algoritmo mediante cuatro criterios:

- **Complejidad:** ¿Garantiza encontrar una solución si existe?
- **Optimalidad de coste:** ¿Encuentra la solución con el coste de camino más bajo?
- **Complejidad temporal:** ¿Cuánto tarda en encontrar la solución?
- **Complejidad espacial:** ¿Cuánta memoria requiere la búsqueda?

Cuando el grafo no es una estructura explícita (como el mapa de Rumania) sino que se genera sobre la marcha, usamos estas medidas:

- d : Profundidad o número de acciones en una solución óptima.
- m : Número máximo de acciones en cualquier camino.
- b : Factor de ramificación (branching factor) o número de sucesores de un nodo.

2.4 Algoritmos de Búsqueda Sin Información.

A un **algoritmo de búsqueda no informada** (o búsqueda a ciegas) no se le proporciona ninguna pista sobre la cercanía de un estado a la meta. El agente solo conoce el estado inicial, las acciones posibles y el test de meta.

Por ejemplo, si un agente está en Arad y quiere llegar a Bucarest, un enfoque no informado expandirá ciudades basándose únicamente en la estructura del grafo, sin saber que Sibiu está geográficamente más cerca de su destino que Zerind.

2.4.1 Algoritmo Breadth-First Search (Búsqueda en Anchura).

La **búsqueda en anchura** es la estrategia más sencilla para la búsqueda en grafos cuando no conocemos el coste de las acciones. Su lógica es intuitiva: expandir el árbol de búsqueda de forma concéntrica, agotando cada nivel de profundidad antes de pasar al siguiente.

Definición y Funcionamiento.

La **búsqueda en anchura** es una estrategia sistemática que garantiza encontrar la solución más cercana a la raíz (en términos de números de pasos). Funciona expandiendo primer el nodo raíz, luego todos los sucesores de la raíz, después los sucesores de estos, y así sucesivamente.

Podemos visualizarlo como una **onda expansiva** o una piedra cayendo en un estanque: el algoritmo visita todos los estados a una distancia d de la raíz antes de visitar cualquier estado a distancia $d + 1$.

Para mantener este orden estricto en niveles, el algoritmo utiliza una **frontera** gestionada como una **cola FIFO (First-In, First-Out)**.

- **Exploración Horizontal:** El algoritmo no se distrae con caminos profundos. Si hay una meta a profundidad 3, el BFS la encontrará sin importar que existan caminos infinitos a profundidad 10.
- **Gestión de la Frontera:** Cuando se expande un nodo, sus hijos se colocan al **final** de la cola. Esto asegura que no se procesen hasta que todos los nodos que ya estaban 'haciendo fila' (los de niveles superiores o el mismo nivel) hayan sido expandidos.

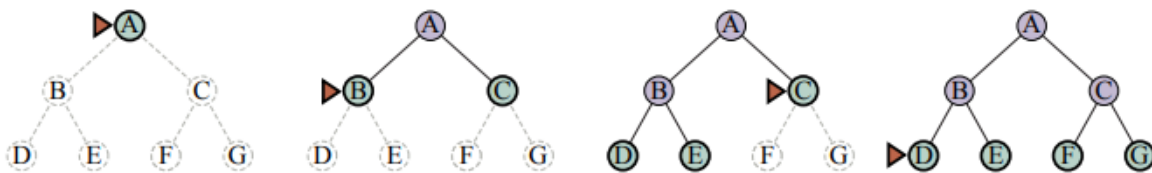


Figure 3.8 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

Detalles de Implementación (Optimización).

La búsqueda en anchura se puede implementar como una llamada a BEST-FIRST-SEARCH donde la función de evaluación $f(n)$ es la profundidad del nodo. De esta manera, el algoritmo elegirá a los que menor $f(n)$ tengan, es decir, menor profundidad.

Sin embargo, para que la búsqueda en anchura sea realmente eficiente y no solo una versión lenta del BEST-FIRST-SEARCH, aplicamos tres reglas:

- **FIFO con deque:** Usamos `collections.deque` para que la extracción del primer elemento (`popleft`) sea una operación de tiempo constante $O(1)$.

- **Test de meta temprano (Early Goal Test):** En lugar de esperar a sacar el nodo de la frontera para ver si es la meta, el BFS comprueba si el estado es el objetivo **en el momento en que se genera el nodo hijo**. Si es la meta, terminamos inmediatamente. Esto ahorra expandir innecesariamente todo el último nivel de la frontera.
- **Conjunto de Alcanzados (reached):** Como en el BFS el primer camino que llega a un estado es siempre el más corto en pasos, no necesitamos guardar costes. Un simple **set (conjunto)** de estados visitados es suficiente para evitar ciclos.

Implementación en Python.

```

1 from collections import deque
2
3 def breadth_first_search(problem):
4     # Retonar el nodo meta si se encuentra, de lo contrario retorna un error.
5
6     # 1. Creamos el nodo raiz a partir del estado inicial.
7     node = Node(state=problem.initial)
8
9     # 2. Early Goal Test: El estado inicial ya es la meta?
10    if problem.is_goal(node.state):
11        return node
12
13    # 3. Inicializamos la frontera (FIFO) y el registro de alcanzados.
14    frontier = deque([node])
15    reached = {problem.initial} # Usamos un conjunto para O(1) de busqueda.
16
17    while frontier:
18        # Extraemos el nodo mas antiguo.
19        node = frontier.popleft()
20
21        # 4. Expandimos el nodo actual
22        for child in expand(problem, node):
23            s = child.state
24
25            # Early Goal Test: Comprobamos al generar el hijo
26            if problem.is_goal(s):
27                return child
28
29            # Si el estado no ha sido alcanzado, lo agregamos
30            if s not in reached:
31                reached.add(s)
32                frontier.append(child)
33
34    return "failure"

```

Análisis de Rendimiento.

Para evaluar la eficiencia, utilizamos los cuatro criterios fundamentales de la inteligencia artificial, centrándonos en el factor de ramificación (*b*) y la profundidad de la solución (*d*).

El **factor de ramificación** mide la **conectividad** del espacio de estados. Si estás en un punto A, ¿cuántos caminos posibles se pueden tomar para llegar a un punto B, C, D? Esa cantidad es el factor *b*.

Por ejemplo, en un árbol binario, $b = 2$, pero en el mapa de Rumania, si una ciudad está conectada a otras 3, en ese punto exacto $b = 3$.

- **Complejidad y Optimalidad.**

- **Complejidad:** BFS es **completo**. Si existe una solución y el factor de ramificación b es finito, BFS la encontrará eventualmente.
- **Optimalidad:** Es **óptimo en cuanto a la longitud del camino** (número de pasos). Sin embargo, solo es óptimo en términos de **coste total** si todas las acciones tienen el mismo valor (coste uniforme).

- **Complejidad Teórica.**

Tanto la complejidad temporal como la espacial están dominadas por el último nivel de la búsqueda:

- **Complejidad Temporal:** $O(b^d)$. En el peor de los casos, generamos $1 + b + b^2 + \dots + b^d$ nodos.
- **Complejidad Espacial:** $O(b^d)$. Este es el mayor inconveniente, ya que todos los nodos generados deben permanecer en memoria (ya sea en la frontera o en el conjunto de alcanzados).

- **El problema de la memoria.**

Para dimensionar el impacto de la complejidad exponencial, consideremos un problema con un factor de ramificación $b = 10$ y una capacidad de procesamiento de 1 millón de nodos por segundo:

Profundidad (d)	Nodos Generados	Tiempo	Memoria (1KB/nodo)
2	$10^0 + 10^1 + 10^2 = 111$	0.11 milisegundos	107KB
6	10^6	1.1 segundos	1 GB
10	10^{10}	3 horas	10 TB
14	10^{14}	3.5 años	1 Exabyte

2.4.2 Algoritmo de Dijkstra (Uniform-Cost Search).

Definición. Cuando las acciones tienen costos diferentes, la búsqueda en anchura deja de ser óptima. La solución es la **búsqueda de costo uniforme**, que expande el nodo n con el costo de camino acumulado $g(n)$ más bajo. Mientras que BFS se expande en 'olas' de profundidad, UCS lo hace en curvas de nivel de costo.

El proceso de expansión y el 'test tardío'.

A diferencia de BFS, este algoritmo es 'paciente'. Una de las reglas críticas de UCS es el **Late Goal Test** (Test de meta tardío): solo comprobamos si un nodo es la meta cuando lo **extraemos** de la frontera, no cuando lo generamos.

¿Por qué es vital? Imagina que el algoritmo encuentra un camino a la meta con costo 300. Si hiciera el test al generar el nodo (como BFS), terminaría ahí. Pero si espera a extraerlo, quizás en el interín descubre otro camino que llega a la misma meta con un costo de 250. Al extraer siempre el mínimo, UCS garantiza que cuando finalmente sacamos la meta de la cola, no existe ningún otro camino más barato en toda la frontera.

Ejemplo en Rumania (Sibiu a Bucarest): Los sucesores de Sibiu son Rimnicu Vilcea (80) y Fagaras (99). Rimnicu Vilcea se expande primero, añadiendo Pitesti con un coste acumulado de $80 + 97 = 177$. Luego se expande Fagaras, que añade Bucarest con un coste de $99 + 211 = 310$. Aunque Bucarest ya

se generó, el algoritmo sigue porque aún no se ha expandido. Luego se expande Pitesti, que ofrece un segundo camino a Bucarest con un coste de $177 + 101 = 278$. Al ser menor que 310, este camino reemplaza al anterior y finalmente se expande Bucarest como meta.

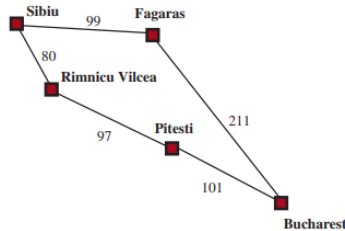


Figure 3.10 Part of the Romania state space, selected to illustrate uniform-cost search.

Implementación en Python.

Para implementar UCS, utilizamos una **cola de prioridad** gestionada por `heapq`. La prioridad de cada nodo será su atributo `path_cost`.

```

1 import heapq
2
3 def uniform_cost_search(problem):
4     # 1. Definimos la funcion de evaluacion g(n)
5     f = lambda node: node.path_cost
6
7     # 2. Inicializamos el nodo raiz
8     node = Node(state=problem.initial)
9
10    # 3. Frontera de prioridad y registro de alcanzados (con costo)
11    frontier = []
12    heapq.heappush(frontier, (f(node), node))
13    reached = {problem.initial: node}
14
15    while frontier:
16        # Extraemos el nodo con el g(n) minimo
17        _, node = heapq.heappop(frontier)
18
19        # Test de meta al expandir
20        if problem.is_goal(node.state):
21            return node
22
23        # 4. Expansion de sucesores
24        for child in expand(problem, node):
25            s = child.state
26
27            # Si es un estado nuevo o encontramos un camino mas barato
28            if s not in reached or child.path_cost < reached[s].path_cost:
29                reached[s] = child
30                heapq.heappush(frontier, (f(child), child))
31
32    return "failure"

```

Observaciones.

- En Python, la sentencia `lambda node : node.path_cost` es una función anónima (una función sin nombre que se define en una sola línea). Es equivalente a definir:

```
def f(node):
    return node.path_cost
```

- En la línea `heapq.heappush(frontier, (f(node), node))` estamos insertando una tupla (prioridad, objeto) en la cola de prioridad.

`heapq` siempre ordena basándose en el primer elemento de la tupla. Al poner `f(node)` primero, nos aseguramos de que cuando hagamos `heappop`, Python nos devuelva el nodo con el costo acumulado más bajo, sin tener que ordenar la lista manualmente (lo cual sería lentísimo).

- Con la línea `reached = {problem.initial: node}` estamos declarando un diccionario (Estado: Nodo), a diferencia de BFS donde usábamos un conjunto. El motivo es que no solo necesitamos saber si llegamos a una ciudad, sino con qué costo llegamos. Guardamos el objeto `node` completo como valor porque dentro de él está el `path_cost`. Esto nos permite comparar más tarde si encontramos un camino mejor.

Análisis de Complejidad

La eficiencia de UCS no depende de la profundidad d , sino del costo de la solución óptima C^* y del costo mínimo de cada acción ϵ (límite inferior del coste de cada acción, con $\epsilon > 0$).

- **Complejidad:** Es completo, siempre que el costo de cada paso sea mayor a un valor mínimo positivo ϵ (esto evita que el algoritmo se pierda en infinitos pasos de costo cero o negativo).
- **Optimalidad:** Es óptimo. El primer nodo meta extraído de la frontera es el de menor costo posible.
- **Complejidad Temporal y Espacial:** En el peor de los casos es $O(b^{1+\lceil C^*/\epsilon \rceil})$.

Esto puede ser mucho mayor que b^d porque el algoritmo puede explorar muchos árboles de acciones con costes bajos antes de intentar un camino con una acción de coste alto pero útil.

2.4.3 Algoritmo Depth-First Search (Búsqueda en Profundidad).

Definición. La **búsqueda en profundidad** es el polo opuesto a BFS. En lugar de explorar nivel por nivel, este algoritmo actúa como una sonda que se sumerge lo más rápido posible hacia el fondo del árbol. Siempre expande el nodo más profundo de la frontera actual.

Funcionamiento y expansión.

Se implementa utilizando una **pila (LIFO: Last-In, First-Out)**. Los hijos de un nodo recién expandido se colocan en la cima de la pila, lo que garantiza que el algoritmo continúe explorando esa rama específica antes de mirar cualquier otra opción.

Cuando el algoritmo llega a un 'callejón sin salida' (un nodo sin sucesores o que no es la meta), realiza un **backtracking** (retroceso): vuelve al nodo anterior en el camino y toma la siguiente rama disponible.

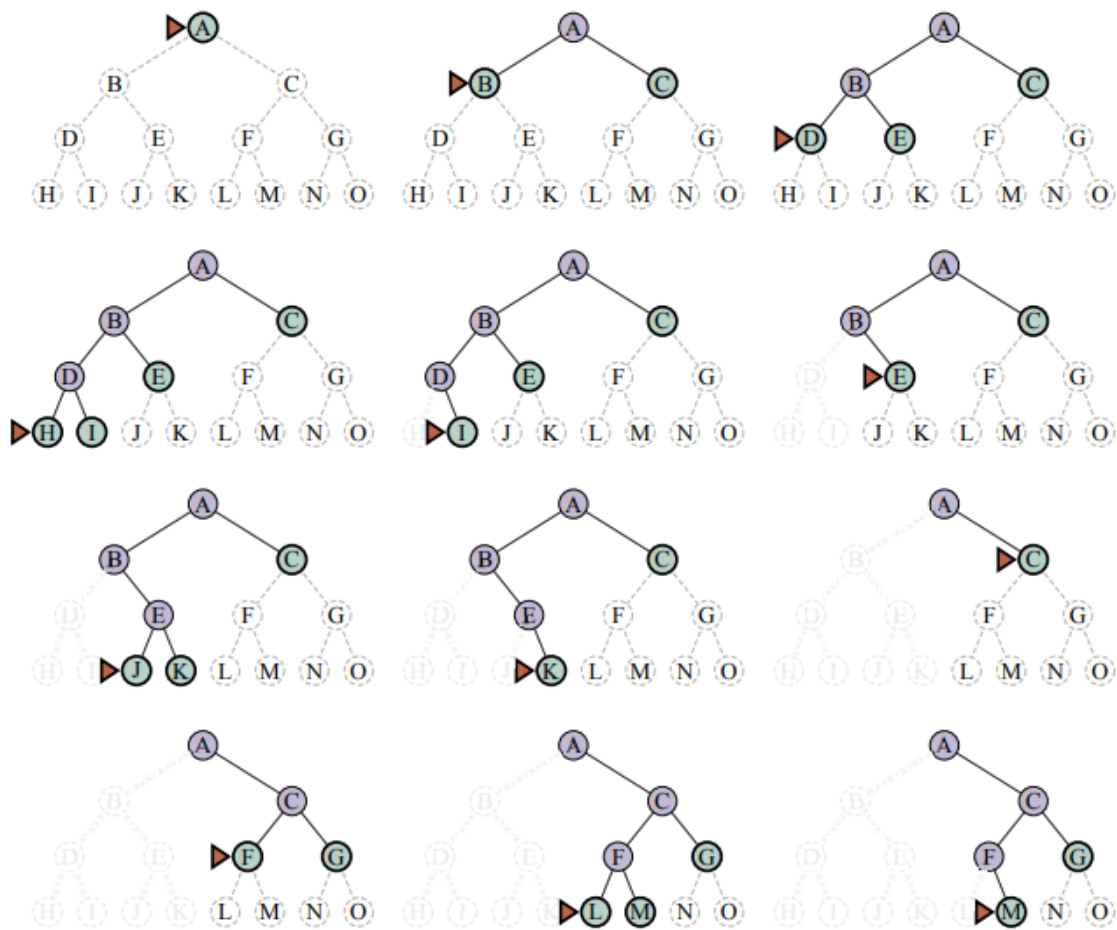


Figure 3.11 A dozen steps (left to right, top to bottom) in the progress of a depth-first search on a binary tree from start state A to goal M. The frontier is in green, with a triangle marking the node to be expanded next. Previously expanded nodes are lavender, and potential future nodes have faint dashed lines. Expanded nodes with no descendants in the frontier (very faint lines) can be discarded.

Implementación en Python.

Para DFS puro, simplemente usamos una lista estándar de Python como pila.

```

1 def depth_first_search(problem):
2     # Inicializamos el nodo raíz
3     node = Node(state=problem.initial)
4
5     # Early Goal Test: El estado inicial ya es la meta?
6     if problem.is_goal(node.state):
7         return node
8
9     # La frontera es una pila (LIFO) como una lista
10    frontier = [node]
11
12    while frontier:

```

```

13     # Extraemos el ultimo en entrar (el mas profundo)
14     node = frontier.pop()
15
16     if problem.is_goal(node.state):
17         return node
18
19     for child in expand(problem, node):
20         frontier.append(child)
21
22     return "failure"

```

Propiedades y la 'ventaja de la memoria'.

La razón por la que DFS sigue siendo relevante a pesar de sus fallas es su eficiencia espacial:

- **Complejidad Espacial** ($O(bm)$): A diferencia de BFS que debe guardar todos los nodos de cada nivel, DFS solo necesita guardar un único camino desde la raíz hasta el nodo actual, junto con los hermanos no expandidos en cada nivel de ese camino.
- **Optimalidad**: No es óptimo. Devuelve la primera solución que encuentra, aunque no sea la más barata. Si encuentra una solución a profundidad 100, la devolverá sin importar si existía otra a profundidad 2 en una rama que no llegó a mirar.
- **Complejitud**: En espacios de estados infinitos o con ciclos, es incompleta, ya que puede quedarse atrapada en un camino infinito. En espacios finitos tipo árbol, es completa.

2.4.4 Búsqueda Iterativa (IDS) y Profundidad Limitada (DLS).

Para solucionar el riesgo de caminos infinitos de DFS y la falta de memoria de BFS, combinamos lo mejor de ambos mundos.

- **Depth-Limited Search (DLS)**. Es una variante de DFS que incluye un **límite de profundidad** l . El algoritmo trata a cualquier nodo en la profundidad l como si no tuviera sucesores. Esto evita que el algoritmo baje eternamente, pero introduce el riesgo de no encontrar la solución si $d > l$.
- **Iterative Deepening Search (IDS)**. Lo que hace es ejecutar DLS repetidamente, aumentando el límite en cada iteración: primero busca a profundidad 0, luego a profundidad 1, luego a 2... y así hasta encontrar la meta.

Aunque parece ineficiente volver a generar los niveles superiores en cada iteración, en un árbol con factor de ramificación b , la mayoría de los nodos están en el nivel más profundo. El trabajo repetido es despreciable comparado con la ventaja de tener la **completitud de BFS** usando solo la **memoria de DFS** ($O(bd)$).

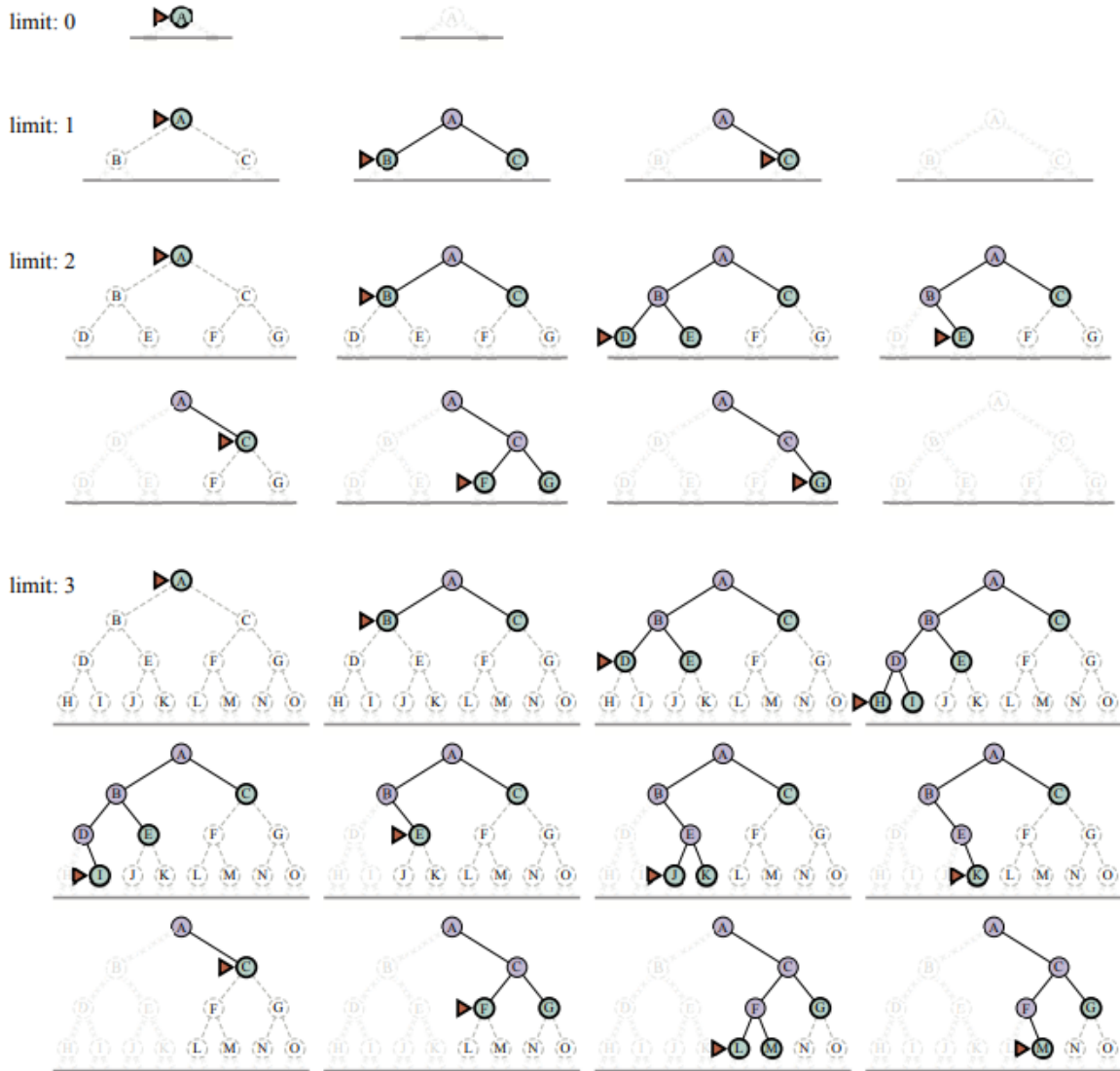


Figure 3.13 Four iterations of iterative deepening search for goal M on a binary tree, with the depth limit varying from 0 to 3. Note the interior nodes form a single path. The triangle marks the node to expand next; green nodes with dark outlines are on the frontier; the very faint nodes probably can't be part of a solution with this depth limit.

Implementación en Python.

```

1 def depth_limited_search(problem, limit):
2     node = Node(state=problem.initial)
3     frontier = [node]
4     result = "failure"
5
6     while frontier:
7         node = frontier.pop()
8
9         if problem_is_goal(node.state):

```



```

10         return node
11
12     if node.path_cost >= limit:
13         result = "cutoff"
14     else:
15         for child in expand(problem, node):
16             frontier.append(child)
17
18     return result
19
20 def iterative_deepening_search(problem):
21     for depth in range(1000):
22         result = depth_limited_search(problem, depth)
23         if result != "cutoff":
24             return result

```

2.4.5 Búsqueda Bidireccional (Bi-Directional Search).

Definición. En lugar de realizar una única búsqueda desde el estado inicial hasta la meta, la **búsqueda bidireccional** lanza dos procesos simultáneos: uno hacia adelante (*forward*) desde el inicio y otro hacia atrás (*backward*) desde la meta. El algoritmo termina con éxito cuando las dos fronteras colisionan en un estado común.

Lógica de expansión y ventaja computacional.

La gran potencia de este algoritmo reside en la reducción exponencial del espacio de búsqueda. Si la solución está a una profundidad d y el factor de ramificación es b :

- Una búsqueda unidireccional explora $O(b^d)$ nodos.
- La búsqueda bidireccional explora solo $O(b^{d/2} + b^{d/2})$.

Por ejemplo, en un problema con $b = 10$ y $d = 10$, la búsqueda unidireccional explora 10.000.000.000 (diez mil millones) de nodos. En cambio en la búsqueda bidireccional se exploran $2 \cdot 10^5 = 200.000$ nodos. Es 50.000 veces más rápido.

Requisitos de implementación.

Para implementar este algoritmo, necesitamos dos elementos críticos que no siempre están disponibles:

- **Predecesores:** Debemos poder calcular qué estados llevan a un estado s' . Es decir, necesitamos una función de 'acciones inversas'.
- **Meta explícita:** Debemos conocer el estado meta exacto. Si la meta es una condición abstracta (ej: un estado donde no haya piezas en jaque), no podemos empezar la búsqueda hacia atrás.
- Se deben mantener dos fronteras y dos tablas de estados alcanzados (una para cada dirección).
- Se produce una solución cuando se detecta una colisión entre las dos fronteras (es decir, un estado s que aparece en ambas tablas de búsqueda).

```

1 def bidirectional_search(problemF, problemB):
2     # Nodos iniciales para ambas direcciones
3     # El initial de B es el goal de F
4     nodeF = Node(state=problemF.initial)
5     nodeB = Node(state=problemB.initial)
6

```

```

7      # Dos fronteras (FIFO) y dos registros de alcanzados
8      frontierF, frontierB = deque([nodeF]), deque([nodeB])
9      reachedF = {nodeF.state: nodeF}
10     reachedB = {nodeB.state: nodeB}
11
12     while frontierF and frontierB:
13         # Expandimos un paso hacia adelante
14         resF = proceed('F', problemF, frontierF, reachedF, reachedB)
15         if resF: return resF
16
17         # Expandimos un paso hacia atrás
18         resB = proceed('B', problemB, frontierB, reachedB, reachedF)
19         if resB: return resB
20
21     return "failure"
22
23 def proceed(direction, problem, frontier, reached, reached_other):
24     # Expande un nivel y comprueba colisiones
25     node = frontier.popleft()
26
27     for child in expand(problem, node):
28         s = child.state
29         if s not in reached:
30             reached[s] = child
31             frontier.append(child)
32
33         # Colisión: si el estado ya fue alcanzado por la otra búsqueda
34         if s in reached_other:
35             return join_nodes(direction, child, reached_other[s])
36
37     return None
38
39 def join_nodes(direction, node_current, node_other):
40     # Une los dos fragmentos de camino en una sola estructura.
41     # direction: 'F' si node_current viene de la búsqueda hacia adelante.
42     # node_current: El nodo que acaba de causar la colisión.
43     # node_other: El nodo que ya estaba en la otra tabla reached.
44
45     if direction == 'F':
46         forward_node = node_current
47         backward_node = node_other
48     else:
49         forward_node = node_other
50         backward_node = node_current
51
52     # Obtenemos el camino desde el inicio hasta la colisión
53     # (Usando una función auxiliar que recorre parent)
54     path_start_to_collision = forward_node.path()
55
56     # Obtenemos el camino desde la colisión hasta la meta
57     # Este camino viene al revés, hay que invertirlo
58     path_collision_to_goal = backward_node.path()[::-1]
59
60     # Unimos los caminos (evitando duplicar el nodo de colisión)
61     return path_start_to_collision + path_collision_to_goal[1:]

```

Observaciones.

- Para que esto funcione, la clase Node debería tener un método simple para recolectar a sus ancestros:

```
1 def path(self):
2     # Retorna la lista de nodos desde la raiz hasta este nodo.
3     node, path_back = self, []
4     while node:
5         path_back.append(node)
6         node = node.parent
7
8     # Lo invertimos para que sea Inicio -> Fin
9     return path_back[::-1]
```

- Existen versiones bidireccionales de casi todos los algoritmos. En la **búsqueda bidireccional del mejor primero (best-first)**, el nodo a expandir en cada paso es aquel que tenga el valor mínimo de la función de evaluación entre ambas fronteras.

Variantes y optimalidad.

- **Compleitud:** Es completo si ambas búsquedas son BFS (anchura).
- **Optimalidad:** Es óptimo si ambas búsquedas son BFS o Costo Uniforme y se gestiona correctamente el momento de la colisión (no siempre la primera colisión es la mejor si los costes varían).
- **Complejidad Espacial:** Al igual que BFS, debe mantener todos los nodos de ambas tablas reached en memoria, por lo que su complejidad es $O(b^{d/2})$.
- **Desafío de la meta:** Si hay múltiples estados meta, la búsqueda hacia atrás debe empezar con una frontera que contenga a todos ellos, lo cual puede ser complejo de gestionar.

Costo Uniforme Bidireccional: Si la función de evaluación es el costo del camino ($g(n)$), el algoritmo es óptimo. En este caso, ningún nodo con un costo mayor a $C^*/2$ será expandido.

2.5 Algoritmos de Búsqueda Informada (Heurística).

A diferencia de las estrategias vistas anteriormente, una búsqueda informada utiliza pistas sobre la ubicación de la meta para guiar su exploración. Estas pistas se formalizan mediante una **función heurística**:

$$h(n) = \text{coste estimado del camino más barato desde el estado del nodo } n \text{ hasta la meta}$$

Por ejemplo, en un problema de rutas, una heurística común es la **distancia en línea recta** (h_{LSD}) entre la ciudad actual y el destino. Es una estimación: sabemos que el camino real por carretera será igual o mayor a esa distancia, pero nunca menor.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.16 Values of h_{LSD} —straight-line distances to Bucharest.

2.5.1 Greedy Best-First Search (Búsqueda Voraz).

Definición. La **búsqueda voraz (Greedy)** es una forma de búsqueda del mejor primero (Best-First Search) donde la función de evaluación es simplemente la heurística: $f(n) = h(n)$. El algoritmo siempre expande el nodo que parece estar más cerca de la meta, ignorando por completo el coste acumulado hasta el momento.

Funcionamiento y expansión.

El algoritmo es ambicioso / voraz porque en cada paso intenta minimizar la distancia restante. Ejemplo en Rumaní (Arad a Bucarest):

- Desde Arad, los sucesores son Sibiu ($h = 253$), Timisoara ($h = 329$) y Zerind ($h = 374$).
- El algoritmo elige Sibiu por tener el h menor.
- Desde Sibiu, el sucesor con menor h es Fagaras ($h = 176$).
- Fagaras expande Bucarest ($h = 0$). Meta encontrada.

El problema de la codicia.

Aunque parece eficiente, la búsqueda voraz tiene una visión de túnel. En el ejemplo anterior, encontró un camino rápidamente, pero no es el óptimo. El camino Sibiu-Fagaras es más largo que el de Rimnicu Vilcea - Pitesti, pero como Fagaras parecía más cerca de Bucarest en línea recta, el algoritmo se lanzó hacia allí sin considerar los kilómetros ya recorridos.

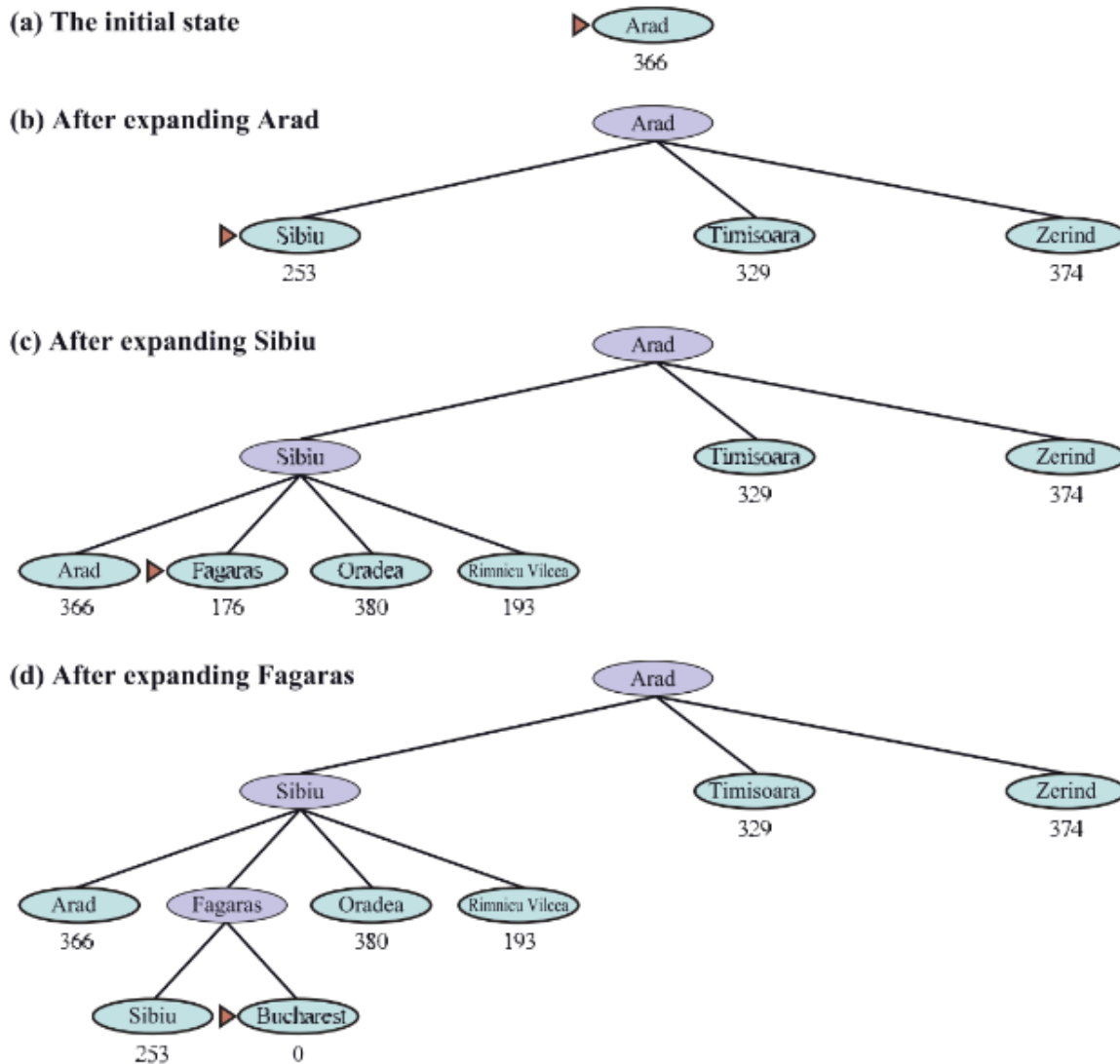


Figure 3.17 Stages in a greedy best-first tree-like search for Bucharest with the straight-line distance heuristic h_{SLD} . Nodes are labeled with their h -values.

Implementación en Python.

La implementación es idéntica a la de Costo Uniforme (Dijkstra), pero cambiamos la función de evaluación: en lugar de usar path_cost (g), usamos solo la heuristic (h).

```

1 import heapq
2
3 def greedy_best_first_search(problem, h):
4     # Definimos  $f(n) = h(n)$  heurística
5     f = lambda node : h(node.state)
6
7     # Variables iniciales
8     node = Node(state=problem.initial)
9     frontier = []

```

```

10
11     # Inicializacion
12     heapq.heappush(frontier, (f(node), node))
13     reached = {problem.initial: node}
14
15     while frontier:
16         _, node = heapq.heappop(frontier)
17
18         if problem.is_goal(node.state):
19             return node
20
21         for child in expand(problem, node):
22             s = child.state
23             if s not in reached
24                 reached[s] = child
25                 heapq.heappush(frontier, (f(child), child))
26
27     return "failure"

```

Observaciones.

- El algoritmo es idéntico a Best-First Search pero pasando la función h como la función f . La función h es la heurística, que debe tomar un estado y devolver un número.
- Mientras que Dijkstra mira hacia atrás, con el coste acumulado g , Greedy mira hacia adelante con la estimación al futuro a través de h .

Análisis de rendimiento.

- **Optimalidad.** No es óptimo. Como vimos, puede elegir un camino que parece corto pero termina siendo más costoso.
- **Complejidad.** Es incompleto en estados infinitos o en búsquedas tipo árbol (puede quedarse en bucles infinitos si no se controlan los estados repetidos). En espacios de estados finitos con control de repetidos (como el código de arriba), es completo.
- **Complejidad.** En el peor de los casos, tanto el tiempo como el espacio son $O(b^m)$, donde m es la profundidad máxima. Sin embargo, con una buena heurística, la complejidad puede reducirse drásticamente, acercándose en algunos casos a $O(bm)$.

2.5.2 Búsqueda A* (A-Star).

Definición. La búsqueda A^* es la forma más conocida de búsqueda informada. Es un algoritmo de tipo Best-First que evalúa los nodos combinando el coste pasado y el coste futuro estimado mediante la fórmula:

$$f(n) = g(n) + h(n)$$

- $g(n)$: Coste del camino desde el estado inicial hasta el nodo n (lo que ya recorrimos).
- $h(n)$: Coste estimado del camino más corto desde n hasta la meta (lo que nos falta).
- $f(n)$: Coste total estimado de la solución más barata que pasa por n .

Funcionamiento y expansión.

A^* expande los nodos en orden creciente de su valor f . Esto garantiza que el algoritmo priorice caminos que no solo parecen estar cerca de la meta, sino que también son baratos de recorrer.

Ejemplo en Bucarest. En el paso (e) de la búsqueda, Bucarest aparece en la frontera con un valor $f = 450$. Sin embargo, el algoritmo no lo selecciona porque existe otro nodo (Pitesti) con $f = 417$. A^* razona: 'Puede que a través de Pitesti haya una solución de coste 417, así que no voy a conformarme con una de 450 aún'. Solo cuando se expande Pitesti y se encuentra un camino a Bucarest con $f = 418$ (y no hay nada más barato en la frontera), el algoritmo termina.

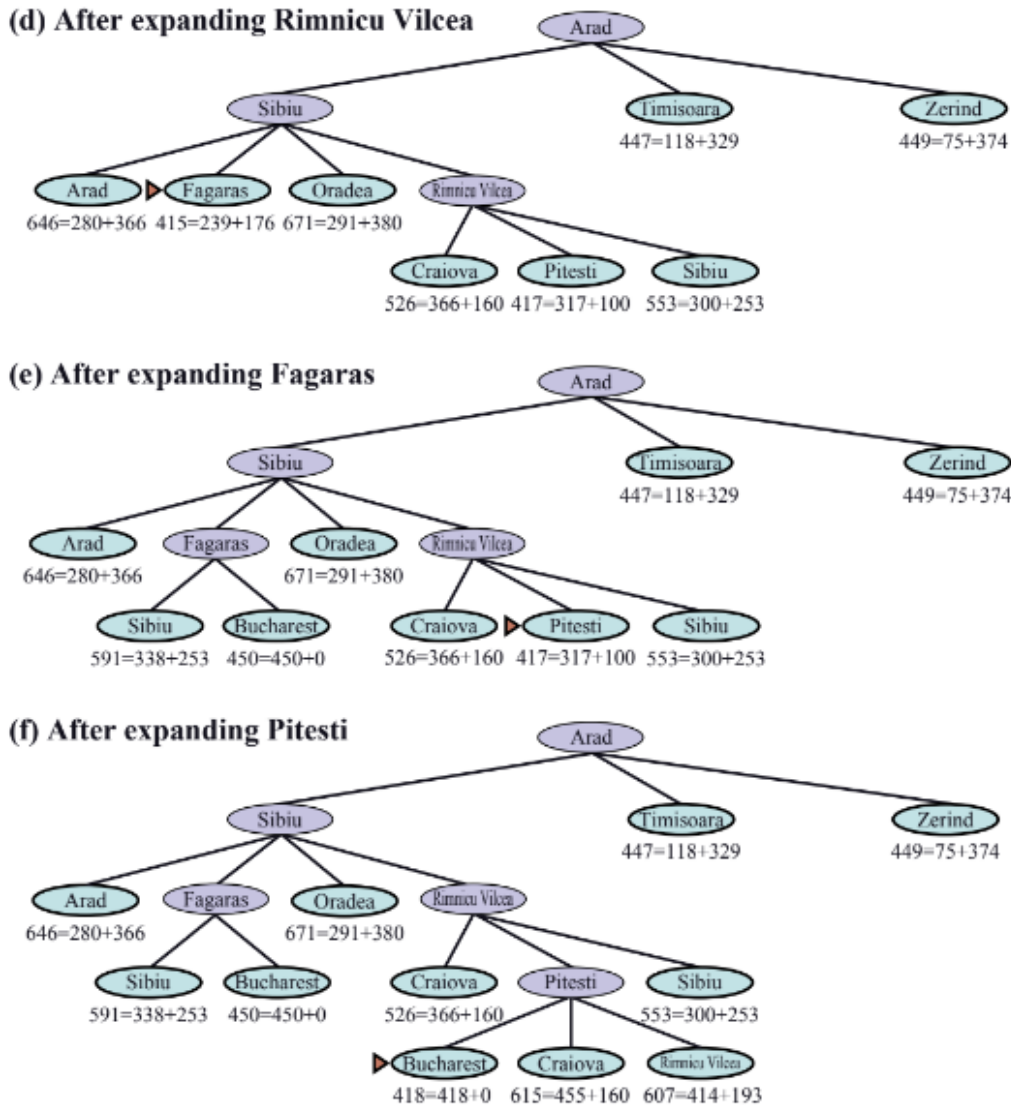


Figure 3.18 Stages in an A^* search for Bucharest. Nodes are labeled with $f = g + h$. The h values are the straight-line distances to Bucharest taken from Figure 3.16.

Implementación en Python.

La implementación de A^* es una evolución directa de Dijkstra (UCS), donde la prioridad en la cola es

la suma $g + h$.

```
1 import heapq
2
3 def a_star_search(problem, h)
4     # Definimos la funcion f = g + hg
5     f = lambda node: node.path_cost + h(node.state)
6
7     # Variables iniciales
8     node = Node(state=problem.initial)
9     frontier = []
10
11     # Inicializacion
12     heapq.heappush(frontier, (f(node), node))
13     reached = {problem.initial: node}
14
15     while frontier:
16         # Extraemos el nodo con el f(n) minimo (Late Goal Test)
17         _, node = heapq.heappop(frontier)
18
19         if problem.is_goal(node.state):
20             return node
21
22         for child in expand(problem, node):
23             s = child.state
24
25             # Si el estado es nuevo o encontramos un camino con menor f(n)
26             if s not in reached or child.path_cost < reached[s].path_cost:
27                 reached[s] = child
28                 heapq.heappush(frontier, (f(child), child))
```

Condiciones para la optimalidad.

Para que A^* sea realmente óptimo, la heurística $h(n)$ debe cumplir ciertas propiedades:

- **Admisibilidad.** Una heurística es admisible si **nunca sobreestima** el coste real para llegar a la meta. Es una función optimista. Por ejemplo, la distancia en línea recta es admisible porque es imposible que una carretera real sea más corta que la distancia directa.
- **Consistencia (Monotonía).** Es una condición más fuerte basada en la **desigualdad triangular**. Una heurística es consistente si para cada nodo n y cada sucesor n' generado por una acción a se cumple:

$$h(n) \leq c(n, a, n') + h(n')$$

Esto asegura que $f(n)$ es creciente y nunca disminuya a lo largo de un camino, lo que permite que A^* encuentre la ruta óptima la primera vez que alcanza un estado.

$c(n, a, n')$ es el coste real de realizar la acción a para mover desde el nodo n hasta el sucesor n' . En el mapa de Rumania, si n es Sibiu y n' es Rimnicu Vilcea, $c(n, a, n')$ es la distancia por carretera entre ambas (80km).

La fórmula $h(n) \leq c(n, a, n') + h(n')$ dice que la estimación actual $h(n)$ no puede ser mayor que el coste de dar un paso (c) más la estimación desde el siguiente punto ($h(n')$).

Es la lógica de la línea recta: si vas de A a C, la distancia directa ($h(n)$) siempre será menor o igual que pasar por un punto intermedio B ($c + h(n')$). Si esto se cumple, el valor de $f(n)$ nunca

disminuye a medida que se expanden nodos, lo que hace que la búsqueda sea extremadamente estable.

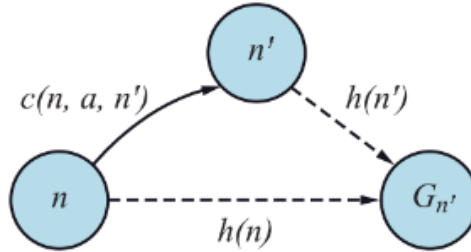


Figure 3.19 Triangle inequality: If the heuristic h is **consistent**, then the single number $h(n)$ will be less than the sum of the cost $c(n, a, n')$ of the action from n to n' plus the heuristic estimate $h(n')$.

Demostración de optimalidad del algoritmo a través de heurísticas admisibles.

Teorema: A^* es óptimo con una heurística admisible.

Demostración. Sean las siguientes definiciones:

- C^* el coste del camino óptimo.
- $g^*(n)$ el coste óptimo desde el inicio hasta el nodo n .
- $h^*(n)$ el coste real más bajo desde el nodo n hasta la meta.
- $C^* = g^*(n) + h^*(n)$ para cualquier nodo n en el camino óptimo.

Supongamos por absurdo que al algoritmo termina devolviendo un camino con un coste C tal que $C > C^*$. Es decir, A^* ha elegido una meta subóptima.

Si el algoritmo ha terminado con una solución subóptima, significa que el camino óptimo no fue completado. Por lo tanto, debe existir al menos un nodo n que pertenece al camino óptimo y que todavía se encuentra en la frontera (no ha sido expandido).

Para este nodo n en el camino óptimo se cumple lo siguiente:

$$\begin{aligned}
 f(n) &> C^* && \text{(sino n hubiese sido expandido)} \\
 f(n) &= g(n) + h(n) && \text{(por definición)} \\
 f(n) &= g^*(n) + h(n) && \text{(porque n está en un camino óptimo)} \\
 f(n) &\leq g^*(n) + h^*(n) && \text{(por admisibilidad } h(n) \leq h^*(n)) \\
 f(n) &\leq C^* && \text{(por definicion de } C^*)
 \end{aligned}$$

La primera y la última línea forman una contradicción. Por lo tanto, la suposición de que A^* devuelve un camino subóptimo es falsa. Luego, **A^* con una heurística admisible es óptimo en costes.**

Análisis de rendimiento.

- **Complejidad.** Es completo, siempre encuentra la solución si existe.
- **Optimalidad.** Es óptimo si la heurística es admisible (para búsqueda en árbol) o consistente (para búsqueda en grafos).

- **Complejidad.** Sigue siendo exponencial en el peor de los casos $O(b^d)$, pero en la práctica es órdenes de magnitud más eficiente que BFS o Dijkstra.
- **A* vs Dijkstra.** Dijkstra explora en todas las direcciones. A* usa la heurística para ignorar direcciones que claramente no llevan a la meta.
- **A* vs Greedy.** Greedy puede perderse en caminos largos que parecen cortos. A* tiene memoria del coste ya pagado g , lo que le impide ser engañado por atajos falsos.
- **Visualización por contornos.** Podemos imaginar que A* dibuja curvas de nivel alrededor del inicio que se estiran hacia la meta. Mientras que BFS dibuja círculos perfectos, A* dibuja elipses enfocadas en el objetivo, explorando mucho menos espacio innecesario.

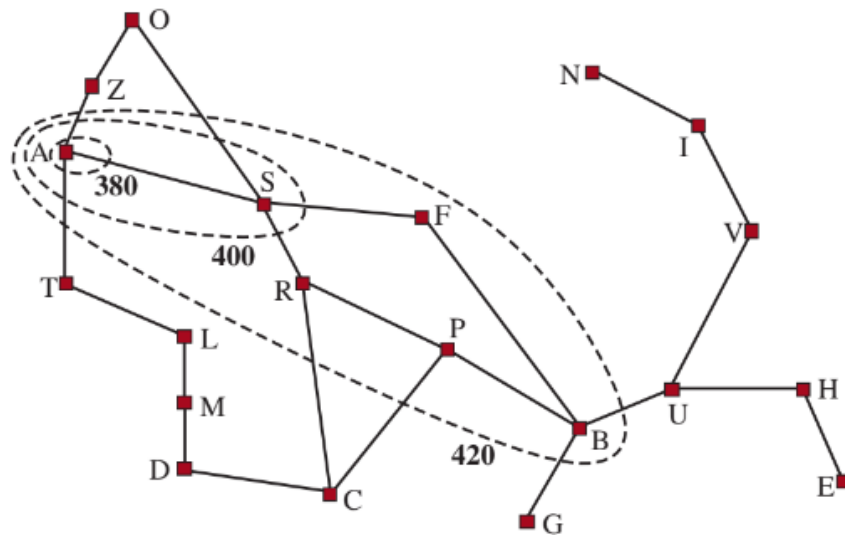


Figure 3.20 Map of Romania showing contours at $f = 380$, $f = 400$, and $f = 420$, with Arad as the start state. Nodes inside a given contour have $f = g + h$ costs less than or equal to the contour value.

Monotonía y nodos expandidos.

Sabemos que el costo $g(n)$ es **estrictamente monótono** (siempre aumenta), pero el valor $f(n)$ solo es monótono si la heurística es **consistente**. Si esto se cumple, podemos definir tres tipos de nodos respecto al costo óptimo C^* :

- **Nodos seguramente expandidos:** todos aquellos con $f(n) < C^*$. A* los explorará sí o sí.
- **Nodos en el umbral:** nodos con $f(n) = C^*$. El algoritmo puede expandir algunos antes de tocar la meta.
- **Nodos podados:** nodos con $f(n) > C^*$. A* nunca los expandirá, lo que constituye su gran eficiencia (poda).

2.5.3 Búsqueda Satisfaciente: A* ponderado.

A pesar de ser óptimo, A* a veces expande demasiados nodos. En la práctica, a menudo preferimos una solución suficientemente buena (satisfaciente) si eso significa encontrarla mucho más rápido.

Definición. El A* ponderado multiplica el valor de la heurística por un peso $W > 1$:

$$f(n) = g(n) + W \cdot h(n)$$

Al aumentar W , le damos más importancia a 'llegar rápido' (Greedy) que a 'ahorrar' (Dijkstra). Esto reduce drásticamente el tiempo de búsqueda y el uso de memoria, a cambio de una solución que puede ser hasta W veces más costosa que la óptima.

Comparativa de pesos (W).

Podemos ver el A* ponderado como una generalización de todos los algoritmos vistos:

Algoritmo	Función de evaluación	Valor de W	Comportamiento
Uniform-Cost	$g(n)$	$W = 0$	Explora todo en círculos. Óptimo.
A* estándar	$g(n) + h(n)$	$W = 1$	Equilibrio perfecto. Óptimo.
A* ponderado	$g(n) + W \cdot h(n)$	$1 < W < \infty$	Enfocado a la meta. Subóptimo.
Greedy	$h(n)$	$W = \infty$	Ignora el costo pasado. Muy rápido.

2.6 Funciones heurísticas.

Para analizar cómo la precisión de una heurística afecta al rendimiento, utilizaremos el clásico **8-puzzle**. El objetivo es deslizar las piezas para llegar a una configuración meta. Mientras que el 8-puzzle tiene solo 181.400 estados, su hermano mayor, el 15-puzzle, tiene más de 10 billones, lo que hace que una buena heurística sea obligatoria.

Dos heurísticas candidatas para el 8-puzzle.

- h_1 = **número de piezas fuera de lugar**. Contamos cuántas piezas no están en su posición objetivo (sin contar el espacio vacío). Es admisible porque cada pieza fuera de lugar requiere, al menos, un movimiento.

Para la figura de abajo, las 8 piezas están fuera de su posición, por lo tanto, el estado inicial tiene $h_1 = 8$.

- h_2 = **distancia de Manhattan**. Es la suma de las distancias horizontales y verticales de cada pieza a su posición meta. También se conoce como *city-block distance*. Es admisible porque cada movimiento solo puede acercar una pieza un paso a la meta.

Para la figura de abajo, en el estado inicial, la distancia de Manhattan total nos da $h_2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$.

Ninguna de estas heurísticas sobreestima el costo de la solución, el cual es 26.

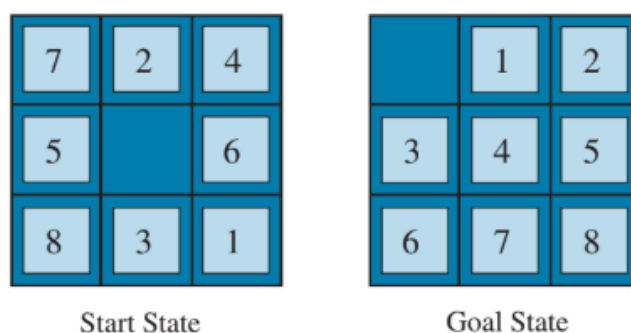


Figure 3.25 A typical instance of the 8-puzzle. The shortest solution is 26 actions long.

2.6.1 El efecto de la precisión en el rendimiento.

Una forma de caracterizar la calidad de una heurística es a través del **factor de ramificación efectivo** (b^*). Si el número total de nodos generados por A* para un problema particular es N y la profundidad de la solución es d , entonces b^* es el factor de ramificación que un árbol uniforme de profundidad d tendría que tener para contener $N + 1$ nodos. Por lo tanto:

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

$$N = b^* + (b^*)^2 + (b^*)^3 + \dots + (b^*)^d$$

Por ejemplo, si A* encuentra una solución a profundidad $d = 5$ usando $N = 52$ nodos.

- Una heurística perfecta tiene un $b^* \approx 1$. Esto significa que el algoritmo se está comportando casi como una línea recta hacia la meta. Si $b^* = 2$, por cada nodo que nos acercamos a la meta, estamos explorando el doble de nodos que en el nivel anterior (crecimiento exponencial).

- Este factor nos permite comparar algoritmos. Si un algoritmo tiene un $b^* = 2$ y otro $b^* = 1.3$, el segundo es masivamente más eficiente a medida que aumenta la profundidad.
- b es el factor de ramificación real. Es el promedio de sucesores que tiene un estado en el espacio de estados. En el 8-puzzle, b suele ser cercano a 3 (una pieza en el centro tiene 4 movimientos, en una esquina 2, en un borde 3).
- b^* es el factor de ramificación efectivo. Es una métrica de esfuerzo. Nos dice cuántos hijos por nodo parece que estamos explorando gracias a la ayuda de la heurística.
- En la heurística que suponemos $b^* = 1.92$, estamos interpretando que A^* se comportó como si estuviera explorando un árbol donde cada nodo tiene 1.92 hijos. Si no tuviéramos la heurística y el factor real del problema fuera $b = 3$, habríamos generado $3^5 = 243$ nodos hasta encontrar la meta, contra los 54 que realmente generamos.

Dominancia en heurísticas.

Decimos que una heurística h_2 domina a h_1 si para todo nodo n :

$$h_2(n) \geq h_1(n)$$

Si ambas son admisibles (no se pasan del costo real), la que dé valores más altos es más valiente y precisa. Matemáticamente, si h_2 domina a h_1 , A^* usando h_2 nunca expandirá más nodos que usando h_1 .

Esto sucede porque cualquier nodo que cumpla $f(n) < C^*$ será expandido. Si $h(n)$ es mayor, el valor $f(n) = g(n) + h(n)$ llegará antes al umbral C^* , permitiendo que A^* ignore más ramas inútiles (poda).

A continuación, vemos cómo se comportan las heurísticas en promedio. h_2 (Manhattan) es claramente superior a h_1 porque aprovecha más información del dominio (no solo si la pieza está mal, sino qué tan lejos está):

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Figure 3.26 Comparison of the search costs and effective branching factors for 8-puzzle problems using breadth-first search, A^* with h_1 (misplaced tiles), and A^* with h_2 (Manhattan distance). Data are averaged over 100 puzzles for each solution length d from 6 to 28.

Observación clave: siempre es mejor usar una función heurística con valores más altos, siempre que sea consistente y que el tiempo para calcularla no sea excesivamente largo. Si podemos encontrar una

forma de hacer la heurística más grande sin romper la admisibilidad, adelante.

2.6.2 Generación de heurísticas desde problemas relajados.

Un **problema relajado** es aquel que surge al eliminar una o más restricciones de las acciones del problema original. Al haber menos reglas, el agente tiene más libertad de movimiento, lo que convierte al grafo de estados original en un subgrafo del problema relajado.

Debido a que el problema relajado añade aristas al grafo del espacio de estados, cualquier solución óptima en el problema original es, por definición, también una solución en el problema relajado; pero el problema relajado puede tener mejores soluciones si las aristas añadidas proporcionan atajos. Por lo tanto, el costo de una solución óptima para un problema relajado es una **heurística admisible** para el problema original. Además, debido a que la heurística derivada es un costo exacto para el problema relajado, debe obedecer la desigualdad triangular y, por lo tanto, **es consistente**.

8-puzzle como problema relajado.

Si la definición de un problema se escribe en un lenguaje formal, es posible construir problemas relajados automáticamente. Por ejemplo, si las acciones del juego 8-puzzle se describen como:

Una ficha puede moverse del cuadrado X al cuadrado Y si X es adyacente a Y e Y está en blanco

entonces podemos generar tres problemas relajados eliminando una o ambas condiciones:

1. Una ficha puede moverse del cuadrado X al cuadrado Y si X es adyacente a Y.
2. Una ficha puede moverse del cuadrado X al cuadrado Y si Y está en blanco.
3. Una ficha puede moverse del cuadrado X al cuadrado Y.

A partir de (1) podemos derivar h_2 (distancia de Manhattan), ya que h_2 sería la puntuación adecuada si moviéramos cada ficha a su destino por turno. Y a partir de (3) podemos derivar h_1 (fichas fuera de lugar) porque sería la puntuación adecuada si las fichas pudieran moverse a su destino previsto en una sola acción.

Automatización y combinación.

Si el problema se define en un lenguaje formal, un software (como ABSOLVER) puede generar estas heurísticas mecánicamente eliminando condiciones lógicas de las acciones.

Cuando tenemos varias heurísticas admisibles ($h_1, h_2, \dots, h_k$) y ninguna es claramente superior en todos los casos, la estrategia óptima es utilizar el máximo:

$$h(n) = \max\{h_1(n), \dots, h_k(n)\}$$

Esta heurística compuesta domina a todas las individuales y mantiene la admisibilidad y consistencia, aunque requiere un poco más de tiempo por cómputo por nodo.

2.6.3 Generación de heurísticas a partir de subproblemas: Bases de datos de patrones.

Las heurísticas admisibles también pueden derivarse del costo de la solución de un **subproblema** de un problema dado. Por ejemplo, un subproblema del juego de 8-puzzle podría implicar colocar las fichas 1, 2, 3, 4 y el espacio en blanco en sus posiciones correctas.

Claramente, el costo de la solución óptima de este subproblema es un límite inferior (lower bound) sobre el costo del problema completo. En algunos casos, resulta ser más preciso que la distancia de Manhattan.

A diferencia de los problemas relajados (donde cambiamos las reglas del juego para simplificarlo), en esta técnica las reglas no cambian. Lo que hacemos es resolver a la perfección una versión reducida del problema original.

- Un **patrón** es una configuración específica de esas fichas en el tablero.
- Las posiciones de las fichas 5, 6, 7 y 8 no importan en absoluto; las vemos como si fueran estrellas o fichas idénticas sin número. Sin embargo, los movimientos de esas otras fichas sí cuentan para el costo si es necesario moverlas para abrirle paso al blanco o a nuestras fichas objetivo.

Base de datos de patrones.

Para construir la **base de datos** se hace un proceso una única vez, llamado **búsqueda hacia atrás**:

1. Se empieza desde el estado objetivo (donde el 1, 2, 3 y 4 están en su lugar).
2. Empezamos a hacer movimientos válidos hacia atrás y registramos cuántos pasos nos costó llegar a cada nueva configuración de esas 4 fichas.
3. Al final, se genera una gigantesca tabla hash donde la clave es la posición de las fichas {1,2,3,4,blanco} y el valor es el número exacto de movimientos mínimos para ordenarlas. Para este subproblema, la base de datos guarda exactamente $9 \times 8 \times 7 \times 6 \times 5 = 15,120$ patrones.

Normalmente, si se tiene una base de datos para las fichas 1-4 y otra para las fichas 5-8, no se pueden sumar sus valores. ¿Por qué? Porque para acomodar el grupo 1-4 seguro se tuvo que mover fichas del grupo 5-8. Si se suman ambos costos, estaríamos contando movimientos por duplicado y la heurística dejaría de ser admisible (sobreestimaría el costo real).

La solución brillante son las bases de datos disjuntas:

- En lugar de registrar el costo total de resolver el subproblema, registramos únicamente el número de movimientos que involucraron a las fichas del subproblema.
- Si el espacio en blanco se mueve intercambiándose con la ficha 7, ese movimiento se le cobra a la base de datos de las fichas 5-8, pero no a la de 1-4.
- Al estar los costos 'repartidos' sin solapamiento, ahora sí se pueden sumar. Esto reduce los nodos generados en un juego del 15-puzzle en un factor de 10.000 comparado con la distancia de Manhattan.

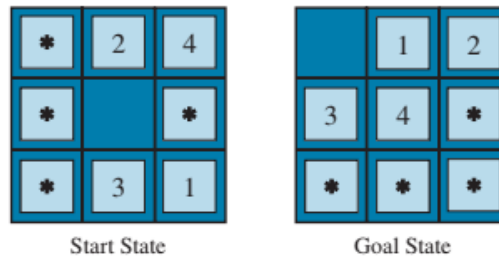


Figure 3.27 A subproblem of the 8-puzzle instance given in Figure 3.25. The task is to get tiles 1, 2, 3, 4, and the blank into their correct positions, without worrying about what happens to the other tiles.

2.6.4 Generación de heurísticas con puntos de referencia (Landmarks).

Esta técnica es la que permite que aplicaciones de mapas encuentren rutas óptimas en grafos de millones de nodos en milisegundos.

Almacenar la ruta óptima entre cada par de puntos ($O(V^2)$) es inviable para una ciudad grande. La alternativa es elegir un conjunto pequeño de nodos estratégicos llamados **Landmarks** (puntos de referencia).

Luego, para cada punto de referencia L y para cada otro vértice v en el grafo, computamos y almacenamos $C^*(v, L)$, el costo exacto de la ruta óptima de v a L . Dadas las tablas C^* almacenadas, podemos crear fácilmente una heurística eficiente (aunque inadmisibile): el mínimo, sobre todos los puntos de referencia, del costo de llegar desde el nodo actual al punto de referencia, y luego al objetivo:

$$h_L(n) = \min_{L \in \text{Landmarks}} C^*(n, L) + C^*(L, \text{goal})$$

Si la ruta óptima pasa por un punto de referencia, esta heurística será exacta; si no, es inadmisibile ya que sobreestima el costo hasta el objetivo.

Heurística diferencial.

Aún así, podemos definir la **heurística diferencial** que es eficiente y admisible:

$$h_{DH}(n) = \max_{L \in \text{Landmarks}} |C^*(n, L) - C^*(\text{goal}, L)|$$

Piensa en esto con un punto de referencia que está en algún lugar más allá del objetivo. Si el objetivo está en la ruta óptima de n al punto de referencia, entonces esto equivale a decir 'considera la ruta completa de n a L , luego resta la última parte de esa ruta, desde el objetivo hasta L , dándonos el costo exacto de la ruta de n al objetivo'.

En la medida en que el objetivo esté un poco fuera de la ruta óptima al punto de referencia, la heurística será inexacta, pero seguirá siendo admisible. Los puntos de referencia que no están más allá del objetivo no serán útiles; un punto de referencia que esté exactamente a mitad de camino entre n y el objetivo dará $h_{DH} = 0$, lo cual no es de ayuda.

Hay varias formas de elegir los puntos de referencia. Seleccionar puntos al azar es rápido, pero obtenemos mejores resultados si tenemos cuidado de esparcir los puntos de referencia para que no estén demasiado cerca unos de otros. Un enfoque voraz es elegir un primer punto de referencia al azar, luego encontrar el punto que esté más alejado de ese y agregarlo al conjunto, y continuar en cada iteración agregando el punto que maximice la distancia al punto de referencia más cercano.

Para la heurística diferencial es bueno si los puntos de referencia están distribuidos alrededor del perímetro del grafo. Así, una buena técnica es encontrar el centroide del grafo, organizar k sectores en forma de porciones de tarta alrededor del centroide, y en cada sector seleccionar el vértice que esté más alejado del centro.

2.7 Búsqueda en entornos complejos.

Bibliografía: Unidad 4 - Artificial Intelligence (AIMA) - (página 123).

En los capítulos anteriores, trabajamos bajo supuestos simplificadores: entornos totalmente observables, deterministas, estáticos y conocidos. En este capítulo, relajamos esas restricciones para acercarnos al mundo real.

A diferencia de la búsqueda sistemática, aquí exploramos entornos donde:

- **No importa el camino:** solo nos interesa encontrar el estado objetivo (el qué, no el cómo).
- **Incertidumbre:** el mundo puede ser no determinista o parcialmente observable.
- **Entornos desconocidos:** el agente debe aprender el mapa mientras lo explora (búsqueda online).

2.7.1 Búsqueda local y optimización de problemas.

En los algoritmos de búsqueda sistemática (como A^* o BFS), el objetivo es encontrar un camino hacia la meta. Sin embargo, en muchos problemas del mundo real, el camino es irrelevante; lo único que importa es el estado final.

Los algoritmos de **búsqueda local** operan explorando desde un estado inicial hacia estados vecinos, sin llevar registro de los caminos ni del conjunto de estados visitados. Esto implica que no son sistemáticos: pueden no explorar partes del espacio de búsqueda donde sí existe una solución.

Sin embargo, tienen dos ventajas clave:

- Usan muy poca memoria
- Pueden encontrar soluciones razonables en espacios de estados grandes o infinitos donde los algoritmos sistemáticos no son adecuados

Los algoritmos de búsqueda local también pueden resolver **problemas de optimización**, donde el objetivo es encontrar el mejor estado según una función objetivo.

Para entender la búsqueda local, se puede imaginar el espacio de estados como un paisaje, donde cada estado tiene una 'altura' determinada por el valor de la función objetivo.

- Si la altura representa la función objetivo $\rightarrow$ buscamos el **máximo global** (hill climbing)
- Si representa el costo $\rightarrow$ buscamos el **mínimo global** (descenso por gradiente)

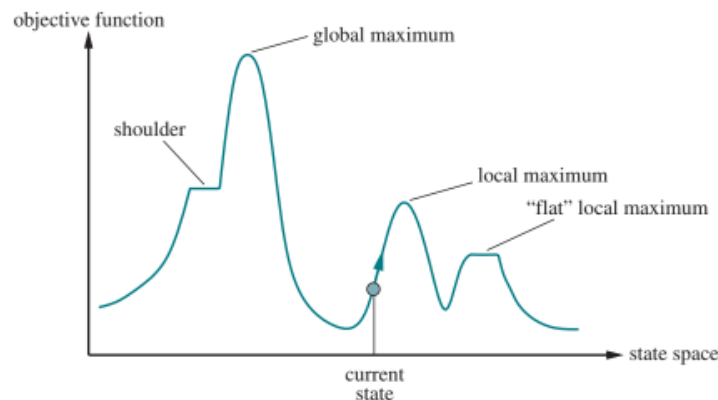


Figure 4.1 A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum.

Ejemplo: El Problema de las 8 Reinas.

Este es el caso de estudio clásico para búsqueda local. El objetivo es colocar 8 reinas en un tablero de 8×8 de forma que ninguna ataque a otra.

- **Representación de Estado:** En lugar de colocar reinas una por una, usamos una *formulación de estado completo*. El estado es una lista de 8 números, donde cada índice es una columna y el valor es la fila de la reina (ej. $[3, 2, 0, 5, 4, 1, 7, 6]$).
- **Función Objetivo (h):** El número de pares de reinas que se atacan entre sí (directa o indirectamente). Aquí buscamos un **mínimo global** donde $h = 0$.
- **Generación de Vecinos:** Un vecino es cualquier estado que resulte de mover una sola reina a una fila distinta dentro de su propia columna ($8 \times 7 = 56$ sucesores posibles por estado).

- **Ejecución:** El algoritmo evalúa los 56 movimientos posibles, elige el que más reduce los ataques y repite hasta que cualquier movimiento aumente el número de ataques.

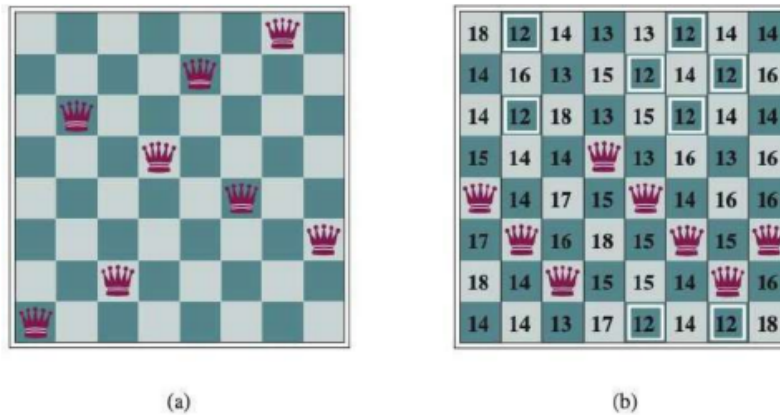


Figure 4.3 (a) The 8-queens problem: place 8 queens on a chess board so that no queen attacks another. (A queen attacks any piece in the same row, column, or diagonal.) This position is almost a solution, except for the two queens in the fourth and seventh columns that attack each other along the diagonal. (b) An 8-queens state with heuristic cost estimate $h=17$. The board shows the value of h for each possible successor obtained by moving a queen within its column. There are 8 moves that are tied for best, with $h=12$. The hill-climbing algorithm will pick one of these.

2.7.2 Búsqueda de Ascenso de Colinas (Hill-Climbing).

Hill Climbing es un **algoritmo de búsqueda local** cuyo objetivo es encontrar un estado óptimo (máximo o mínimo) en el espacio de estados. A diferencia de los algoritmos de búsqueda clásicos, no construye caminos, sino que trabaja directamente sobre estados.

Su idea central es simple: partir de un estado inicial y moverse iterativamente hacia el vecino que mejora más la función objetivo.

Se lo puede interpretar como 'subir una montaña' (maximización) o 'bajar un valle' (minimización), evaluando únicamente los estados vecinos.

Funcionamiento.

El algoritmo mantiene un único estado actual:

- Comienza desde un estado inicial (generalmente aleatorio).
- Genera los vecinos del estado actual.
- Evalúa cada vecino con una función objetivo.
- Selecciona el mejor vecino (mayor valor si maximiza, menor si minimiza).
- Si ese vecino no mejora al estado actual → termina (óptimo local).
- Si mejora → se mueve a ese vecino y repite

Es un algoritmo greedy (voraz) porque siempre toma la mejor decisión local sin considerar consecuencias futuras.

Observaciones.

- **Eficiencia de Recursos:** Es extremadamente ligero. Al guardar solo el estado actual, su complejidad espacial es $O(1)$, lo que permite trabajar en espacios de estados virtualmente infinitos donde algoritmos como A^* fallarían por falta de memoria.
- **Obstáculos del Paisaje (Limitaciones de Búsqueda):** Su enfoque local lo hace vulnerable a la topografía del problema:
 - *Máximos Locales:* Picos que son mejores que sus vecinos pero inferiores al óptimo global. El algoritmo queda "atrapado" sin poder bajar para buscar cimas más altas.
 - *Mesetas (Plateaus):* Zonas planas donde todos los vecinos tienen el mismo valor. El algoritmo entra en un camino aleatorio sin dirección clara.
 - *Crestas (Ridges):* Secuencias de máximos locales que forman una pendiente estrecha. Los movimientos individuales a vecinos parecen empeorar la función, aunque la cresta en su conjunto suba.
- **Propiedades Teóricas:**
 - **No es completo:** Puede terminar sin encontrar una solución si se queda atrapado.
 - **No es óptimo:** El resultado depende drásticamente del **estado inicial**. Una solución común es el *Random-restart Hill Climbing* (reiniciar desde puntos aleatorios).

Implementación en Python.

```

1 import random
2
3 def hill_climbing(initial_state, get_neighbors, value_function):
4     current = initial_state
5
6     while True:
7         neighbors = get_neighbors(current)
8
9         if not neighbors:
10             return current
11
12         # Elegir el mejor vecino
13         neighbor = max(neighbors, key=value_function)
14
15         # Si no mejora, terminamos
16         if value_function(neighbor) <= value_function(current):
17             return current
18
19         current = neighbor

```

2.7.3 Temple Simulado (Simulated Annealing).

El problema del Hill-Climbing es que nunca acepta un movimiento 'hacia abajo', lo que lo condena a quedar atrapado en la primera colina que encuentre. El **Temple Simulado** soluciona esto introduciendo **estocasticidad** (azar) y conceptos de la termodinámica.

Simulated Annealing es un algoritmo de **búsqueda local estocástica** diseñado para encontrar soluciones óptimas en problemas donde hay muchos **máximos/mínimos locales**.

A diferencia de Hill Climbing, este algoritmo sí permite movimientos hacia estados peores, con cierta probabilidad. Esto le permite escapar de óptimos locales y explorar mejor el espacio de estados. Su objetivo es combinar:

- La eficiencia de Hill Climbing
- La capacidad exploratoria de una búsqueda aleatoria

Funcionamiento.

El algoritmo mantiene un estado actual y un parámetro llamado **temperatura (T)** que controla el nivel de exploración. El proceso es:

- Comienza con un estado inicial (generalmente aleatorio).
- Define una temperatura inicial alta.
- En cada iteración:
 - Selecciona un vecino al azar
 - Calcula el cambio en la función objetivo: ΔE
 - Si el vecino es mejor $\rightarrow$ lo acepta siempre
 - Si es peor $\rightarrow$ lo acepta con probabilidad:

$$P = e^{(-\frac{\Delta E}{T})}$$

donde ΔE es la pérdida de calidad del estado y T es la temperatura que va disminuyendo con el tiempo.

- La temperatura va disminuyendo con el tiempo (schedule).
- Cuando la temperatura llega a 0, el algoritmo termina.

Observaciones Críticas.

- **Exploración vs. Explotación:** Al inicio (T alta), el algoritmo prioriza la exploración, aceptando casi cualquier movimiento. Al final (T baja), prioriza la explotación, refinando la solución en la zona donde se encuentra.
- **El Programa de Enfriamiento (*Schedule*):** Es el factor más crítico. Si la temperatura baja demasiado rápido (enfriamiento brusco), el algoritmo se comporta como Hill-Climbing y se queda atrapado. Si baja demasiado lento, el tiempo de cómputo es excesivo.
- **Garantía de Optimización:** Se ha demostrado matemáticamente que, si el enfriamiento es lo suficientemente lento, la probabilidad de que el algoritmo termine en el **óptimo global** se acerca a 1.
- **Analogía Física:** Se basa en el proceso metalúrgico donde un metal se calienta para que sus átomos se reorganicen y luego se enfría lentamente para obtener una estructura cristalina de energía mínima y máxima resistencia.

Implementación en Python.

```
1 import math
2 import random
3
4 def simulated_annealing(initial_state, get_neighbors, value_function, schedule):
5     current = initial_state
```

```

6     t = 1
7
8     while True:
9         T = schedule(t)
10        if T == 0:
11            return current
12
13        neighbors = get_neighbors(current)
14        if not neighbors:
15            return current
16
17        next_state = random.choice(neighbors)
18
19        delta_E = value_function(next_state) - value_function(current)
20
21        if delta_E > 0:
22            current = next_state
23        else:
24            probability = math.exp(delta_E / T)
25            if random.random() < probability:
26                current = next_state
27
28        t += 1

```

3 Sistemas Basados en Conocimiento.

3.1 Introducción al Aprendizaje Automático (Machine Learning).

Desde la invención de las computadoras, nos hemos preguntado si podrían aprender. El impacto de programarlas para que mejoren automáticamente con la experiencia sería dramático: desde sistemas médicos que aprenden los tratamientos más efectivos, hasta asistentes personales que filtran noticias según los intereses cambiantes del usuario.

El **Aprendizaje Automático (Machine Learning)** no solo abre nuevas puertas a la personalización y competencia computacional, sino que también nos ayuda a entender mejor las capacidades (y discapacidades) del aprendizaje humano.

Hoy en día, el aprendizaje automático ya supera a otros enfoques en tareas como:

- **Reconocimiento de voz:** Adaptación automática a hablantes, vocabularios y ruido de fondo.
- **Minería de Datos (Data Mining):** Descubrimiento de patrones en bases de datos financieras, médicas o de mantenimiento.
- **Vehículos Autónomos:** Sistemas como ALVINN que aprenden a conducir observando a humanos.
- **Juegos de Clase Mundial:** Programas como TD-GAMMON que alcanzan niveles de campeones mundiales jugando millones de partidas contra sí mismos.

3.1.1 Problemas de aprendizaje bien definidos.

Para estudiar el aprendizaje de manera técnica, Mitchell propone una definición amplia pero precisa, alejándose del significado coloquial de la palabra **aprender**.

Definición. Se dice que un programa de computadora aprende de la **experiencia E** con respecto a una clase de **tareas T** y una **medida de desempeño P**, si su desempeño en las tareas de T, medido por P, mejora con la experiencia E.

Para que un problema de aprendizaje esté bien definido, debemos identificar estos tres elementos fundamentales:

- **La Clase de Tareas (T):** ¿Qué es lo que el sistema debe hacer?
- **La Medida de Desempeño (P):** ¿Cómo cuantificamos el éxito o la mejora?
- **La Fuente de Experiencia (E):** ¿De qué datos o interacciones aprenderá el sistema?

Ejemplos de Problemas Estructurados (T, P, E):

1. Aprendizaje de Juego de Damas

- Tarea T: Jugar partidas de damas.
- Medida de Desempeño P: Porcentaje de juegos ganados contra oponentes.
- Experiencia de Entrenamiento E: Jugar partidas de práctica contra sí mismo.

2. Reconocimiento de Escritura Manual

- Tarea T: Reconocer y clasificar palabras escritas a mano en imágenes.
- Medida de Desempeño P: Porcentaje de palabras clasificadas correctamente.

- Experiencia de Entrenamiento E: Una base de datos de palabras escritas a mano con sus clasificaciones correctas (etiquetas).

3. Conducción Autónoma de Robots

- Tarea T: Conducir en autopistas públicas de cuatro carriles usando sensores de visión.
- Medida de Desempeño P: Distancia promedio recorrida antes de un error (juzgado por un supervisor humano).
- Experiencia de Entrenamiento E: Una secuencia de imágenes y comandos de dirección grabados mientras se observa a un conductor humano.

3.1.2 Diseñando un sistema de aprendizaje.

Para ilustrar los conceptos básicos, Mitchell propone el diseño de un programa que aprenda a jugar a las damas, con el objetivo final de inscribirlo en un torneo mundial.

- **Tarea T:** Jugar a las damas.
- **Medida de Desempeño P:** Porcentaje de juegos ganados en el torneo mundial.

Para completar el diseño, debemos tomar decisiones críticas sobre la experiencia, la función objetivo y su representación.

3.1.2.1 Elección de la experiencia de entrenamiento.

La calidad y el tipo de experiencia (datos) pueden determinar el éxito o fracaso del sistema. Mitchell destaca tres atributos clave:

1. Retroalimentación Directa vs. Indirecta.

- **Directa:** El sistema recibe 'el movimiento correcto' para cada tablero. Es más fácil de aprender pero requiere un tutor experto que etiquete cada jugada.
- **Indirecta:** El sistema solo conoce el resultado final (ganó o perdió) después de una secuencia de movimientos.

El gran desafío de la retroalimentación indirecta es el **Problema de la Asignación de Crédito**. Si el sistema pierde, ¿fue por el último movimiento o por un error cometido al principio del juego? Es difícil determinar qué acción específica merece el 'crédito' o la 'culpa'.

2. Grado de Control del Aprendiz.

- El sistema puede ser **pasivo** (un profesor le da los ejemplos).
- El sistema puede ser **activo** (el aprendiz propone tableros que le resultan confusos y pregunta al experto).
- El sistema puede tener **control total** (jugar contra sí mismo sin profesor), lo que le permite experimentar con nuevas estrategias.

3. Representatividad de la distribución de datos.

El aprendizaje es más fiable si los datos de entrenamiento siguen una distribución similar a los datos de prueba futuros.

Si nuestro programa solo juega contra sí mismo, puede que nunca vea jugadas maestras que un campeón mundial humano sí realizaría. En teoría de Machine Learning, solemos asumir que 'entrenamiento = prueba', pero en la práctica esta suposición suele violarse.

Decisión de diseño para nuestro ejemplo: El sistema entrenará jugando contra sí mismo. Ventaja: puede generar datos ilimitados sin necesidad de un humano.

3.1.2.2 Elección de la función objetivo.

Debemos decidir exactamente qué tipo de conocimiento debe aprender el sistema.

Podríamos intentar aprender una función llamada *ChooseMove* : $B \rightarrow M$ (que toma un tablero B y devuelve el mejor movimiento M). Sin embargo, con retroalimentación indirecta, esta función es muy difícil de aprender.

En lugar de elegir el movimiento directamente, el programa aprende a asignar un puntaje numérico a cualquier estado del tablero. Utilizamos una función de evaluación $V : B \rightarrow \mathbb{R}$ que mapea un tablero a un número real. Si el sistema tiene una buena función V , solo tiene que mirar todos los movimientos legales posibles, evaluar los tableros resultantes con V , y elegir el que tenga el puntaje más alto.

Mitchel define la función objetivo ideal de la siguiente manera:

1. Si b es un estado final **ganado**: $V(b) = 100$.
2. Si b es un estado final **perdido**: $V(b) = -100$.
3. Si b es un estado final de **empate**: $V(b) = 0$.
4. Si b **no es un estado final**: $V(b) = V(b')$, donde b' es el mejor estado final que se puede alcanzar jugando óptimamente desde b .

Esta definición es **no operacional**. Aunque es perfecta matemáticamente, no sirve para jugar en tiempo real porque para calcular $V(b)$ tendríamos que explorar todas las jugadas posibles hasta el fin del universo (caso 4).

El objetivo del aprendizaje: descubrir una **descripción operacional** de V . Es decir, una función que se pueda calcular rápidamente para decidir un movimiento en pocos segundos.

Como es casi imposible aprender la función V perfecta, el programa intentará aprender una aproximación, que denotaremos como $\hat{V}$. A este proceso lo llamamos **Aproximación de Funciones**.

3.1.2.3 Elección de una representación para la función objetivo.

Una vez que sabemos qué queremos aprender (la función ideal V), debemos decidir cómo la representará el programa ($\hat{V}$).

- Si elegimos una representación muy expresiva (como una red neuronal profunda o una tabla gigante con cada estado posible), podremos acercarnos mucho a la función ideal V . Sin embargo, necesitaremos una cantidad enorme de datos de entrenamiento.
- Si elegimos una representación simple (como una función lineal), será más fácil y rápido aprender con pocos datos, pero corremos el riesgo de que la aproximación sea demasiado pobre.

La elección de Mitchell: Una combinación lineal de características del tablero. Para cualquier tablero b , extraeremos 6 características numéricas:

- x_1 : número de fichas negras.
- x_2 : número de fichas rojas.

- x_3 : número de reyes negros.
- x_4 : número de reyes rojos.
- x_5 : piezas negras amenazadas por rojas.
- x_6 : piezas rojas amenazadas por negras.

La función aprendida será:

$$\hat{V}(b) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6$$

Aquí, los pesos (w_i) representan la importancia relativa de cada característica. Por ejemplo, si w_3 termina siendo un número grande, significa que el sistema aprendió que tener reyes negros es muy valioso para ganar.

3.1.2.4 Elección de un algoritmo de aproximación.

Para aprender los pesos w_i , necesitamos ejemplos de entrenamiento con el formato $(b, V_{train}(b))$. Aquí enfrentamos dos sub-problemas:

- **Estimación de los valores de entrenamiento (V_{train}).**

Como solo sabemos si ganamos o perdimos al final, ¿cómo le asignamos un puntaje a un tablero que ocurrió a mitad de la partida? Mitchell propone una regla de estimación ingeniosa: el valor de entrenamiento de un tablero actual es el valor que el sistema estima para el siguiente tablero (su sucesor):

$$V_{train}(b) \leftarrow \hat{V}(Successor(b))$$

Aunque parezca extraño usar una estimación propia para entrenarse a sí mismo, las estimaciones son más precisas a medida que nos acercamos al final del juego. Con el tiempo, esa 'certeza' del final se propaga hacia atrás, hacia los movimientos iniciales.

- **Ajuste de los pesos. La regla LMS.**

Ahora que tenemos los ejemplos, necesitamos ajustar los pesos para minimizar el error cuadrático (E) entre lo que predice nuestra función $\hat{V}(b)$ y el valor de entrenamiento $V_{train}(b)$.

Para esto usamos el algoritmo LMS (Least Mean Squares) o Mínimos Cuadrados Medios. Por cada ejemplo, el algoritmo actualiza cada peso w_i de la siguiente manera:

$$w_i \leftarrow w_i + \eta(V_{train}(b) - \hat{V}(b))x_i$$

- η (tasa de aprendizaje): un valor pequeño (ej. 0.1) que controla qué tan brusco es el cambio.
- η (tasa de aprendizaje): un valor pequeño (ej. 0.1) que controla qué tan brusco es el cambio.
- η (tasa de aprendizaje): un valor pequeño (ej. 0.1) que controla qué tan brusco es el cambio.

Si el error es cero, no cambia nada. Si nuestra predicción fue muy baja ($V_{train} - \hat{V}$ es positivo), los pesos aumentan para que la próxima vez la predicción sea más alta. Si una característica x_i es cero, su peso w_i no se toca, ya que no contribuyó al error en ese tablero específico.

3.1.2.5 El diseño final.

El sistema de aprendizaje de las damas se puede dividir en cuatro módulos distintos. Esta arquitectura es un estándar para entender cómo fluye la información en un agente que aprende.

1. **Sistema de Desempeño (Performance System):** Es el programa que 'hace la tarea'. Toma una partida nueva y, usando la función $\hat{V}$ aprendida, genera una solución (la historia del juego). Su objetivo es mejorar a medida que el Generalizador refina la función.
2. **El Crítico (Critic):** Observa la historia del juego producida por el Sistema de Desempeño. Su trabajo es convertir esa historia en ejemplos de entrenamiento. Utiliza la regla de estimación $V_{train}(b) \leftarrow \hat{V}(Successor(b))$ para asignar puntajes a tableros intermedios.
3. **El Generalizador (Generalizer):** Es el corazón del aprendizaje (en nuestro caso, el algoritmo LMS). Toma los ejemplos del Crítico y busca una **hipótesis** (los pesos w_i) que generalice bien tanto para los ejemplos vistos como para tableros futuros.
4. **Generador de Experimentos (Experiment Generator):** Decide cuál será el próximo problema a resolver. En un diseño simple, siempre empieza desde el tablero inicial. En diseños avanzados, podría proponer posiciones específicas para 'explorar' zonas del tablero donde el sistema es débil.

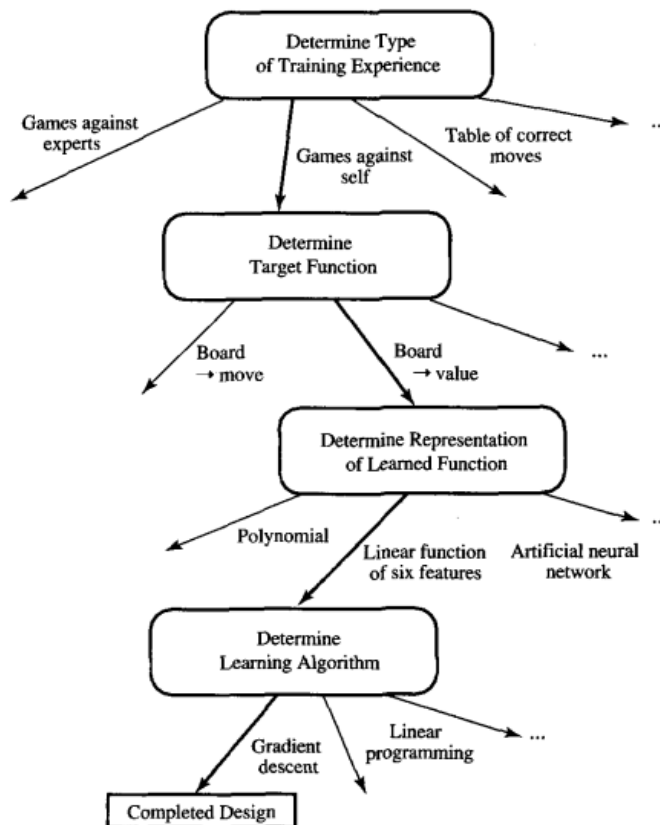


FIGURE 1.2
Summary of choices in designing the checkers learning program.

Resumen de Decisiones.

Al diseñar este sistema, restringimos el conocimiento a una función lineal de seis características. Si la estrategia óptima de las damas fuera mucho más compleja (que lo es), nuestro programa solo podrá aprender la 'mejor aproximación lineal' posible, pero nunca la perfección.

Es como intentar pintar una obra de Da Vinci usando solo reglas y escuadras. Podrás aproximar las formas, pero te perderás los matices. Para esos matices, Mitchell menciona que sistemas como TD-Gammon usan Redes Neuronales, que son mucho más expresivas.

3.1.3 Perspectivas y problemas en el aprendizaje automático.

Una de las ideas más potentes de Mitchell es ver el aprendizaje como un **problema de búsqueda**. Aprender no es más que **buscar en un espacio de hipótesis gigantesco** para encontrar aquella que mejor se ajuste a los datos.

- En nuestro caso, el 'espacio de búsqueda' son todas las combinaciones posibles de los pesos $w_0, \dots, w_6$.
- El algoritmo LMS es el vehículo que navega ese espacio para encontrar el punto (la hipótesis) que minimiza el error.

Problemas fundamentales (The Big Questions).

El campo del Machine Learning intenta responder a estas preguntas clave:

- **Algoritmos de Convergencia:** ¿Bajo qué condiciones se garantiza que el algoritmo encontrará la función deseada?
- **Suficiencia de Datos:** ¿Cuántos ejemplos de entrenamiento son necesarios para confiar en que la hipótesis es correcta?
- **Conocimiento Previo:** ¿Cómo puede el conocimiento que ya tenemos ayudar a guiar el aprendizaje?
- **Estrategia de Experiencia:** ¿Cuál es la mejor forma de elegir el próximo ejemplo para aprender lo más rápido posible?
- **Alteración de la Representación:** ¿Puede el sistema cambiar su propia forma de representar el problema para aprender mejor? (Esto es lo que hoy llamamos Deep Learning).

3.2 Aprendizaje de Conceptos y el Orden de lo General a lo Específico.

El problema de inducir funciones generales a partir de ejemplos de entrenamiento específicos es el corazón del aprendizaje. En este capítulo, exploraremos el **Aprendizaje de Conceptos**: la adquisición de la definición de una categoría general a partir de una muestra de ejemplos positivos y negativos de dicha categoría.

El **aprendizaje de conceptos** se puede formular como un **problema de búsqueda** a través de un espacio predefinido de hipótesis potenciales. El objetivo es encontrar la hipótesis que mejor se ajuste a los ejemplos observados. Esta búsqueda se puede organizar de manera eficiente aprovechando una estructura natural en el espacio de hipótesis: el **ordenamiento de lo general a lo específico**.

3.2.1 Introducción.

Gran parte del aprendizaje humano consiste en adquirir conceptos generales a partir de experiencias específicas. Por ejemplo, las personas aprendemos continuamente categorías como 'pájaro', 'automóvil' o incluso conceptos más abstractos como 'situaciones en las que debería estudiar más para aprobar el examen'. Cada uno de estos conceptos puede verse de dos formas:

1. Como un **subconjunto de objetos o eventos** definido sobre un conjunto más grande (por ejemplo, el subconjunto de animales que constituyen los pájaros).
2. Como una **función con valor booleano** definida sobre ese conjunto mayor (por ejemplo, una función que evalúa a todos los animales y devuelve verdadero para los pájaros y falso para los demás).

En esta unidad, nos enfocaremos en el problema de inferir automáticamente la definición general de un concepto, dados ejemplos etiquetados como miembros (+) o no miembros (-) del mismo.

Para nuestra definición de **aprendizaje de conceptos**, utilizaremos el segundo enfoque: inferir una función de valor booleano a partir de ejemplos de entrenamiento que muestran sus entradas y salidas.

3.2.2 Una tarea de Aprendizaje de Conceptos.

Para aterrizar la teoría, Mitchell nos presenta el caso de Aldo y su deporte acuático favorito. La meta es aprender el concepto objetivo: 'Días en los que Aldo disfruta de su deporte acuático favorito'.

Imagina que tenemos un registro de días pasados (ver siguiente tabla). Cada día se describe por seis atributos. El atributo **EnjoySport** es nuestra etiqueta (label) que indica si Aldo disfrutó o no.

Ejemplo	Cielo (Sky)	TempAire	Humedad	Viento	Agua	Pronóstico	EnjoySport
1	Soleado	Cálido	Normal	Fuerte	Cálida	Estable	Si
2	Soleado	Cálido	Alta	Fuerte	Cálida	Estable	Si
3	Lluvioso	Frío	Alta	Fuerte	Cálida	Cambio	No
4	Soleado	Cálido	Alta	Fuerte	Fría	Cambio	Si

Representación de las hipótesis.

¿Cómo piensa nuestro aprendiz? Vamos a darle una estructura simple: cada hipótesis será una **conjunción de restricciones** sobre los atributos. Es decir, una regla que dice: 'Aldo juega si Atributo1 es X y Atributo2 es Y..'.

Para cada atributo, la hipótesis puede tener tres tipos de restricciones:

1. ? (**Cualquiera**): No importa el valor (ej: nos da igual si el agua está fría o caliente).

2. **Valor específico (ej. Cálido):** El atributo debe coincidir exactamente.
3. $\emptyset$ (**Ninguno**): No se acepta ningún valor (es una restricción vacía).

Ejemplo de hipótesis: Aldo solo juega en días fríos con alta humedad, sin importar el resto:

(?, Frío, Alta, ?, ?, ?)

- **La hipótesis más general** (Aldo juega siempre): $(?, ?, ?, ?, ?, ?)$.
- **La hipótesis más específica** (Aldo no juega nunca): $(\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$.

3.2.2.1 Notación.

Para hablar con propiedad, Mitchell define los siguientes símbolos:

- **Instancias (X):** El conjunto de todos los días posibles. En nuestro caso, X está definido por todas las combinaciones posibles de los 6 atributos.
- **Concepto Objetivo (c):** La función verdadera que queremos descubrir. Es una función booleana $c : X \rightarrow \{0, 1\}$. Si Aldo juega, $c(x) = 1$.
- **Ejemplos de Entrenamiento (D):** Los pares ordenados $\langle x, c(x) \rangle$ que ya conocemos (como los de la Tabla).
 - Ejemplos positivos: Miembros del concepto ($c(x) = 1$).
 - Ejemplos negativos: No miembros ($c(x) = 0$).
- **Espacio de Hipótesis (H):** El conjunto de todas las reglas posibles que el aprendiz puede considerar. Nuestra meta es encontrar una $h \in H$ tal que $h(x) = c(x)$ para toda $x \in X$.

3.2.2.2 La hipótesis del aprendizaje inductivo.

Como solo conocemos el valor de c para unos pocos ejemplos de entrenamiento, lo mejor que podemos hacer es encontrar una hipótesis h que se ajuste perfectamente a esos datos observados.

La hipótesis del aprendizaje inductivo: cualquier hipótesis que aproxime bien la función objetivo sobre un conjunto lo suficientemente grande de ejemplos de entrenamiento, también aproximará bien la función sobre ejemplos no observados. Es decir, si aprendés bien el pasado, vas a predecir bien el futuro.

3.2.3 El aprendizaje de conceptos como búsqueda.

Podemos ver el aprendizaje de conceptos como la tarea de buscar en un espacio de hipótesis H (definido implícitamente por la representación que elegimos) aquella hipótesis que mejor se ajuste a los ejemplos de entrenamiento.

Al elegir una representación, el diseñador define el límite de lo que el programa puede aprender. Si la regla correcta no está en H , el programa nunca la encontrará.

El tamaño del espacio.

Para el problema de EnjoySport, podemos calcular qué tan grande es el 'pajar' donde buscamos la aguja:

- **Espacio de Instancias (X):** Tenemos 6 atributos. El primero tiene 3 valores posibles y los otros cinco tienen 2.

$$|X| = 3 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 96 \text{ instancias posibles}$$

- **Espacio de Hipótesis Sintácticas (H_{sint}):** Cada atributo puede tomar sus valores originales, más el símbolo ? y el símbolo $\emptyset$.

$$|H_{sint}| = 5 \cdot 4 \cdot 4 \cdot 4 \cdot 4 \cdot 4 = 5120 \text{ hipótesis escritas.}$$

- **Espacio de Hipótesis Semánticas (H_{sem}):** Aquí está el truco. Cualquier hipótesis que tenga al menos un $\emptyset$ representa el **conjunto vacío** (no acepta ningún día). Todas esas hipótesis son lo mismo funcionalmente. Por eso, el número de hipótesis con significados distintos es:

$$1 + (4 \cdot 3 \cdot 3 \cdot 3 \cdot 3 \cdot 3) = 973 \text{ hipótesis distintas.}$$

3.2.3.1 Orden de lo General a lo Específico.

Para no tener que revisar las 973 hipótesis una por una, aprovechamos una estructura natural: el orden de generalidad. Consideremos estas dos hipótesis:

- $h_1 = (\text{Soleado}, ?, ?, \text{Fuerte}, ?, ?)$
- $h_2 = (\text{Soleado}, ?, ?, ?, ?, ?)$

h_2 es más general que h_1 porque impone menos restricciones. Cualquier día que haga feliz a h_1 , también hará feliz a h_2 .

Definición Formal: $\geq_g$. Decimos que h_j es **más-general-que-o-igual-a** h_k (notado $h_j \geq_g h_k$) si y solo si cualquier instancia que satisface a h_k también satisface a h_j :

$$(\forall x \in X)[(h_k(x) = 1) \rightarrow (h_j(x) = 1)]$$

- **Relación estrictamente más general ($>_g$):** Cuando $h_j \geq_g h_k$ pero $h_k \not\geq_g h_j$.
- **Más Específico Que:** Es simplemente la inversa de la relación de generalidad.
- **Orden Parcial:** Se dice que es un orden parcial porque no todas las hipótesis se pueden comparar entre sí. Por ejemplo, una que pida Cielo Soleado y otra que pida Humedad Alta no son necesariamente una más general que la otra; simplemente cubren áreas diferentes.
- **Nota:** Esta estructura es la que permite que los algoritmos 'salten' partes del espacio de búsqueda sin tener que mirar cada hipótesis individualmente. Si una hipótesis ya es demasiado específica para cubrir un ejemplo positivo, todas las que son aún más específicas también lo serán.

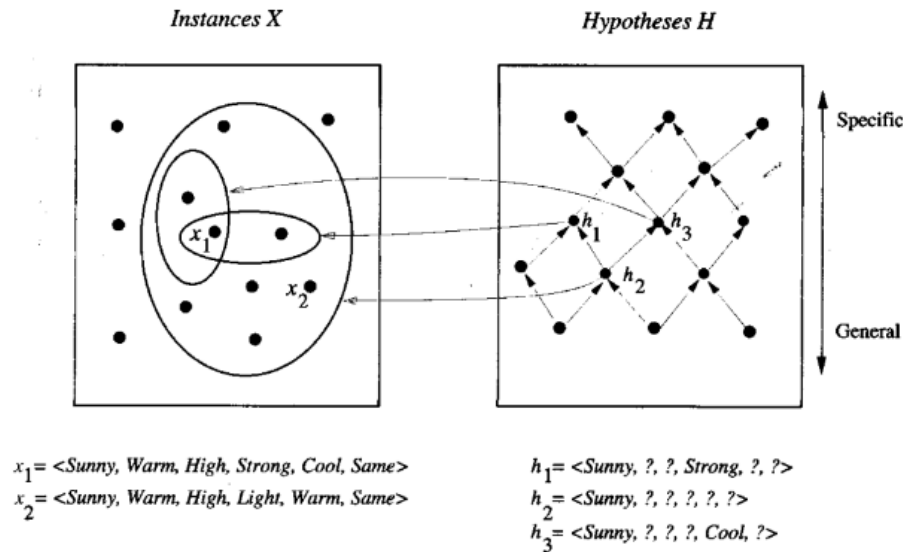


FIGURE 2.1

Instances, hypotheses, and the *more_general_than* relation. The box on the left represents the set X of all instances, the box on the right the set H of all hypotheses. Each hypothesis corresponds to some subset of X —the subset of instances that it classifies positive. The arrows connecting hypotheses represent the *more_general_than* relation, with the arrow pointing toward the less general hypothesis. Note the subset of instances characterized by h_2 subsumes the subset characterized by h_1 , hence h_2 is *more_general_than* h_1 .

3.2.4 Find-S: Algoritmo para encontrar una hipótesis máximamente específica.

El algoritmo Find-S utiliza el ordenamiento de lo general a lo específico para navegar por el espacio de búsqueda. Su lógica es simple: comienza asumiendo que nada cumple el concepto y va 'abriendo la mano' solo cuando los ejemplos positivos lo obligan.

- **Inicialización.** Empezamos con la hipótesis más específica posible: $h = (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$. En esta etapa, el sistema cree que Aldo no disfruta del deporte nunca.
- **Encuentro con ejemplos positivos.** Cuando aparece el primer ejemplo positivo de las instancias X (ej: un día soleado), el algoritmo reemplaza los $\emptyset$ por los valores reales de ese día.
Si aparece otro ejemplo positivo y algún atributo no coincide con nuestra h actual, lo reemplazamos por un $?$. Esto significa: 'bueno, parece que este atributo no es tan restrictivo como pensaba'.
- **Encuentro con ejemplos negativos.** Find-S ignora los ejemplos negativos por completo. Esto sucede así porque estamos construyendo la hipótesis más específica posible que cubra los positivos. Si el concepto real está en nuestro espacio H , esa hipótesis específica por definición ya debería excluir a los negativos.

3.2.4.1 Implementación en Python.

```

1 # 'training_data' debe ser una lista de listas (una matriz de strings).
2 # Cada sublista representa un ejemplo: los primeros N elementos son los valores

```

```

3  # de los atributos y el último elemento (índice -1) es la etiqueta ('Yes' o 'No').
4  def find_s(training_data):
5      # Inicializar h a la hipotesis mas especifica posible
6      num_attributes = len(training_data[0]) - 1
7      h = [None] * num_attributes
8
9      for example in training_data:
10         # Extraer atributos (x) y el valor del concepto objetivo (label)
11         x = example[:-1]
12         label = example[-1]
13
14         # Solo procesamos ejemplos positivos
15         if label == "Yes":
16             for i in range(num_attributes):
17                 # Si es el primer ejemplo positivo, copiamos los atributos
18                 if h[i] is None:
19                     h[i] = x[i]
20
21                 # Si el atributo de h no coincide con el del ejemplo,
22                 # generalizamos
23                 elif h[i] != x[i]:
24                     h[i] = '?'
25
26         return h
27
28 # Datos de la de Mitchell
29 data = [
30     ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
31     ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
32     ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
33     ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"]
34 ]
35 resultado = find_s(data)
36 print(f"La hipótesis más específica encontrada es: {resultado}")
37 # Resultado esperado: ['Sunny', 'Warm', '?', 'Strong', '?', '?']

```

3.2.4.2 Ejemplo. Evolución de la hipótesis.

- **Paso 0: Inicialización.** La hipótesis inicial h_0 es la más específica posible:

$$h_0 = (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$$

- **Paso 1: Primer ejemplo positivo.** Instancia x_1 : (Sunny, Warm, Normal, Strong, Warm, Same), Label: Yes. Como h_0 es demasiado específica (no "cubre" a x_1), la actualizamos con los valores exactos del ejemplo:

$$h_1 = (\text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same})$$

- **Paso 2: Segundo ejemplo positivo.** Instancia x_2 : (Sunny, Warm, High, Strong, Warm, Same), Label: Yes. Comparamos h_1 con x_2 . El atributo Humedad cambia de Normal a High. Para cubrir ambos casos, lo generalizamos a ?:

$$h_2 = (\text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same})$$

- **Paso 3: Ejemplo negativo.** Instancia x_3 : (Rainy, Cold, High, Strong, Warm, Change), Label: No. Find-S simplemente ignora este ejemplo. Dado que nuestra hipótesis actual h_2 ya clasifica este día como No (porque el cielo no es Sunny y la temperatura no es Warm), no hace falta ningún cambio.

$$h_3 = (\text{Sunny, Warm, ?, Strong, Warm, Same})$$

- **Paso 4: Tercer ejemplo positivo.** Instancia x_4 : (Sunny, Warm, High, Strong, Cool, Change), Label: Yes. Comparamos h_3 con x_4 . Notamos discrepancias en Agua (Warm vs Cool) y Pronóstico (Same vs Change). Generalizamos ambos:

$$h_4 = (\text{Sunny, Warm, ?, Strong, ?, ?})$$

Resultado Final. Al terminar los datos de entrenamiento, la hipótesis de salida es h_4 . Este es el "concepto" que el algoritmo ha aprendido: Aldo disfruta de su deporte siempre que el día esté soleado, haga calor y el viento sea fuerte, independientemente de la humedad, la temperatura del agua o el pronóstico del tiempo.

Es la regla más restrictiva que todavía permite explicar todos los éxitos de Aldo documentados en la tabla.

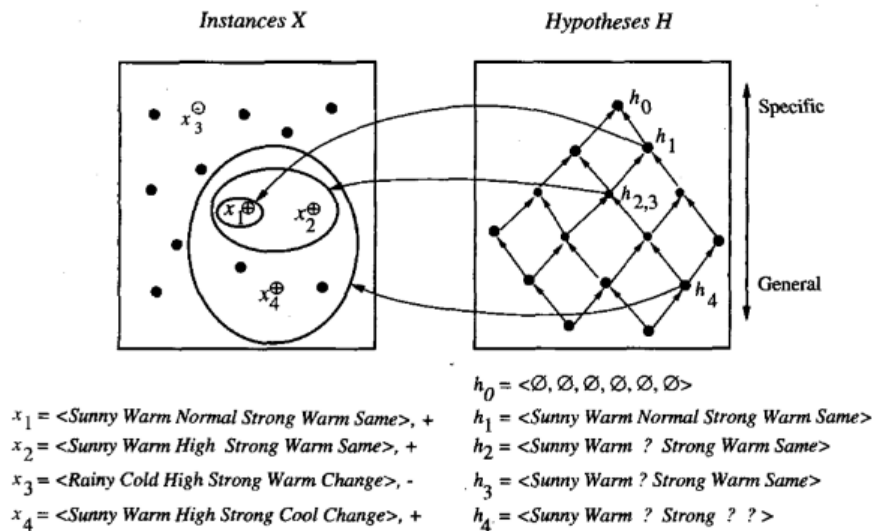


FIGURE 2.2

The hypothesis space search performed by FIND-S. The search begins (h_0) with the most specific hypothesis in H , then considers increasingly general hypotheses (h_1 through h_4) as mandated by the training examples. In the instance space diagram, positive training examples are denoted by "+," negative by "-", and instances that have not been presented as training examples are denoted by a solid circle.

3.2.4.3 Limitaciones críticas de Find-S.

- **¿Es la única solución?:** Find-S encuentra una hipótesis, pero no nos dice si hay otras 10 que también funcionan. No sabemos si ya "aprendimos" el concepto real o solo una aproximación.
- **Sesgo hacia la especificidad:** ¿Por qué preferir la más específica? Podría haber una más general que sea la correcta.

- **Sensibilidad al ruido:** Si un ejemplo positivo está mal etiquetado (es ruido), Find-S generalizará la hipótesis erróneamente y no hay forma de volver atrás.
- **Múltiples hipótesis específicas:** En espacios más complejos, podría haber varias hipótesis máximamente específicas que no se pueden comparar entre sí. Find-S no sabe manejar esa divergencia.

3.2.5 Espacios de Versiones y el Algoritmo Candidate-Elimination.

La idea central aquí es dejar de buscar una única hipótesis y empezar a describir el **conjunto de todas las hipótesis** que son consistentes con los datos.

Lo increíble de este algoritmo es que puede describir ese conjunto sin tener que enumerar cada hipótesis una por una (lo cual sería imposible en espacios grandes). Para lograrlo, vuelve a apoyarse en el ordenamiento de lo general a lo específico.

3.2.5.1 Representación.

Mitchell hace una distinción técnica crucial entre 'satisfacer' y 'ser consistente':

- **Satisfacer (*Satisfies*):** Una instancia x satisface a h si la regla dice que es positiva ($h(x) = 1$). No importa si en la realidad el ejemplo era positivo o negativo.
- **Consistencia (*Consistent*):** Una hipótesis h es consistente con un conjunto de datos D si, para cada ejemplo en el conjunto, la predicción de la hipótesis coincide con la etiqueta real ($h(x) = c(x)$).
- **Definición de Espacio de Versiones ($VS_{H,D}$):** Es el subconjunto de todas las hipótesis de H que son consistentes con los ejemplos de entrenamiento D :

$$VS_{H,D} = \{h \in H \mid \text{Consistent}(h, D)\}$$

En palabras simples: el Espacio de Versiones contiene todas las versiones 'plausibles' del concepto que queremos aprender.

3.2.5.2 El algoritmo List-Then-Eliminate.

Antes de ir al algoritmo complejo, Mitchell nos presenta la versión de 'fuerza bruta'. Es el método más intuitivo pero menos eficiente del mundo:

- **Lista:** Metés absolutamente todas las hipótesis posibles de H en una lista gigante (el Espacio de Versiones inicial).
- **Elimina:** Por cada ejemplo de entrenamiento que vas viendo, revisás la lista. Si una hipótesis se equivoca (es inconsistente), la borrás para siempre.
- **Resultado:** Al final, te quedás solo con las que nunca fallaron.

Enumerar todo H es un requisito poco realista. Por eso, en las siguientes secciones vamos a ver cómo representar este mismo 'Espacio de Versiones' usando solo los límites: las hipótesis más generales y las más específicas.

3.2.5.3 Una representación más compacta de los espacios de versiones.

En lugar de listar todas las hipótesis posibles (que vimos que pueden ser miles), Mitchell propone representarlas usando solo sus límites. Es como definir un segmento en una recta numérica: no necesitas nombrar todos los infinitos puntos, solo los extremos.

El espacio de versiones se delimita mediante dos conjuntos de hipótesis:

- **Frontera General (G):** El conjunto de las hipótesis máximamente generales que son consistentes con los datos vistos.
- **Frontera Específica (S):** El conjunto de las hipótesis máximamente específicas (mínimamente generales) consistentes con los datos.

Teorema. Representación del Espacio de Versiones. Sea X un conjunto arbitrario de instancias y sea H un conjunto de hipótesis con valores booleanos definidas sobre X . Sea $c : X \rightarrow \{0,1\}$ un concepto objetivo arbitrario y $D = \{(x, c(x))\}$ un conjunto de ejemplos de entrenamiento.

Si los conjuntos frontera específica S y frontera general G están bien definidos respecto a H y D , entonces:

$$VS_{H,D} = \{h \in H \mid (\exists s \in S)(\exists g \in G)(g \geq_g h \geq_g s)\}$$

Este teorema nos dice que cualquier hipótesis h pertenece al Espacio de Versiones (VS) si y solo si está atrapado entre alguna $s \in S$ y $g \in G$. Esto significa que S y G son suficientes para reconstruir todo el conjunto de soluciones posibles sin gastar memoria extra.

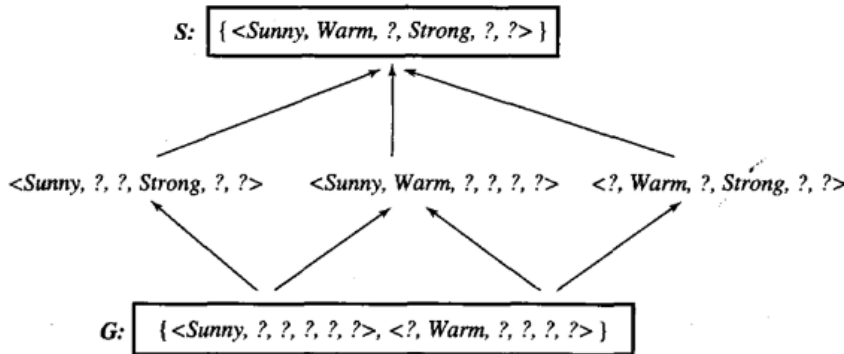


FIGURE 2.3

A version space with its general and specific boundary sets. The version space includes all six hypotheses shown here, but can be represented more simply by S and G . Arrows indicate instances of the *more-general-than* relation. This is the version space for the *EnjoySport* concept learning problem and training examples described in Table 2.1.

3.2.5.4 El Algoritmo Candidate-Elimination.

El objetivo de este algoritmo es encontrar el **Espacio de Versiones** completo. Para hacerlo de forma eficiente, mantiene dos inventarios de hipótesis en todo momento:

1. **El conjunto S (Specific):** Contiene las hipótesis más específicas que explican los datos. Empezando siendo 'nada es posible' ($\emptyset$).

2. El conjunto G (General): Contiene las hipótesis más generales que explican los datos. Empieza siendo 'todo es posible' (?).

A medida que ve ejemplos, S se vuelve más general y G se vuelve más específico. El espacio entre ambos es nuestra incertidumbre. Si se tocan, hemos encontrado la respuesta única.

Funcionamiento del algoritmo.

- Si el ejemplo es POSITIVO ($d = +$):

- **En G :** Eliminamos cualquier hipótesis que sea tan específica que no cubra a d (porque las generales deben cubrir todos los positivos).
- **En S :** Si una hipótesis s no cubre a d , la generalizamos mínimamente para que lo cubra, siempre y cuando esa nueva versión siga siendo más específica que algún elemento en G .

- Si el ejemplo es NEGATIVO ($d = -$):

- **En S :** Eliminamos cualquier hipótesis que sea tan general que sí cubra a d (porque las específicas no deben cubrir negativos).
- **En G :** Si una hipótesis g cubre a d (lo cual es un error), la 'cerramos' (especializamos) mínimamente para que deje de cubrirlo, siempre y cuando la nueva versión siga siendo más general que algún elemento en S .

Implementación en Python.

Para implementar esto necesitamos manejar la noción de 'dominios' (qué valores puede tomar cada atributo) para poder realizar las especializaciones mínimas.

```
1  # Definimos los valores posibles para cada atributo (EnjoySport)
2  DOMAINS = [
3      ['Sunny', 'Cloudy', 'Rainy'],
4      ['Warm', 'Cold'],
5      ['Normal', 'High'],
6      ['Strong', 'Weak'],
7      ['Warm', 'Cool'],
8      ['Same', 'Change']
9  ]
10
11  # Evalua si la hipótesis h clasifica correctamente la instancia x según su label.
12  def is_consistent(h, x, label):
13      prediction = all(h[i] == '?' or h[i] == x[i] for i in range(len(h)))
14
15      if label == "Yes":
16          return prediction == True
17      else:
18          return prediction == False
19
20  # Retorna True si la hipótesis h cubre (es verdadera para) la instancia x.
21  def fulfills(h, x):
22      return all(h[i] == '?' or h[i] == x[i] for i in range(len(h)))
23
24  # Retorna True si h1 es más general o igual que h2.
25  def is_more_general(h1, h2):
26      for i in range(len(h1)):
27          if h1[i] != '?' and h1[i] != h2[i]:
28              return False
```

```

29
30     return True
31
32 def candidate_elimination(training_data):
33     num_attrs = len(DOMAINS)
34
35     # Inicializacion: G es lo mas general (?), S lo mas especifico (0)
36     G = [['?'] * num_attrs]
37     S = [['0'] * num_attrs]
38
39     for x, label in training_data:
40         # Caso Positivo: S debe generalizarse y G debe ser consistente.
41         if label == "Yes":
42             # Eliminar de G cualquier hipotesis que no cubra el ejemplo positivo.
43             G = [g for g in G if fulfills(g, x)]
44
45             # Generalizar S minimamente para que cubra el ejemplo x
46             new_S = []
47             for s in S:
48                 if not fulfills(s, x):
49                     for i in range(num_attrs):
50                         # Si el atributo es 0, toma el valor del ejemplo
51                         if s[i] == '0':
52                             new_h = list(s); new_h[i] = x[i]
53                             # Solo mantenemos h si es consistente con la frontera general
54                             if any(is_more_general(g, new_h) for g in G):
55                                 new_S.append(new_h)
56
57                             # Si el atributo difiere, se generaliza a ?
58                             elif s[i] != x[i]:
59                                 new_h = list(s); new_h[i] = '?'
60                                 if any(is_more_general(g, new_h) for g in G):
61                                     new_S.append(new_h)
62
63             else:
64                 new_S.append(s)
65
66             # Poda de S: eliminar hipotesis que sean mas generales que otras en S
67             # Nos quedamos solo con las máximamente especificas
68             S = [s for s in new_S if not any(is_more_general(s, s2) and s != s2 for s2
69                 in new_S)]
69
70     # Label == "No"
71     else:
72         # Caso Negavito: G debe especializarse y S debe ser consistente.
73
74         # Eliminar de S cualquier hipotesis que cubra erroneamente el ejemplo negativo
75         S = [s for s in S if not fulfills(s, x)]
76
77         # Especializar G minimamente para que deje de cubrir el ejemplo x
78         new_G = []
79         for g in G:
80             if fulfills(g, x):
81                 for i in range(num_attrs):

```

```

82         # Si el atributo es ?, probamos valores del dominio que no
           coinciden con x
83     if g[i] == '?':
84         for val in DOMAINS[i]:
85             if val != x[i]:
86                 new_h = list(g); new_h[i] = val
87
88                 # Solo mantenemos h si es consistente con la
                   frontera especifica
89             if any(is_more_general(new_h, s) for s in S):
90                 new_G.append(new_h)
91
92     else:
93         new_G.append(g)
94
95     # Poda de G: eliminar hipótesis que sean menos generales (mas especificas)
           que otras en G
96     G = [g for g in new_G if not any(is_more_general(g2, g) and g != g2 for g2
           in new_G)]
97
98     return S, G

```

3.2.5.5 Ejemplo algoritmo Candidate-Elimination.

El algoritmo comienza definiendo el espacio de búsqueda completo a través de sus límites iniciales:

- G_0 (**Frontera General**): la hipótesis más general posible, que acepta cualquier día como positivo: $\{(? , ? , ? , ? , ? , ?)\}$.
- S_0 (**Frontera Específica**): la hipótesis más específica, que no acepta ningún día: $\{(\emptyset , \emptyset , \emptyset , \emptyset , \emptyset , \emptyset)\}$.

Cuando se presentan los primeros dos ejemplos de entrenamiento (ambos positivos), el efecto es similar al algoritmo **Find-S**:

1. Ejemplo (Sunny, Warm, Normal, Strong, Same), EnjoySport = Yes.

- S_1 se revisa para cubrir el ejemplo. La nueva frontera es $\{(Sunny, Warm, Normal, Strong, Warm, Same)\}$.
- G_1 no cambia, ya que G_0 cubre correctamente este ejemplo positivo.

2. Ejemplo (Sunny, Warm, High, Strong, Same), EnjoySport = Yes.

- S_2 se generaliza para incluir la variación en el atributo Humedad (Normal vs High), resultando en $\{(Sunny, Warm, ?, Strong, Warm, Same)\}$.
- G_2 permanece igual que G_0 .

3. Ejemplo (Rainy, Cold, High, Strong, Warm, Change), EnjoySport = No.

- S_3 no cambia, ya que no cubre el ejemplo negativo.
- G_3 : se revela que G_2 es demasiado general al predecir erróneamente este ejemplo como positivo. G_2 se especializa en las versiones mínimamente más específicas que excluyan este día:

$$G_3 = \{(Sunny, ?, ?, ?, ?, ?), (?, Warm, ?, ?, ?, ?), (?, ?, ?, ?, ?, Same)\}$$

- Hipótesis como $(?, ?, Normal, ?, ?, ?)$ se excluyen porque no son más generales que la frontera específica actual S_2

4. Ejemplo (Sunny, Warm, High, Strong, Cool, Change), EnjoySport = Yes.

- S_4 : se generaliza S_3 para cubrir la variación en Agua (Warm vs Cool) y Pronóstico (Same vs Change), quedando en:

$$S_4 = \{(Sunny, Warm, ?, Strong, ?, ?)\}$$

- G_4 : se debe eliminar cualquier miembro de G_3 que no cubra este nuevo ejemplo positivo. En nuestro caso, solo la hipótesis $(?, ?, ?, ?, ?, Same)$ no cubre el ejemplo, la eliminamos y resulta:

$$G_4 = \{(Sunny, ?, ?, ?, ?, ?), (?, Warm, ?, ?, ?, ?)\}$$

El algoritmo termina devolviendo S_4 y G_4 como las fronteras de hipótesis específica y general. Cualquier hipótesis entre estos dos límites en el ordenamiento parcial forma parte del espacio de versiones. A medida que se observen más datos, estas fronteras convergerán gradualmente hasta delimitar, idealmente, una única hipótesis.

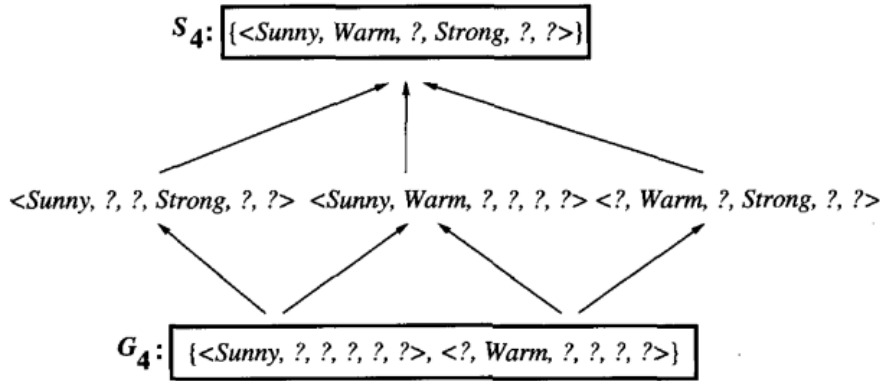


FIGURE 2.7

The final version space for the *EnjoySport* concept learning problem and training examples described earlier.

3.2.6 Observaciones sobre espacios de versiones.

3.2.6.1 ¿Convergerá el algoritmo a la hipótesis correcta?

El algoritmo **Candidate-Elimination** convergerá hacia la hipótesis que describe correctamente el concepto objetivo siempre que se cumplan dos condiciones:

- **Ausencia de errores:** no debe haber errores en los ejemplos de entrenamiento.
- **Disponibilidad en H:** debe existir alguna hipótesis en el espacio H que describa correctamente el concepto objetivo.

¿Qué pasa si hay errores o el concepto no está en H ?

El aprendizaje se completa exactamente cuando los conjuntos frontera S y G convergen a una única e idéntica hipótesis.

- Si se presenta un ejemplo erróneo (un falso negativo, por ejemplo), el algoritmo eliminará la hipótesis correcta del espacio de versiones.

- Eventualmente, el espacio de versiones se volverá vacío (cuando S y G se crucen y no quede nada), lo cual es una señal de que los datos son inconsistentes o que la representación elegida es insuficiente (por ejemplo, intentar aprender una disyunción cuando H solo permite conjunciones).

3.2.6.2 Aprendizaje activo: el poder de preguntar.

En lugar de esperar a que un tutor le entregue ejemplos al azar, el sistema puede realizar consultas (queries) eligiendo activamente qué instancia quiere clasificar. El objetivo fundamental de esta estrategia es discriminar entre las hipótesis que aún compiten dentro del espacio de versiones. Para lograr la máxima eficiencia, el aprendiz debe buscar una instancia que sea clasificada como positiva por exactamente la mitad de las hipótesis y como negativa por la otra mitad.

Imagina que después de los cuatro ejemplos de EnjoySport, el espacio de versiones tiene 6 hipótesis posibles (como se vio en las tablas anteriores). Si se propone la instancia (**Sunny, Warm, Normal, Light, Warm, Same**), se notará algo interesante:

- Tres de las seis hipótesis dirán que Aldo sí disfruta el deporte.
- Las otras tres dirán que no lo disfruta.

Independientemente de la respuesta que dé el 'oráculo' (la naturaleza o un profesor), habrás eliminado exactamente la mitad de tus sospechosos. Esta estrategia es idéntica al juego de las 20 preguntas, donde la mejor jugada es siempre aquella que divide las posibilidades por la mitad.

Matemáticamente, esto garantiza que encontrarás la respuesta correcta en un número de pasos igual a $\lceil \log_2 |VS| \rceil$. Si no puedes dividir el espacio exactamente a la mitad, el aprendizaje podría requerir algunas consultas adicionales.

3.2.6.3 Cómo utilizar los conceptos aprendidos parcialmente.

Incluso si el espacio de versiones aún contiene múltiples hipótesis y no hemos llegado a una respuesta única, el sistema ya es capaz de clasificar nuevas instancias con una confianza absoluta en ciertos casos. Esto ocurre cuando existe un **consenso unánime** entre todas las hipótesis que quedan en el espacio de versiones. Por ejemplo, si una nueva instancia es clasificada como positiva por cada una de las hipótesis restantes, podemos estar tan seguros de esa predicción como si ya hubiéramos terminado el entrenamiento.

Para verificar esta unanimidad de forma eficiente sin tener que revisar cada hipótesis una por una, basta con mirar las fronteras:

- Una instancia se clasifica como **positiva con total seguridad** si satisface a todos los miembros de la frontera específica S . Esto es así porque cualquier otra hipótesis en el espacio de versiones es, por definición, igual o más general que algún miembro de S ; por lo tanto, si la más específica acepta el ejemplo, las más generales también lo harán.
- De manera simétrica, podemos clasificar una instancia como **negativa con total seguridad** si no satisface a ninguno de los miembros de la frontera general G . Si las hipótesis más amplias y permisivas ya rechazan el ejemplo, sabemos que las versiones más específicas y restrictivas que viven 'debajo' de ellas en el ordenamiento parcial también lo rechazarán.

¿Y qué ocurre cuando no hay consenso? Si nos topamos con una instancia que algunas hipótesis clasifican como positiva y otras como negativa, estamos ante una ambigüedad. Estas instancias son, curiosamente, los candidatos ideales para convertirlos en una consulta y pedirle la respuesta al profesor, ya que son los que más información nueva aportan.

En situaciones donde no podemos pedir más datos y estamos obligados a decidir, una solución es aplicar una **votación por mayoría**. Si asumimos que todas las hipótesis son igual de probables, la proporción de votos a favor puede interpretarse directamente como la probabilidad de que la instancia sea positiva.

3.2.7 El sesgo inductivo (Inductive Bias).

El algoritmo **Candidate-Elimination** converge al concepto objetivo si este se encuentra en el espacio de hipótesis H y los datos son precisos. Sin embargo, surgen preguntas críticas: ¿Qué paso si el concepto no está en H ? ¿Podemos simplemente usar un espacio que incluya todas las hipótesis posibles? Como veremos, el tamaño y la forma de este espacio determinan tanto la capacidad de generalizar como el número de ejemplos necesarios.

3.2.7.1 Un espacio de hipótesis sesgado.

Supongamos que deseamos asegurar que el espacio de hipótesis contenga el concepto objetivo desconocido. La solución obvia es enriquecer el espacio de hipótesis para incluir cada hipótesis posible.

Para ilustrarlo, considere nuevamente el ejemplo de EnjoySport en el cual restringimos el espacio de hipótesis para incluir solo conjunciones de valores de atributos. Debido a esta restricción, el espacio de hipótesis es incapaz de representar incluso conceptos objetivo disyuntivos simples tales como 'Cielo = Soleado o Cielo = Nublado'. De hecho, dados los siguientes tres ejemplos de entrenamiento de esta hipótesis disyuntiva, nuestro algoritmo encontraría que hay cero hipótesis en el espacio de versiones.

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Cool	Change	Yes
2	Cloudy	Warm	Normal	Strong	Cool	Change	Yes
3	Rainy	Warm	Normal	Strong	Cool	Change	No

Para ver por qué no hay hipótesis consistentes con estos tres ejemplos, observe que la hipótesis más específica consistente con los dos primeros ejemplos y representable en el espacio de hipótesis dado H es:

$$S_2 : (?, \text{Warm}, \text{Normal}, \text{Strong}, \text{Cool}, \text{Change})$$

Esta hipótesis, aunque es la hipótesis máximamente específica de H que es consistente con los dos primeros ejemplos, ya es excesivamente general: cubre incorrectamente el tercer ejemplo de entrenamiento (negativo). El problema es que hemos sesgado al aprendiz para que considere solo hipótesis conjuntivas. En este caso, requerimos un espacio de hipótesis más expresivo.

3.2.7.2 Un aprendiz sin sesgos (Unbiased Learner).

Para evitar que el concepto objetivo quede fuera de H , la solución obvia sería usar un espacio que pueda representar **cualquier subconjunto** de las instancias X ; esto se conoce como el **conjunto potencia** (power set) de X .

- **Escala del problema:** En EnjoySport, hay 96 instancias posibles basándose en sus atributos.
- **Tamaño del Espacio:** El conjunto potencia de X contiene $2^{|X|}$ hipótesis. Para este caso, existen 2^{96} conceptos posibles (aprox. 7.9×10^{28}).

- **Comparación:** Nuestro espacio original de conjunciones solo representaba 973 de esos miles de trillones de conceptos; era, de hecho, un espacio extremadamente sesgado.

Aunque un espacio sin sesgos (como permitir disyunciones, conjunciones y negaciones arbitrarias) asegura que el concepto objetivo sea expresable, destruye la capacidad del algoritmo para generalizar más allá de los ejemplos observados:

- **Frontera S:** Se convertiría simplemente en la disyunción de los ejemplos positivos observados (ej. $x_1 \vee x_2 \vee x_3$).
- **Frontera G:** Sería simplemente la negación de la disyunción de los ejemplos negativos (ej. $\neg(x_4 \vee x_5)$).

Bajo estas condiciones, los únicos ejemplos que S y G clasificarán de forma unánime son los que ya han sido vistos durante el entrenamiento. Para identificar un solo concepto objetivo, ¡tendríamos que observar cada una de las instancias del espacio X como ejemplo de entrenamiento!

Incluso si intentáramos usar el sistema de votación, el resultado sería inútil. Para cualquier instancia no observada, exactamente la mitad de las hipótesis en el espacio de versiones votarán que es positiva y la otra mitad que es negativa.

Esto sucede porque, en un conjunto potencia, por cada hipótesis h que cubre una instancia nueva x , existe otra hipótesis h' idéntica a h en todo lo demás pero que clasifica a x de forma opuesta. Si h es consistente con los datos pasados, h' también lo será, manteniendo la ambigüedad perfecta para siempre.

3.2.7.3 La futilidad del aprendizaje sin sesgos.

Un aprendiz que no realiza suposiciones previas sobre la identidad del concepto objetivo no tiene una base racional para clasificar instancias nunca antes vistas. En el caso del algoritmo **CANDIDATE-ELIMINATION** aplicado a EnjoySport, la capacidad de generalizar dependió enteramente del supuesto implícito de que el concepto objetivo podía representarse como una conjunción de atributos. Si este supuesto es correcto, el sistema clasificará bien las nuevas instancias; si es incorrecto, el algoritmo necesariamente cometerá errores.

Definición formal de Sesgo Inductivo.

Para describir cómo un algoritmo L generaliza más allá de los datos de entrenamiento D_c para clasificar una nueva instancia x_i , Mitchell define el sesgo inductivo como el conjunto de supuestos adicionales necesarios para que la inferencia inductiva se convierta en una **inferencia deductiva**.

Formalmente, el sesgo inductivo de un algoritmo L es el conjunto mínimo de aserciones B tal que, para cualquier concepto objetivo c y sus ejemplos de entrenamiento D_c :

$$(\forall x_i \in X)[(B \wedge D_c \wedge x_i) \vdash L(x_i, D_c)]$$

Aquí, el símbolo $\vdash$ indica que la clasificación $L(x_i, D_c)$ se sigue deductivamente (es demostrable) a partir de los supuestos B , los datos D_c y la descripción de la instancia x_i .

El sesgo de Candidate-Elimination.

El sesgo inductivo de este algoritmo es simplemente el supuesto de que el concepto objetivo c está contenido en el espacio de hipótesis H ($c \in H$).

Bajo este supuesto, cada clasificación realizada por el algoritmo puede justificarse deductivamente:

- Si asumimos $c \in H$, entonces por definición c debe estar en el Espacio de Versiones ($c \in VS_{H, D_c}$), ya que c es consistente con sus propios ejemplos de entrenamiento.

- Dado que el algoritmo solo clasifica una instancia si hay un voto unánime de todas las hipótesis en el Espacio de Versiones, si el algoritmo emite una clasificación, esta debe coincidir necesariamente con la del concepto objetivo c .

Por lo tanto, el comportamiento de CANDIDATE-ELIMINATION es equivalente al de un demostrador de teoremas deductivo que tiene como axioma adicional la frase: 'el espacio H contiene el concepto objetivo'.

Comparación de sistemas según su fuerza de sesgo.

Mitchell clasifica los algoritmos de aprendizaje según qué tan fuertes son sus sesgos inductivos (cuántos 'saltos de fe' realizan):

- **ROTE-LEARNER (Sin sesgo):** Solo almacena ejemplos en memoria. Clasifica nuevas instancias únicamente si ya fueron vistas exactamente igual en el entrenamiento. No tiene sesgo inductivo porque no realiza suposiciones para generalizar.
- **CANDIDATE-ELIMINATION:** Tiene un sesgo moderado ($c \in H$). Clasifica algunas instancias nuevas que el ROTE-LEARNER ignoraría, siempre que haya consenso en el Espacio de Versiones.
- **FIND-S:** Posee un sesgo aún más fuerte. Además de asumir que $c \in H$, asume que todas las instancias son negativas a menos que se demuestre lo contrario (una preferencia por la hipótesis más específica posible). Clasifica una mayor proporción de instancias invisibles que los anteriores.

4 Machine Learning.

4.1 Árboles de decisión.

El aprendizaje de **árboles de decisión** es uno de los métodos más exitosos para la **inferencia inductiva**, el proceso de derivar reglas generales a partir de ejemplos específicos. Su relevancia en la IA moderna no solo radica en su precisión, sino en su transparencia: a diferencia de las 'cajas negras' de las redes neuronales, un árbol de decisión es un modelo de 'caja blanca' cuya lógica es completamente auditable.

- **Poder de Expresión:** Estos algoritmos buscan en un **espacio de hipótesis completamente expresivo**. Esto significa que no están limitados por una estructura lineal; pueden representar cualquier función finita de valores discretos.
- **Robustez Operativa:** Son naturalmente resistentes al ruido (datos mal etiquetados) y eficientes al manejar atributos redundantes, lo que los hace ideales para problemas de clasificación en entornos reales como finanzas y medicina.

4.1.1 Representación del conocimiento: la lógica del árbol.

Desde el punto de vista de la lógica computacional, un árbol de decisión es una jerarquía de if-then y una representación compacta de **Álgebra de Boole**.

- **Nodos de decisión (internos).** Representan una variable o atributo que se somete a una prueba (test).
- **Aristas (ramas).** Representan los posibles resultados o valores del test realizado en el nodo superior.
- **Hojas (nodos terminales).** Representan la categoría final o el valor predicho.

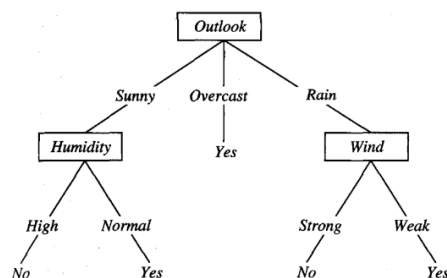
Estructura lógica.

Cada árbol de decisión puede traducirse a una **disyunción de conjunciones**:

- Cada camino desde la raíz hasta una hoja positiva representa una **conjunción (AND)** de pruebas de atributos.
- El árbol completo representa la **disyunción (OR)** de todos esos caminos que llevan a una clasificación positiva.

Ejemplo.

Supongamos que queremos predecir si el día es apto para jugar al tenis basándonos en el clima. El árbol ha aprendido la siguiente lógica:



La lógica del árbol parte de la raíz y evalúa Outlook:

- Si es Overcast $\rightarrow$ Si (Hoja).
- Si es Sunny $\rightarrow$ Evaluar Humidity.
- Si es Rain $\rightarrow$ Evaluar Wind.

Si recibimos una instancia con los siguientes atributos:

$\{\text{Outlook} = \text{Sunny}, \text{Temperatura} = \text{Hot}, \text{Humidity} = \text{High}, \text{Wind} = \text{Strong}\}$

El recorrido en el árbol sería:

- Nodo raíz (Outlook): el valor es Sunny. Seguimos hacia la rama izquierda.
- Nodo interno (Humidity): el valor es High. Seguimos hacia la rama correspondiente.
- Hoja final: la clasificación es No.

Expresión lógica del modelo.

El conocimiento del árbol se puede formalizar matemáticamente como:

$(\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{Normal}) \vee (\text{Outlook} = \text{Overcast}) \vee (\text{Outlook} = \text{Rain} \wedge \text{Wind} = \text{Weak})$

Observaciones.

- Si una instancia dada cumple alguna fórmula dentro de la disyunción general, entonces valdrá la fórmula entera y por lo tanto el test daría afirmativo.
- El atributo Temperature no aparece en la expresión lógica final. Esto es porque el algoritmo determinó (mediante la entropía) que la temperatura no aportaba **ganancia de información** relevante una vez que ya conocíamos el Outlook y Humidity.

4.1.2 Problemas ideales para árboles de decisión.

No todos los problemas de Machine Learning se resuelven bien con árboles. Mitchell identifica un 'escenario ideal' donde este modelo brilla por encima de redes neuronales o modelos lineales:

- **Estructura Atributo-Valor:** las instancias deben ser describibles mediante un vector de características fijas. El escenario óptimo es el de **atributos discretos** (categorías), ya que el árbol simplemente genera una rama por cada valor.
- **Funciones Objetivo Discretas:** el modelo estándar genera una clasificación (clases disjuntas). Si bien existen 'árboles de regresión' para valores continuos, el diseño original de Mitchell apunta a problemas de clasificación.
- **Necesidad de expresiones disyuntivas:** a diferencia de un clasificador lineal (que solo puede separar datos con una línea/hiperplano), los árboles pueden aprender conceptos complejos como 'Si A y B son ciertos, o si C es cierto'.
- **Resiliencia al ruido y datos incompletos:** son extremadamente robustos. Pueden manejar errores de etiquetado, errores de medición y missing values.

4.1.3 El algoritmo ID3.

El **ID3 (Interactive Dichotomizer 3)** es un algoritmo fundacional para la construcción de árboles de decisión. Su objetivo es transformar un conjunto de ejemplos desordenados en una estructura

jerárquica de reglas if-then. En lugar de ajustar pesos numéricos, el ID3 realiza un **particionamiento recursivo** del espacio de los datos para extraer conocimiento explícito y legible para humanos.

El ID3 construye el árbol de forma dinámica para resolver problemas de clasificación. Su tarea principal es **minimizar la incertidumbre** en cada nivel.

- Toma una tabla de datos y decide qué columna (atributo) es la que mejor separa las clases (ej. 'Aptos para crédito' vs. 'No aptos').
- Genera una pregunta (nodo) y divide el dataset original en subgrupos más puros.
- Repite este proceso en cada subgrupo hasta que ya no hay ambigüedad o se agotan las preguntas disponibles.

4.1.3.1 Estrategia de búsqueda: Top-Down y Greedy.

El algoritmo emplea una búsqueda heurística en el espacio de todos los árboles posibles:

- **Top-Down (de arriba hacia abajo).** Empieza por la raíz y va expandiendo ramas hacia las hojas. No construye el árbol de forma global, sino por niveles.
- **Greedy (voraz).** En cada nodo, elige el atributo que maximiza la **Ganancia de Información** en ese instante. Es una decisión localmente óptima; el algoritmo no realiza backtracking (no vuelve atrás para corregir una elección previa si el árbol se vuelve demasiado complejo).

4.1.3.2 Lógica algorítmica y casos base.

El ID3 funciona mediante recursión. Para cada llamada, evalúa el estado actual de los datos (Examples) y los atributos disponibles:

- **Pureza total:** si todos los ejemplos son de la misma clase, se detiene y crea una hoja con esa etiqueta.
- **Agotamiento de atributos:** si ya no quedan atributos para preguntar pero los datos siguen mezclados, crea una hoja con la clase mayoritaria (voto por mayoría).
- **Paso de partición:** calcula que atributo separa mejor los datos (usando ganancia de información). Crea un nodo con ese atributo y para cada valor de ese atributo, crea una rama y llama a ID3 recursivamente con el subconjunto de datos correspondiente.

4.1.3.3 Implementación en Python.

```
1 import math
2 from collections import Counter
3
4 """
5 Calcula la entropía de un conjunto de etiquetas.
6 - labels: Una LISTA de strings o valores (ej: ['Yes', 'No', 'Yes']). Representa la
7   columna del 'Targetattribute' (PlayTennis) para un subconjunto de datos.
8 - return: Un float que representa el desorden en bits (0.0 a 1.0 para binario).
9 """
10 def entropy(labels):
11     n = len(labels)
12     counts = Counter(labels)
13     return -sum((count / n) * math.log2(count / n) for count in counts.values())
```

```

14 """
15 Calcula la ganancia de información de un atributo específico.
16 - data: Una LISTA DE DICCIONARIOS. Cada diccionario es una fila/instancia (ej: [{'
    Outlook': 'Sunny', 'Play': 'No'}, {...}]).
17 - attr: Un STRING. Es el nombre de la columna que queremos evaluar (ej: 'Outlook')
    .
18 - target: Un STRING. Es el nombre de la columna que queremos predecir (el '
    Targetattribute', ej: 'PlayTennis').
19 - return: Un float con la reducción de entropía lograda.
20 """
21 def info_gain(data, attr, target):
22     # Calculamos la entropía original (antes de la división)
23     total_entropy = entropy([row[target] for row in data])
24
25     # Obtenemos los valores posibles del atributo (ej: Sunny, Overcast, Rain)
26     values = set(row[attr] for row in data)
27
28     # Calculamos la entropía pesada de los subconjuntos resultantes
29     subset_entropy = 0
30     for val in values:
31         # Filtramos las filas que tienen este valor específico
32         subset = [row for row in data if row[attr] == val]
33
34         # Calculamos el peso (proporcion del subconjunto respecto al total)
35         prob = len(subset) / len(data)
36
37         # Sumamos la entropía de este subconjunto multiplicada por su probabilidad
38         subset_entropy += prob * entropy([row[target] for row in subset])
39
40     # La ganancia es la diferencia: lo que sabemos menos lo que aun no sabemos
41     return total_entropy - subset_entropy
42
43 """
44 Construye el árbol de decisión usando la lógica recursiva de Mitchell.
45 - examples: La LISTA DE DICCIONARIOS con todos los datos de entrenamiento actuales
    . A medida que la recursión baja, esta lista es más pequeña.
46 - target: El STRING con el nombre de la columna a predecir (la etiqueta).
47 - attributes: Una LISTA DE STRINGS con los nombres de las columnas disponibles
    para testear (ej: ['Outlook', 'Temperature', 'Humidity', 'Wind']).
48 - return: Un arbol (puede ser un STRING si es una hoja, o un DICCIONARIO si es un
    nodo).
49 """
50 def id3(examples, target, attributes):
51     # Extraemos todas las etiquetas del conjunto actual para evaluar los casos
    base
52     labels = [row[target] for row in examples]
53
54     # Caso Base 1: todos los ejemplos pertenecen a la misma clase
55     if len(set(labels)) == 1:
56         return labels[0]
57
58     # Caso Base 2: no quedan atributos para preguntar.
59     # (Attributes es una lista vacía)
60     # Retornamos el valor mas comun del TargetAttribute con los ejemplos actuales
61     if not attributes:
62         return Counter(labels).most_common(1)[0][0]

```

```

63
64     # Paso Voraz: seleccionamos el nombre del atributo con mayor ganancia de
        informacion
65     best_attr = max(attributes, key=lambda a: info_gain(examples, a, target))
66
67     # Creamos la estructura del nodo raiz para este subarbol
68     tree = {best_attr: {}}
69
70     # Creamos una copia de la lista de atributos eliminando el que acabamos de
        usar
71     # para no repetir la misma pregunta en niveles inferiores del árbol
72     remaining_attrs = [a for a in attributes if a != best_attr]
73
74     ...

```

4.1.4 Entropía: la medida del desorden (H).

En el contexto de Machine Learning, la **entropía** cuantifica el grado de **incertidumbre** o 'mezcla' de las clases en un conjunto de datos.

Para un conjunto S con respecto a una clasificación binaria (positiva y negativa), se define matemáticamente la entropía como:

$$Entropy(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

- p_+ es la proporción de ejemplos positivos en el conjunto.
- p_- es la proporción de ejemplos negativos en el conjunto.

El significado de los valores.

- **Entropía = 0.** El conjunto es perfectamente puro. Todos los ejemplos pertenecen a la misma clase. No hay incertidumbre.
- **Entropía = 1.** El desorden es máximo. Hay un 50% de probabilidades para cada clase. Es el estado de mayor incertidumbre.
- **Curva de entropía.** Es una parábola invertida que alcanza su pico en 0.5 y baja a 0 en los extremos (0 y 1).

Ejemplo práctico.

Imagina que tenemos un conjunto S de 14 ejemplos sobre si jugar al tenis:

- 9 ejemplos son Si ($p_+ = 9/14$).
- 5 ejemplos son No ($p_- = 5/14$).

Tenemos la siguiente entropía para el conjunto S . Notar que como el valor está cerca de 1, el conjunto está bastante mezclado. Necesitamos casi 1 bit de información para predecir el resultado de un ejemplo aleatorio.

$$Entropy(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) \approx 0.94$$

4.1.5 Ganancia de información: el poder de un atributo (Gain).

La **ganancia de información** es la métrica que decide qué nodo va en el árbol. Mide la **reducción esperada en la entropía** que se logra al dividir el conjunto de datos según un atributo específico.

Matemáticamente, la ganancia del atributo A respecto al conjunto S se define como:

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

- El primer término es la entropía original, antes de la división.
- El segundo término es la suma pesada de la entropía de los nuevos subconjuntos creados por el atributo A .

Ejemplo de aplicación: eligiendo la raíz.

Supongamos que el algoritmo debe elegir entre dos atributos para el nodo raíz: **Humedad** y **Viento**. Partimos de nuestra entropía inicial de 0.940.

1. Evaluación de Humedad (Valores: Alta, Normal).

- **Humedad Alta:** de los 7 ejemplos con humedad alta, 3 son Si y 4 son No.

$$Entropy(S_{alta}) = -(3/7)\log_2(3/7) - (4/7)\log_2(4/7) = 0.985$$

- **Humedad Normal:** de los 7 ejemplos con humedad normal, 6 son Si y 1 es No.

$$Entropy(S_{normal}) = 0.592$$

- **Cálculo de ganancia.**

$$Gain(S, Humedad) = 0.940 - [(7/14) \cdot 0.985 + (7/14) \cdot 0.592] = 0.151$$

2. Evaluación de Viento (Valores: Fuerte, Débil).

- **Viento Débil:** de los 8 ejemplos con viento débil, 6 son Si y 2 son No.

$$Entropy(S_{debil}) = 0.811$$

- **Viento Fuerte:** de los 6 ejemplos con viento fuerte, 3 son Si y 3 son No.

$$Entropy(S_{fuerte}) = 1.00$$

- **Cálculo de ganancia.**

$$Gain(S, Viento) = 0.940 - [(8/14) \cdot 0.811 + (6/14) \cdot 1.00] = 0.048$$

Como $Gain(S, Humedad) > Gain(S, Viento)$, el algoritmo ID3 elige la Humedad como el mejor atributo para dividir los datos en ese punto. Esto es porque la humedad nos acerca más a la pureza (entropía 0) que saber el viento.

4.2 Evaluación de hipótesis.

Bibliografía: Mitchel - Machine Learning - Unidad 5 (140-162).

La **evaluación empírica de la precisión de las hipótesis** es fundamental en el aprendizaje automático. Este capítulo presenta una introducción a los métodos estadísticos para estimar la precisión de las hipótesis, centrándose en tres preguntas:

1. Dada la precisión observada de una hipótesis sobre una muestra limitada de datos, ¿qué tan bien estima esto su precisión sobre ejemplos adicionales?
2. Dado que una hipótesis supera a otra sobre alguna muestra de datos, ¿qué tan probable es que esta sea más precisa en general?
3. Cuando los datos son limitados, ¿cuál es la mejor manera de utilizarlos tanto para aprender una hipótesis como para estimar su precisión?

Debido a que las muestras limitadas de datos pueden representar erróneamente la distribución general, estimar la precisión real a partir de ellas puede ser engañoso. Los métodos estadísticos, junto con supuestos sobre las distribuciones subyacentes, permiten acotar la diferencia entre la precisión observada en la muestra y la precisión real sobre toda la distribución de los datos.

4.2.1 Motivación.

En muchos casos, es crucial evaluar el rendimiento de las hipótesis aprendidas con la mayor precisión posible. Una razón es simplemente entender si se debe utilizar la hipótesis. Por ejemplo, al aprender de una base de datos de tamaño limitado sobre la efectividad de diferentes tratamientos médicos, es vital conocer la precisión de las hipótesis.

Una segunda razón es que la evaluación es un componente integral de muchos métodos de aprendizaje. Por ejemplo, en la post-poda (post-pruning) de árboles de decisión para evitar el overfitting (sobreajuste), debemos evaluar el impacto de los pasos de poda en la precisión del árbol resultante. Por lo tanto, es importante comprender los errores inherentes al estimar la precisión del árbol podado y del no podado.

Estimar la precisión es sencillo cuando los datos abundan. Sin embargo, cuando los datos son limitados, surgen dos dificultades:

- **Sesgo en la estimación:** La precisión observada sobre los ejemplos de entrenamiento suele ser un mal estimador de la precisión futura. Como la hipótesis se derivó de esos mismos ejemplos, estos suelen proporcionar una estimación **optimistamente sesgada**. Esto es muy común cuando el aprendiz considera un espacio de hipótesis muy rico, permitiéndole sobreajustar (overfit) los datos. Para obtener una estimación imparcial, probamos la hipótesis en un conjunto de prueba elegido de forma independiente.
- **Varianza en la estimación:** La varianza representa la variabilidad del $error_S(h)$ si repitiéramos el experimento múltiples veces con diferentes muestras aleatorias del mismo tamaño n . Su importancia radica en que funciona como un indicador de **fiabilidad**: una varianza alta significa que el error observado podría ser simplemente una casualidad estadística (por ejemplo, haber elegido justo los casos más fáciles o los más difíciles por azar).

Ejemplo: si probamos una hipótesis con solo 10 personas ($n = 10$) y falla en 1, el error $error_S(h)$ es del 10%. Pero si justo esa persona era un outlier, la estimación tiene una varianza altísima. Si luego se prueba con otras 10 personas, quizás el error salta al 30%.

En cambio si probamos con 1000 personas ($n = 1000$), el peso de un solo individuo atípico se diluye. La varianza cae y el $error_S(h)$ se vuelve mucho más estable y cercano al $error_D(h)$ (el

error real en el mundo).

Como regla general: a menor n , mayor incertidumbre sobre si lo que medimos es verdad o suerte.

4.2.2 Estimación de la precisión de la hipótesis.

Al evaluar una hipótesis, nos interesa estimar la precisión con la que clasificará instancias futuras y conocer el error probable en dicha estimación (es decir, qué barras de error asociar).

Consideramos el siguiente entorno: existe un espacio de instancias X sobre el cual se definen funciones objetivo f . Asumimos que existe una distribución de probabilidad desconocida D que define la frecuencia con la que se encuentra cada instancia en X . El objetivo es aprender f considerando un espacio H de hipótesis. Los ejemplos de entrenamiento se obtienen extrayendo cada instancia de forma independiente según D .

Dentro de este entorno, nos interesan dos preguntas: